

Dispense di Architetture degli Elaboratori

Marco Galletti

A.A. 2025/2026

Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Architetture degli Elaboratori con il professore Salvatore Pontarelli del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

1	Introduzione	2
1.1	Instruction Set Architecture	2
2	Rappresentazione delle istruzioni	6
3	Aritmetica dei computer e rappresentazioni numeriche	13
3.1	Addizione floating point	18
3.2	Moltiplicazione floating point	20
4	Progettazione di un processore	24
4.1	Metodologia di clock	26
4.2	Costruire un datapath	26
4.3	Pipeline	27
4.4	Hazard nella pipeline	29
4.4.1	Hazard strutturali	29
4.4.2	Hazard sui dati	29
4.4.3	Hazard di controllo	30
4.5	Instruction Level Parallelism	33
4.6	Dynamic Multiple Issue	36
4.7	Dynamic Pipeline Scheduling	36
5	Istruzioni vettoriali (SIMD)	37
6	Le memorie	39
6.1	Terminologia della Gerarchia	39
6.2	Tecnologia DRAM	40
6.3	Memoria Cache	40
6.4	Gestione delle Scritture (Writes)	42
6.5	Memoria Cache Associativa	43
6.6	Interazioni con CPU Avanzate e Software	46

7	Processori Multi-Core e Cache Coherence	47
7.1	Snooping Protocols	47
7.2	False Sharing	48
7.3	Introduzione al Parallelismo	48
7.4	Hardware e Software nel Parallelismo	48
7.5	Amdahl's Law	49
7.6	Classificazione dei Flussi (Flynn) e Multithreading	49
7.7	Processori Multicore e Memoria Condivisa	50
7.8	Interconnection Networks	52
8	Architetture specifiche per dominio (DSA)	53
8.1	Motivazioni e Opportunità	53
8.2	Differenza tra DSA e ASIC	53
8.3	Perché i DSA sono più efficienti?	53
8.3.1	Systolic Arrays (Array Sistolici)	54
8.3.2	Double Buffering	54
8.4	Programmazione dei DSA	54
8.5	Sfide nella Progettazione dei DSA	54
8.5.1	Il "Memory Wall" negli Acceleratori	54
8.5.2	Riduzione dell'Overhead Energetico	55
8.6	Approcci Implementativi: Istruzioni Speciali vs Motori Dedicati	55
8.6.1	1. Istruzioni Specializzate in Processori General-Purpose	55
8.6.2	2. Acceleratori Dedicati (Motori Specializzati)	55
9	Google TPU: Caso di Studio di un DSA	56
9.1	Cenni sulle Deep Neural Networks	56
9.2	Architettura della TPUv1	57
9.3	ISA della TPU	58
9.4	Distribuzione dell'Area del Chip (Floorplan)	58
9.5	Roofline Model	59
9.6	Linee Guida per la Progettazione di DSA	59
10	Graphic Processing Unit (GPU)	60
10.1	Confronto Architetture: CPU vs GPU	60
10.2	Programmazione Scalabile e Gerarchia CUDA	62
10.3	Scheduling dei Thread e Warps	66
10.4	Ottimizzazione: Tiling e Shared Memory	69
10.5	Considerazioni Avanzate: Occupancy e Shared Memory	72
10.6	Shared Memory Banks e Conflitti	73
10.7	Accesso alla Global Memory e Coalescing	75
10.8	Trasferimento Dati Host-Device e Asincronia	76

1 Introduzione

1.1 Instruction Set Architecture

L'applicazione software viene scritta in un linguaggio ad alto livello, che viene tradotto in istruzioni in linguaggio macchina (interprete o compilatore - software di sistema) per essere eseguita dall'hardware. Il sistema operativo è il software di sistema che gestisce e alloca le risorse del calcolatore, allocando i processi in memoria e gestendo operazioni di I/O. L'hardware è composto da componenti fisici (CPU, memoria, periferiche) e componenti logici (registri, bus, ALU, etc.). I linguaggi tradizionali sono

compilati in assembly. L'assembler sarà il software che traduce l'assembly in istruzioni in linguaggio binario.

In generale classifichiamo le metriche:

- Response time: quanto impiega a fare un'operazione
- Throughput: lavoro fatto per unità di tempo

Ci si può chiedere come si comporta la performance di un sistema in funzione del numero di processori.

Definizione 1.1 (Relative Performance). Definiamo il Relative Performance di un sistema come:

$$\frac{P_x}{P_y} = \frac{E_y}{E_x}$$

dove E è l'**execution time** e P la performance che è data da $1/E$.

Abbiamo due possibilità di calcolare l'execution time:

- Misurando l'elapsed time: quanto tempo impiega a fare un'operazione (include processing time, I/O time, OS overhead, idle time). Considera il WTC (Wall Clock Time)
- Misurando il CPU time: quanto tempo impiega la CPU a fare un job. Comprende il tempo di processing time del programma e del sistema operativo.

Definizione 1.2 (CPU Clock). Il **clock** della CPU è un segnale periodico che scandisce il ritmo di esecuzione delle operazioni all'interno del processore. Ogni operazione fondamentale viene sincronizzata con i fronti del clock.

- **Clock period** (o clock time): durata di un ciclo di clock, ovvero l'intervallo di tempo tra due fronti consecutivi del segnale (tipicamente misurato in nanosecondi).
- **Clock frequency** (o clock rate): numero di cicli di clock per secondo (misurato in Hertz, tipicamente in GHz). Una frequenza più alta permette di eseguire più operazioni nell'unità di tempo.

Il clock determina quindi la velocità con cui la CPU può eseguire le istruzioni, ma non è l'unico fattore che influenza le prestazioni complessive.

Il **CPU time** (o tempo di CPU) è dato da:

$$\text{CPU time} = \text{CPU Clock Cycles} \times \text{Clock Time} = \frac{\text{CPU Clock Cycles}}{\text{Clock rate}}$$

Non è detto sempre che raddoppiando la frequenza del clock si raddoppi la performance, infatti potrebbe aumentare il numero di istruzioni per ciclo di clock.

$$\text{Clock cycles} = \text{Instruction Count} \times \text{Cycles per Instruction (CPI)}$$

$$\Rightarrow \text{CPU time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock rate}}$$

Istruzioni differenti hanno diversi cicli per istruzione (CPI). ISA indica l'**instruction set architecture**. Il clock cycle è anche dato da:

$$\text{Clock cycle} = \sum_{i=1}^n \text{CPI}_i \times \text{Instruction Count}_i$$

e possiamo pesare il CPI ponendo:

$$\text{CPI} = \sum_{i=1}^n \text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}}$$

Esempio 1.1. Consideriamo due possibili sequenze di codice compilato che utilizzano istruzioni appartenenti a tre classi (A, B, C), ognuna con un diverso CPI (Cycles Per Instruction):

Classe	A	B	C
CPI per classe	1	2	3
IC nella sequenza 1	2	1	2
IC nella sequenza 2	4	1	1

Sequenza 1:

Instruction Count (IC) totale = $2 + 1 + 2 = 5$

Clock Cycles = $2 \times 1 + 1 \times 2 + 2 \times 3 = 2 + 2 + 6 = 10$

CPI medio = $\frac{10}{5} = 2.0$

Sequenza 2:

Instruction Count (IC) totale = $4 + 1 + 1 = 6$

Clock Cycles = $4 \times 1 + 1 \times 2 + 1 \times 3 = 4 + 2 + 3 = 9$

CPI medio = $\frac{9}{6} = 1.5$

Osservazione:

Anche se la seconda sequenza ha un numero maggiore di istruzioni, il suo CPI medio è inferiore grazie all'uso prevalente di istruzioni della classe A (che hanno CPI più basso). Questo esempio mostra come la scelta delle istruzioni e la loro distribuzione influenzino il CPI medio e, di conseguenza, le prestazioni complessive del programma.

Possiamo dunque spezzare il CPU time in:

$$\text{CPU time} = \underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Quante istruzioni}} \times \underbrace{\frac{\text{Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \frac{1(s)}{\text{Clock cycle}}$$

Le performace dipendono da:

- Algoritmo
- Linguaggio di programmazione
- Compilatore
- Set di istruzioni

e tutte influenzano IC e CPI.

Il consumo energetico è dato da:

$$\text{Power} = \text{Capacity load} \times \text{Voltage}^2 \times \text{Frequency}$$

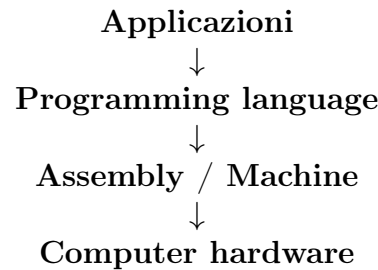
Il power wall è un limite fisico dato dalla temperatura e dal fatto che non possiamo ridurre oltre un certo limite la differenza di potenziale. Si è passati da un'architettura single core a un'architettura multi core, ma questa architettura richiede che le applicazioni siano parallelizzabili.

Gli elementi di una **instruction set architecture** sono:

- Stati
- Operazioni di computazione
- Operazioni di memoria
- Control flow

- Program structures
- Procedure

Si ha:



D'ora in avanti quando parleremo di instruction set faremo riferimento all'architettura RISC-V.

Lo stato di memoria di una CPU è composto da registri, e identificano lo stato di memoria a cui si può accedere direttamente. RISC-V ha 32 registri generali di cui ciascuno è a 64 bit. I risultati delle computazioni vengono memorizzati nei registri. Ogni operazione è costituita da:

- Legge 2 operandi
- Computa risultato = operando 1 operazione operando 2
- Scrive il risultato in un registro

Operazioni sono ad esempio: add, mul, sl (shift left), xor... Gli operandi possono anche essere **immediate** (al più uno dei due), quindi il valore dell'operando è già presente nell'istruzione.

Esempi di istruzioni RISC-V per alcune operazioni comuni:

- **32 + 19**

```

addi x1, x0, 32    # x0 è zero
addi x2, x1, 19
  
```

- **750 - 128**

```

addi x1, x0, 750
subi x2, x1, 128
  
```

- **121 « 3**

```

addi x1, x0, 121
slli x2, x1, 3
  
```

- **0x00FF | 0x01F0**

```

addi x1, x0, 0x00FF
ori x2, x1, 0x01F0
  
```

Tabella 1: Tabella di conversione binario-esadecimale

Binario	Esadecimale
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

2 Rappresentazione delle istruzioni

Le istruzioni sono codificate in binario. Per le RISC-V ogni istruzione è codificata in 32 bit. Ricordiamo che la conversione da binario a esadecimale è facile: basta raggruppare i bit a 4 a partire da destra, e ogni gruppo di 4 bit è convertito in un simbolo esadecimale.

Esempio 2.1. $\text{eca86420} \rightarrow 1110\ 1100\ 1010\ 1000\ 0110\ 0100\ 0010\ 0000$

Registri I registri sono elementi di memoria molto rapidi e vicini alla CPU. Loading e storing in memoria sono operazioni più lente. Il compilatore deve dunque ottimizzare al meglio l'uso dei registri attraverso operazioni di **spill**.

Attraverso le operazioni di **store** (sd) e **load** (ld) si possono spostare dati tra memoria e registri.

Esempio 2.2. Consideriamo alcune operazioni di memoria in RISC-V:

- **ld x5, 16(x10)**
Carica 8 byte dalla memoria all'indirizzo $16 + x10$ nel registro $x5$:
 $x5 \leftarrow \text{Mem}[16 + x10]$ (8 byte)
- **lb x6, 4(x10)**
Carica 1 byte dalla memoria all'indirizzo $4 + x10$ nel registro $x6$ (con segno):
 $x6 \leftarrow \text{Mem}[4 + x10]$ (1 byte)
- **lbu x7, 4(x10)**
Carica 1 byte dalla memoria all'indirizzo $4 + x10$ nel registro $x7$ (senza segno):
 $x7 \leftarrow \text{Mem}[4 + x10]$ (1 byte)
- **sd x5, 24(x10)**
Salva 8 byte dal registro $x5$ nella memoria all'indirizzo $24 + x10$:
 $\text{Mem}[24 + x10] \leftarrow x5$ (8 byte)
- **sb x6, 8(x10)**
Salva 1 byte dal registro $x6$ nella memoria all'indirizzo $8 + x10$:
 $\text{Mem}[8 + x10] \leftarrow x6$ (1 byte)

Chiarimento:

Le istruzioni di load (**ld**, **lb**, **lbu**) leggono dati dalla memoria e li copiano in un registro. La dimensione dei dati dipende dall'istruzione: **ld** per 8 byte (64 bit), **lb/lbu** per 1 byte (8 bit). La differenza tra **lb** e **lbu** è che la prima estende il byte letto con segno (sign extension), mentre la seconda lo estende senza segno (zero extension).

Le istruzioni di store (**sd**, **sb**) scrivono il contenuto di un registro in memoria, rispettivamente per 8 byte o 1 byte.

L'indirizzo effettivo di memoria è sempre calcolato come la somma di un offset (costante) e il valore di un registro base.

Control flow Le istruzioni di control flow sono:

- **j** (jump): salta a un indirizzo specificato (viene sempre eseguita, non è condizionale)
- **beq** (branch if equal): salta se due registri sono uguali, altrimenti continua all'istruzione successiva
- **bne** (branch if not equal): salta se due registri sono diversi, altrimenti continua all'istruzione successiva
- **blt** (branch if less than): salta se il primo registro è minore del secondo, altrimenti continua all'istruzione successiva

Esempio 2.3. Consideriamo alcune istruzioni di **control flow** in RISC-V e il loro comportamento:

- **j target**
Salta incondizionatamente all'indirizzo **target**.
La prossima istruzione eseguita sarà quella all'indirizzo **Mem[target]**.
- **beq x1, x2, target**
Salta all'indirizzo **target** se il contenuto dei registri $x1$ e $x2$ è uguale, altrimenti prosegue con l'istruzione successiva.
La prossima istruzione sarà:
 - **Mem[PC+target]** se $x1 == x2$ (PC => Program Counter, ovvero l'indirizzo dell'istruzione corrente)
 - altrimenti **PC+4**
- **bne x1, x2, target**
Salta all'indirizzo **target** se il contenuto dei registri $x1$ e $x2$ è diverso, altrimenti prosegue con l'istruzione successiva.
La prossima istruzione sarà:
 - **Mem[PC+target]** se $x1 \neq x2$
 - altrimenti **PC+4**
- **blt x1, x2, target**
Salta all'indirizzo **target** se il contenuto di $x1$ è strettamente minore di $x2$, altrimenti prosegue con l'istruzione successiva.
La prossima istruzione sarà:
 - **Mem[PC+target]** se $x1 < x2$
 - altrimenti **PC+4**

Nota: In tutti i casi, se la condizione non è soddisfatta, l'esecuzione prosegue con l'istruzione immediatamente successiva (**PC+4**, dato che ogni istruzione RISC-V è lunga 4 byte = 32 bit).

Esempio 2.4. Simuliamo il comportamento di una struttura `if-else` utilizzando le istruzioni di controllo di flusso in RISC-V. Consideriamo il seguente pseudocodice:

```
if (x1 == x2) {
    x3 = x3 + 1;
} else {
    x3 = x1 + 2;
}
```

La sequenza di istruzioni RISC-V corrispondente potrebbe essere:

Indirizzo	Istruzione
100	<code>beq x1, x2, Label1</code>
104	<code>addi x3, x1, 2</code>
108	<code>j Label2</code>
112	<code>Label1: addi x3, x3, 1</code>
116	<code>Label2: ...</code>

Spiegazione:

- `beq x1, x2, Label1`: se $x1 = x2$, salta a `Label1` (indirizzo 112).
- Se la condizione non è vera, esegue `addi x3, x1, 2` (else: $x3 = x1 + 2$).
- Dopo l'else, `j Label2` salta oltre il ramo then, evitando di eseguire anche il ramo if.
- `Label1: addi x3, x3, 1` viene eseguito solo se la condizione $x1 = x2$ è vera (then: $x3 = x3 + 1$).
- `Label2`: rappresenta il punto di continuazione del programma dopo la struttura if-else.

Esempio 2.5 (Somma di Gauss). `addi x3, x0, 100`

```
addi x1, x0, 0
Label1: beq x1, x3, End
add x2, x2, x1
addi x1, x1, 1
j Label1
End:
sd x2, Gauss(x0)
```

- `addi x3, x0, 100`: carica 100 in $x3$
- `addi x1, x0, 0`: carica 0 in $x1$
- `beq x1, x3, End`: se $x1 = x3$, salta a `End`.
- `add x2, x2, x1`: somma $x1$ a $x2$
- `addi x1, x1, 1`: incrementa $x1$ di 1
- `j Label1`: salta a `Label1`.
- `End: sd x2, Gauss(x0)`: salva $x2$ in memoria all'indirizzo 0.

Osservazione 2.1. Una volta che dati parziali del registro non sono più utilizzati, vengono sovrascritti non appena nuovi dati devono essere allocati.

Data structures

- `long a[16]` ciascun elemento è a 8 byte
- `char b[144]` ciascun elemento è a 1 byte
- `struct both {long c[20]; char d[128]}` struttura dati più complessa
- `both e[20]` array di 20 elementi di tipo `both`

Esempio 2.6. Consideriamo il seguente codice:

```
count = a[7];
Tenthchar = b[9];
TwTirth = b[23];
```

questo viene tradotto in:

```
ld x10, 56(x1)
lb x11, 9(x1)
lb x12, 23(x1)
```

Mentre il codice

```
e[10].c[5];
```

viene tradotto in:

```
addi x10, x3, 2000 # abbiamo limite 11 bit => 2047 numero massimo
addi x10, x10, 880
ld x11, 40(x10)
```

Esempio 2.7. Esempio di traduzione di costrutti `if-else` in assembly, mantenendo la corrispondenza tra variabili e valori come nell'immagine seguente:

C

```
if (a > 7) {
    a = a + 5;
} else {
    a = 19;
};

if (b != 15) {
    b = b - 5;
} else {
    b = b + 19;
};
```

Assembly

```
subi x3, x1, 7
ble x3, x0, ElClause
addi x1, x1, 5
j End1
ElClause:
    addi x1, x0, 19
End1:

subi x5, x4, 15
beq x5, x0, ElClauseee
subi x4, x4, 5
j End2
ElClauseee:
    addi x4, x4, 19
End2:
```

Esempio 2.8. Esempio di traduzione di costrutti **for-while** in assembly, mantenendo la corrispondenza tra variabili e valori come nell'immagine seguente:

C	Assembly
	add x1, x0, x0
	add x2, x0, x0
a=0;	loop1: addi x1, x1, 7
for(i=0; i<15; i++) {	addi x2, x2, 1
a = a + 7;	subi x3, x2, 15
};	blt x3, x0, loop1
	add x2, x0, x0
i=0;	loop2: subi x3, x1, 15
while(a<15) {	bge x3, x0, End
a = a + 3;	addi x1, x1, 3
i = i + a;	add x2, x2, x1
};	j loop2
	End: ...

Osservazione 2.2. Il comando *subi* non esiste, si usa invece *addi* con immediato negativo.

RISC-V (base RV32I) usa per default istruzioni di **32 bit** a formato fisso. La posizione dei campi **rd**, **rs1**, **rs2** e **funct3** è *coerente* tra i formati: questo semplifica l'hardware di *decode* e il *bypass* nella pipeline (i registri sorgente compaiono sempre negli stessi bit, riducendo mux e logica di controllo). La specifica usa i campi **funct3**/**funct7** come *sotto-opcode* per distinguere operazioni che condividono la stessa classe (stesso opcode).

RISC-V R-format instructions (Register) Il formato R usa 3 registri: due sorgenti e una destinazione.

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Ciascun campo ha uno scopo specifico:

- **opcode (7)**: classe d'istruzioni (aritmetiche R, aritmetiche immediate, load/store, branch, ...).
- **rd (5)**: registro destinazione ($2^5 = 32$ registri indirizzabili).
- **funct3 (3)** e **funct7 (7)**: selezionano l'operazione precisa dentro la classe; ad es. **add/sub** condividono opcode e funct3, ma si distinguono tramite funct7.
- **rs1, rs2 (5+5)**: registri sorgente.

Esempio 2.9. add x9, x20, x21 ($x9 \leftarrow x20 + x21$)

Campi in *decimale* (funct7 / rs2 / rs1 / funct3 / rd / opcode):

0 21 20 0 9 51

Campi in *binario* (larghezze corrette):

0000000	10101	10100	000	01001	0110011
---------	-------	-------	-----	-------	---------

RISC-V I-format instructions (Immediate) Il formato I usa 2 registri e un immediato di 12 bit con **estensione di segno** ($-2048 \dots 2047$). È usato per le istruzioni aritmetiche immediate e le istruzioni di load.

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

Importante: il campo **rs1** occupa sempre i bit $[19 : 15]$ e **rd** i bit $[11 : 7]$. Questo vale in tutti i formati principali.

RISC-V S-format instructions (Store) Usato per gli **store**. L'immediato di 12 bit è spezzato per tenere fissi i campi registro (*motivazione*: decode più semplice, meno cablaggio).

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Per RV32I, opcode degli store è 0100011. L'indirizzo è $rs1 + imm$; il dato proviene da **rs2**.

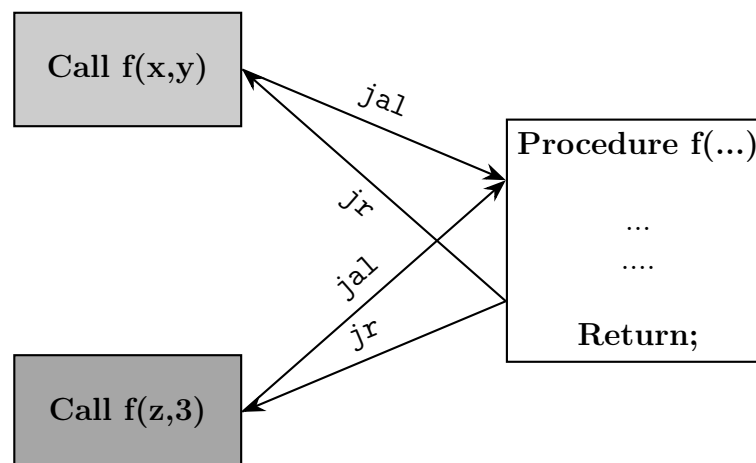
RISC-V B-format instructions (Branch) Usato per i **branch** condizionali, con immediato PC-relativo e *bit LSB implicito a 0* (Least Significant Bit). In questo modo il salto è sempre $\times 2$ byte, abilitando compatibilità con le istruzioni compresse a 16 bit e risparmiando 1 bit nell'encoding.

imm[12], imm[10:5]	rs2	rs1	funct3	imm[4:1], imm[11]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Osservazione 2.3. *Estensione C (compressed): formato opzionale a 16 bit che coesiste con le istruzioni a 32 bit. L'allineamento del bit LSB nei salti e la posizione fissa dei registri nei formati principali rendono più semplice supportare C senza complicare il decode.*

Procedure call instructions RISC-V prevede due primitive:

- **jal x1, target**: salto PC-relativo; scrive $PC + 4$ in **x1** e cattura l'indirizzo di ritorno in **x1** (jump and link)
- **jr x1**: salto indiretto; salta a **x1** (jump register)



Calling convention (ABI) essenziale:

- **ra** ($x1$) = return address; **sp** ($x2$) = stack pointer; **s0/fp** ($x8$) = frame pointer.
- Valori di ritorno: **a0** ($x10$) e **a1** ($x11$).
- Registri **a0-a7** ($x10-x17$) e **t0-t6** ($x5, x6, x7, x28-x31$) sono *caller-saved*; **s0-s11** ($x8, x9, x18-x27$) sono *callee-saved* e vanno preservati dalla funzione chiamata se modificati.
- Lo stack cresce verso indirizzi *decrementi* e deve restare allineato (tipicamente a 16 byte).

Perché queste scelte di encoding funzionano bene

- **Campi registro allineati:** i campi `rs1`, `rs2` e `rd` si trovano sempre nelle stesse posizioni nei formati R, I, S e B. Questo significa che il circuito che legge quali registri servono può essere lo stesso per tutti i tipi di istruzione, riducendo la complessità dell'hardware di decodifica.
- **Immediati spezzati (S e B):** in questi formati l'immediato è diviso in due parti per poter mantenere fissi i campi registro nelle stesse posizioni. In questo modo si evita di “spostare” i bit dei registri e si semplifica il cablaggio interno del processore.
- **Branch e JAL con LSB implicito a 0:** significa che il bit meno significativo dell'indirizzo di salto (LSB) non è memorizzato perché vale sempre 0. I salti, infatti, sono sempre a indirizzi allineati a 2 byte. Questo permette di risparmiare un bit nell'istruzione e di essere compatibili con il formato compresso a 16 bit.
- **PC-relative + link in rd:** nelle istruzioni di salto e chiamata di funzione (`jal` e `jalr`), l'indirizzo di destinazione non è assoluto, ma **relativo al Program Counter (PC)**. In pratica, il processore calcola il nuovo indirizzo come $PC + \text{offset}$. Questo approccio permette di scrivere codice *position-independent*, cioè che funziona anche se viene caricato in un punto diverso della memoria (utile per sistemi operativi e librerie condivise). Inoltre, l'istruzione non solo salta, ma **salva l'indirizzo di ritorno** ($PC + 4$) nel registro di destinazione `rd`. Questa operazione si chiama “link”: il registro `rd` diventa un *collegamento* con il punto da cui siamo partiti. Per convenzione, si usa il registro `ra` (`x1`) per memorizzare questo indirizzo. Così, quando la funzione termina, può semplicemente eseguire

```
jalr x0, 0(ra)
```

per tornare al chiamante, senza dover salvare o recuperare nulla dalla memoria. Solo se si fanno più chiamate annidate, il valore di `ra` deve essere salvato nello stack per non essere perso.

Appendice: opcode più comuni (RV32I)

R-type ALU	0110011
I-type ALU (<code>addi</code> , <code>andi</code> , ...)	0010011
Load (<code>lb</code> / <code>lh</code> / <code>lw</code>)	0000011
Store (<code>sb</code> / <code>sh</code> / <code>sw</code>)	0100011
Branch (<code>beq</code> / <code>bne</code> / <code>blt</code> /...)	1100011
<code>jal</code>	1101111
<code>jalr</code>	1100111
<code>lui</code>	0110111
<code>auipc</code>	0010111
System (<code>ecall</code> / <code>ebreak</code> / <code>csrr*</code>)	1110011

Lo **stack** permette di effettuare chiamate annidate e procedure ricorsive. Ciascuna procedura ha un proprio stack frame. Ecco le **Procedure di chiamata**:

- **Chiamante (caller)** salva i registri *caller-saved* (`a0-a7`, `t0-t6`) se necessario, e prepara gli argomenti in `a0-a7`.
- Il chiamante esegue `jal ra, funzione` per saltare alla procedura e salvare l'indirizzo di ritorno in `ra`.
- **Chiamata (callee):** la procedura alloca spazio nello stack decrementando `sp` (es. `addi sp, sp, -16`).
- La procedura salva i registri *callee-saved* (`s0-s11`, `ra` se chiama altre funzioni) nello stack appena allocato.

- La procedura esegue il suo corpo, utilizzando i registri liberi e salvando/ripristinando quelli necessari.
- Prima di terminare, la procedura scrive il valore di ritorno in `a0` (e `a1` se necessario).
- La procedura ripristina i registri *callee-saved* dallo stack.
- La procedura dealloca lo spazio dello stack incrementando `sp` (es. `addi sp, sp, 16`).
- La procedura esegue `jalr x0, 0(ra)` (equivalente a `ret`) per tornare al chiamante.
- Il chiamante estrae il valore di ritorno da `a0/a1` e ripristina eventuali registri *caller-saved* salvati prima della chiamata.

3 Aritmetica dei computer e rappresentazioni numeriche

Utilizziamo la codifica binaria per rappresentare i numeri (è detta **2s-Complement Signed Integer**). Distinguiamo infatti notazione *signed* e *unsigned*.

Signed

$$x = -2^{n-1} \cdot b_{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

Unsigned

$$x = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

dove b_i è il bit i -esimo della rappresentazione binaria. Introduciamo la seguente regola per trovare l'opposto di un numero:

$$-x = \bar{x} + 1$$

Dove il $\bar{x}$ è il complemento a 1 di x (negazione bit a bit: gli 1 diventano 0 e viceversa).

Esempio 3.1. $0111_2 = 7 \Rightarrow -7 = \overline{0111}_2 + 1 = 1000_2 + 1 = 1001_2$ che sarebbe $-8+1 = -7$.

Osservazione 3.1. Se lavoriamo a 4 bit allora

$$1111_2 + 1_2 = 0000_2 \quad (-1 + 1 = 0)$$

Possiamo chiaramente estendere la rappresentazione in bit per ottenere numeri più grandi. A seconda del segno del numero possiamo estenderlo con 0 o con 1 (con shift aritmetico o logico, a seconda del caso).

Le operazioni aritmetiche che si possono eseguire sono le canoniche (somma, sottrazione, moltiplicazione, divisione).

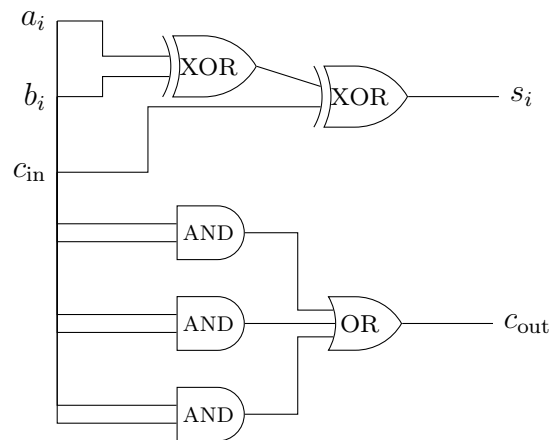
Esempio 3.2 (Integer addition). Somma di due numeri binari a 6 bit: 000111_2 (7) + 000110_2 (6) = 001101_2 (13)

carries:	0	0	1	1	0	
	0	0	0	1	1	1
+	0	0	0	1	1	0
=	0	0	1	1	0	1

I riporti (carries) vengono propagati da destra verso sinistra.

Osservazione 3.2. La sottrazione è immediata conseguenza della somma, in quanto è sufficiente sommare l'opposto del sottraendo (quindi il negato $+ 1$).

Osservazione 3.3 (Caso di overflow). Se lavoriamo a 4 bit (signed) allora la somma di 7 e 6 è 13, che non può essere rappresentato in 4 bit. Il calcolatore darebbe come risultato -3.



Moltiplicazione La moltiplicazione binaria segue lo stesso principio della moltiplicazione decimale in colonna: si generano i *prodotti parziali* (uno per ciascun bit del moltiplicatore) e si sommano con gli opportuni shift a sinistra.

Esempio 3.3 (Moltiplicazione binaria in colonna). Consideriamo 1011_2 (11 in decimale) moltiplicato per 1101_2 (13 in decimale).

		1	0	1	1		(moltiplicando: 11)
×		1	1	0	1		(moltiplicatore: 13)
		1	0	1	1		(1×1011)
		0	0	0	0		$(0 \times 1011, \text{shift di } 1)$
	1	0	1	1			$(1 \times 1011, \text{shift di } 2)$
	1	0	1	1			$(1 \times 1011, \text{shift di } 3)$
	1	0	0	0	1	1	1
							(risultato: 143)

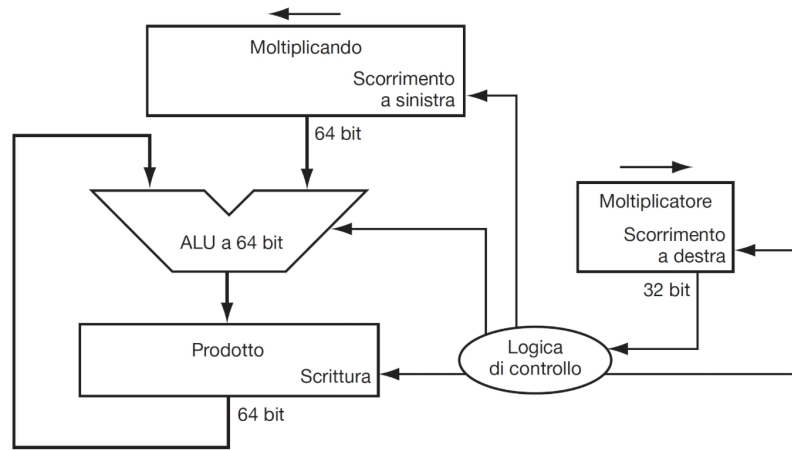
Il risultato è $10001111_2 = 143_{10}$, confermando che $11 \times 13 = 143$.

Ogni prodotto parziale si ottiene moltiplicando il moltiplicando per un singolo bit del moltiplicatore:

- Se il bit è 1, si copia il moltiplicando.
- Se il bit è 0, il prodotto parziale è zero.

Ogni prodotto parziale viene traslato (shift) verso sinistra in base alla posizione del bit corrispondente nel moltiplicatore. Infine, tutti i prodotti parziali vengono sommati per ottenere il risultato finale.

Circuito moltiplicatore hardware L'implementazione hardware della moltiplicazione utilizza un'architettura iterativa che processa un bit del moltiplicatore alla volta. Lo schema seguente mostra i componenti principali:

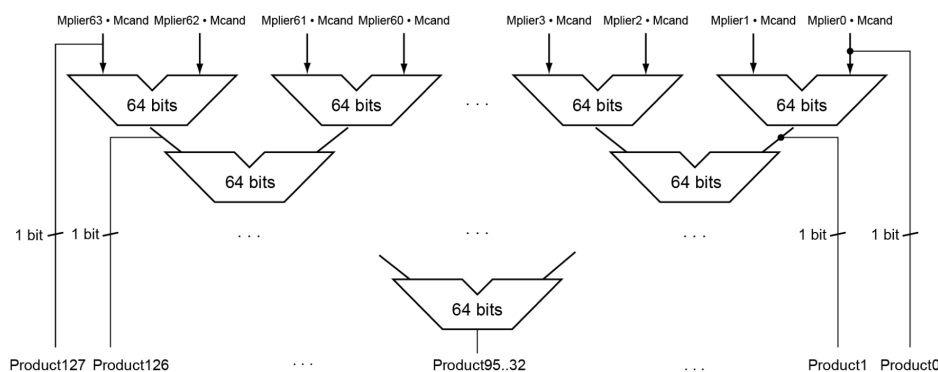


Componenti e funzionamento:

- **Registro Moltiplicando (64 bit):** contiene il moltiplicando e supporta lo scorrimento a sinistra per allineare i prodotti parziali.
- **Registro Moltiplicatore (32 bit):** contiene il moltiplicatore e scorre a destra per processare un bit alla volta.
- **ALU a 64 bit:** esegue la somma del prodotto parziale corrente con l'accumulatore.
- **Registro Prodotto (64 bit):** accumula i prodotti parziali e alla fine contiene il risultato finale.
- **Logica di controllo:** coordina le operazioni, legge il bit LSB del moltiplicatore per decidere se sommare o meno, gestisce gli shift e il conteggio delle iterazioni.

Ad ogni ciclo:

1. La logica di controllo esamina il bit meno significativo del moltiplicatore.
2. Se il bit è 1, l'ALU somma il moltiplicando al registro prodotto; se è 0, non fa nulla.
3. Il moltiplicando viene shiftato a sinistra di 1 posizione.
4. Il moltiplicatore viene shiftato a destra di 1 posizione.
5. Si ripete per 32 cicli (tanti quanti i bit del moltiplicatore).



Deduciamo dunque che la moltiplicazione di due numeri a n bit richiede al più $2n$ bit per il risultato, onde evitare overflow. La moltiplicazione su RISC-V ha 4 differenti istruzioni:

- **mul:** moltiplicazione
→ restituisce i 64 bit meno significativi del prodotto.

- **mulh**: "high" (signed $\times$ signed)
→ restituisce i 64 bit più significativi del prodotto.
- **mulhu**: "high" unsigned (unsigned $\times$ unsigned)
→ restituisce i 64 bit più significativi del prodotto.
- **mulhsu**: "high" signed/unsigned (signed $\times$ unsigned)
→ restituisce i 64 bit più significativi del prodotto.

Per rilevare un overflow a 64 bit:

- **unsigned**: c'è overflow se **mulhu** $\neq$ 0.
- **signed**: c'è overflow se **mulh** $\neq$ **arithmetic_shift_right**(**mul**, 63) (lo shift aritmetico di 63 posizioni replica il bit di segno: produce una maschera di tutti 0 se il risultato è positivo e di tutti 1 se è negativo).

Divisione Data una coppia (dividendo D , divisore V) con $V \neq 0$, la divisione produce quoziente Q e resto R tali che

$$D = V \cdot Q + R \quad \text{con} \quad 0 \leq R < |V|.$$

Con operandi a n bit, sia Q sia R sono a n bit.

Idea di base (divisione in colonna). È l'analogo binario della divisione "a mano": si confronta iterativamente il divisore con i bit più significativi del dividendo, producendo un bit del quoziente alla volta. Se "sta dentro" (cioè è minore o uguale), si sottrae e si scrive un 1 nel quoziente; altrimenti si scrive 0 e si "porta giù" il bit successivo del dividendo.

Esempio 3.4 (Divisione binaria in colonna). Dividiamo 1001010_2 (74) per 1000_2 (8).

1. Confronto sui bit alti: $1001_2 \geq 1000_2 \Rightarrow$ scrivo 1 nel quoziente e sottraggo: $1001_2 - 1000_2 = 0001_2$.
2. Porto giù il prossimo bit $\Rightarrow 10_2 (< 1000_2)$: quoziente bit 0.
3. Porto giù il prossimo bit $\Rightarrow 101_2 (< 1000_2)$: quoziente bit 0.
4. Porto giù l'ultimo bit $\Rightarrow 1010_2 (\geq 1000_2)$: quoziente bit 1 e sottraggo: $1010_2 - 1000_2 = 10_2$.

Quindi $Q = 1001_2$ (9) e $R = 10_2$ (2), verificando $74 = 8 \cdot 9 + 2$.

$$\begin{array}{r}
 \begin{array}{cccc} 1 & 0 & 0 & 0 \end{array} \begin{array}{l}) \\ - \end{array} \begin{array}{|cccccc} \hline 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ \hline 1 & 0 & 0 & 0 & & & \\ \hline & & & 1 & 0 & & \\ & & & 1 & 0 & 1 & \\ & & & 1 & 0 & 1 & 0 \\ \hline & & & 1 & 0 & 0 & 0 \\ & & & & & 1 & 0 \\ \hline \end{array} \begin{array}{l} Q \\ 1 \\ \\ 10 \\ 100 \\ 1001 \end{array} \\
 Q = 1001_2, \quad R = 10_2.
 \end{array}$$

Algoritmo *restoring*. Una forma comoda per l'hardware (ma fedele alla procedura in colonna) è:

1. Inizializza $R \leftarrow 0$, $Q \leftarrow 0$.
2. Per $i = n - 1, \dots, 0$ (dal bit MSB al LSB del dividendo):
 - (a) $R \leftarrow (R \ll 1) \mid D_i$ (porto giù il prossimo bit di D). Matematicamente è $R \leftarrow R \times 2 + D_i$.

(b) $R \leftarrow R - V$.

(c) Se $R < 0$ allora *ripristina* ($R \leftarrow R + V$) e poni $Q_i \leftarrow 0$; altrimenti poni $Q_i \leftarrow 1$.

Questo produce un bit di Q per iterazione; alla fine R è il resto non negativo. Esiste anche la forma equivalente che mantiene fisso il dividendo nel registro *Remainder* e fa scorrere a destra il *Divisor* ad ogni passo: il test del resto decide 1/0 nel quoziente e, se negativo, si “restauro” aggiungendo indietro il divisore.

Divisione con segno. Si opera sui valori assoluti e si corregge il segno alla fine:

- $\text{segno}(Q) = \text{segno}(D) \oplus \text{segno}(V)$.
- $\text{segno}(R) = \text{segno}(D)$ (con $R = 0$ ammesso).

Circuito divisore hardware L’unità di divisione iterativa riusa gli stessi principi (e gran parte dell’hardware) del moltiplicatore:

- **Registro Remainder (es. 128 bit):** contiene il parziale e, inizialmente, il dividendo allineato.
- **Registro Divisor (es. 128 bit):** contiene il divisore, inizialmente allineato sul mezzo alto e poi *shiftato* a destra di 1 ad ogni iterazione.
- **Registro Quotient (es. 64 bit):** si riempie *shiftando* a sinistra; ogni ciclo imposta il nuovo bit meno significativo a 1 o 0 in base al test.
- **ALU a larghezza doppia** (128 bit nell’esempio): esegue $R \leftarrow R - \text{Divisor}$ e, se serve, il ripristino $R \leftarrow R + \text{Divisor}$.
- **Logica di controllo:** confronta il segno/zero del resto, seleziona il bit del quoziente, coordina *shift* e conteggio dei cicli.

Un ciclo per sottrazione del resto parziale. Dopo un numero di iterazioni pari alla larghezza degli operandi, il quoziente è completo e R è il resto. La somiglianza con il moltiplicatore consente di riutilizzare componenti (ALU ampia, registri e logica di *shift*).

Osservazione 3.4. Divisore nullo. Prima di iniziare si verifica $V = 0$. Le ISA moderne (inclusa RISC-V) definiscono un risultato senza generare eccezioni per la maggior parte dei casi pratici.

Divisione in RISC-V Quattro istruzioni:

- **div, rem:** divisione/resto con segno;
- **divu, remu:** divisione/resto senza segno.

La divisione per zero in RISC-V non produce errori, ma restituisce un valore definito

Floating point Rappresentazione di numeri non interi. È simile alla notazione scientifica (e.g., 1.23×10^4). È definito dallo standard IEEE Std 754-1985 ed è adottata universalmente. Ci sono due tipi di rappresentazione in single (32 bit) e double precision (64 bit)

Single Precision (32 bit):

S	Exponent	Fraction
1 bit	8 bits	23 bits

Double Precision (64 bit):

S	Exponent	Fraction
1 bit	11 bits	52 bits

Dove:

- **S** (Sign): bit di segno (0 = positivo, 1 = negativo)
- **Exponent**: esponente in rappresentazione biased
- **Fraction**: mantissa (parte frazionaria normalizzata)

Il generico numero si scrive come:

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - \text{Bias}}$$

Solitamente preferiamo rappresentazioni normalizzate, ovvero dove la cifra significativa è tra 1.0 e 2.0.

Alcune rappresentazioni sono riservate: 00000000 e 11111111. Il più piccolo valore per l'esponente è 00000001 (-126) e il più grande è 11111110 (127) (in 32 bit), mentre in 64 bit è 000000000001 (-1022) e 11111111110 (1023).

Se l'esponente ha tutti 1 e la mantissa è 0, il numero è considerato infinito, mentre se la frazione è diversa da 0, il numero è considerato NaN (Not a Number), ovvero il risultato di un'operazione matematica non valida (e.g., 0/0).

Rappresentazioni normalizzate hanno tutte la stessa accuratezza, mentre per rappresentazioni non normalizzate si perde di precisione, ma permettono di rappresentare numeri molto piccoli e molto grandi.

Esempio 3.5. Rappresentiamo il numero -0.75 in single precision e double precision.

- Single precision: posso scrivere -0.75 come $-\frac{3}{4}$ e dato che $3 = 11_2$ e devo dividere per 4 (due shift a destra), posso scrivere $0.75 = 0.11_2 \times 2^0$ e normalizzando abbiamo $1.1_2 \times 2^{-1}$ e quindi $S = 1$, $E = 126$ dato che $126 = 127 - 1$ (dal bias)

3.1 Addizione floating point

Esempio in base 10, mantissa normalizzata con $t = 4$ cifre significative. Sommiamo

$$9.999 \times 10^1 + 1.610 \times 10^{-1}.$$

1. Allineamento degli esponenti

Si sposta il numero che presenta l'esponente minore:

$$1.610 \times 10^{-1} = 0.01610 \times 10^1 \Rightarrow 9.999 \times 10^1 + 0.016 \times 10^1$$

(tagliato a $t=4$ durante l'allineamento).

2. Somma delle significande

$$(9.999 + 0.016) \times 10^1 = 10.015 \times 10^1.$$

3. Normalizzazione e controllo over/underflow

$$10.015 \times 10^1 = 1.0015 \times 10^2.$$

4. Arrotondamento e rinormalizzazione se serve

Con $t = 4$ cifre:

$$1.0015 \times 10^2 \longrightarrow \boxed{1.002 \times 10^2}.$$

Osservazione 3.5. *L'errore deriva dall'allineamento (bit/cifre "perse") e dall'arrotondamento finale.*

Esempio in base 2, significando normalizzato con $t = 4$ cifre.

$$1.000_2 \times 2^{-1} + (-1.110_2) \times 2^{-2} \quad (\text{cioè } 0.5 + (-0.4375))$$

1. **Allinea i punti binari** (sposta il numero con esponente minore):

$$(-1.110_2) \times 2^{-2} = (-0.1110_2) \times 2^{-1} \Rightarrow 1.000_2 \times 2^{-1} + (-0.111_2) \times 2^{-1}.$$

2. **Somma le significande:**

$$(1.000_2 + (-0.111_2)) \times 2^{-1} = 0.001_2 \times 2^{-1}.$$

3. **Normalizza e verifica over/underflow:**

$$0.001_2 \times 2^{-1} = 1.000_2 \times 2^{-4} \quad (\text{nessun over/underflow}).$$

4. **Arrotonda e, se serve, rinormalizza** (round-to-nearest-even, $t = 4$):

$$\boxed{1.000_2 \times 2^{-4}} \quad (\text{nessuna variazione}) = 0.0625.$$

Floating point adder hardware L'hardware per l'addizione in virgola mobile è più complesso rispetto al caso intero, poiché deve eseguire l'allineamento degli esponenti, la somma delle mantisse, la normalizzazione e l'arrotondamento. Per questo motivo, l'operazione richiede più cicli di clock per essere completata e viene spesso implementata come unità multi-ciclo o pipelined, in modo da non penalizzare le prestazioni complessive del processore.

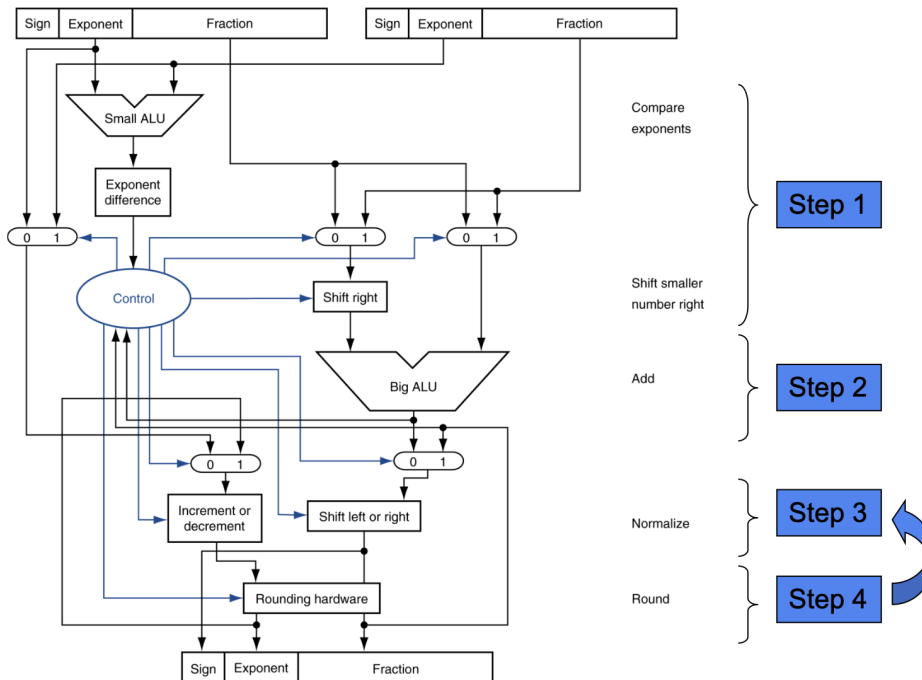


Figura 1: Floating point adder hardware

3.2 Moltiplicazione floating point

Esempio in base 10, significando normalizzato con $t = 4$.

$$(1.110 \times 10^{10}) \times (9.200 \times 10^{-5})$$

1. **Somma degli esponenti** (con esponenti biasati si sottrae il bias dopo la somma):

$$e_{\text{nuovo}} = 10 + (-5) = 5.$$

2. **Moltiplica le significande:**

$$1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^5.$$

3. **Normalizza e controlla over/underflow:**

$$10.212 \times 10^5 = 1.0212 \times 10^6.$$

4. **Arrotonda ed eventualmente rinormalizzare:**

$$\boxed{1.021 \times 10^6}.$$

5. **Segno** dal prodotto dei segni degli operandi:

$$\boxed{+ 1.021 \times 10^6}.$$

Esempio in base 2, significando normalizzato con $t = 4$ cifre.

$$(1.000_2 \times 2^{-1}) \times (-1.110_2 \times 2^{-2}) \quad (\text{cioè } 0.5 \times -0.4375)$$

1. **Somma degli esponenti**

$$\text{Unbiased: } (-1) + (-2) = -3,$$

$$\text{Biased: } [(-1 + 127) + (-2 + 127)] - 127 = -3 + 254 - 127 = -3 + 127.$$

2. **Moltiplica le significande**

$$1.000_2 \times 1.110_2 = 1.110_2 \Rightarrow 1.110_2 \times 2^{-3}.$$

3. **Normalizza e controlla over/underflow**

$$1.110_2 \times 2^{-3} \quad (\text{nessun cambiamento, nessun overflow né underflow}).$$

4. **Arrotondamento ed eventualmente rinormalizzazione**

$$1.110_2 \times 2^{-3} \quad (\text{nessuna variazione}).$$

5. **Determina il segno**

$$(+) \times (-) \Rightarrow (-), \quad \boxed{-1.110_2 \times 2^{-3}}.$$

Conversione decimale:

$$-1.110_2 \times 2^{-3} = -0.21875.$$

Osservazione 3.6. Nel formato IEEE 754 l'esponente è memorizzato in forma biasata, cioè traslato di un valore costante (bias) per evitare di gestire bit di segno e semplificare i confronti in hardware (esegue solo operazioni con interi non negativi). L'esponente reale si ottiene come $E_{\text{unbiased}} = E_{\text{biased}} - \text{Bias}$. Tipicamente il bias vale 127 (single precision) o 1023 (double precision).

Floating point arithmetic hardware L'hardware per la moltiplicazione in virgola mobile ha una complessità simile al floating point adder, ma usa un moltiplicatore per i significandi invece di un sommatore. Il FP arithmetic hardware può implementare diverse operazioni: addizione, sottrazione, moltiplicazione, divisione, reciproco e radice quadrata. Come il floating point adder, questo richiede più cicli di clock per essere completato.

Istruzioni FP in RISC-V Si utilizzano i registri $f_0, \dots, f_{31}$ e la rappresentazione può essere single (32 bit) o double (64 bit) precision. Le istruzioni FP possono operare solo su questi registri, e generalmente non vengono utilizzati per operazioni su interi. Le istruzioni per FP load e store sono:

- **flw**: load floating point word
- **fld**: load floating point double word
- **fsw**: store floating point word
- **fsd**: store floating point double word

Per le operazioni aritmetiche in single/double precision:

- **fadd.s/d**
- **fsub.s/d**
- **fmul.s/d**
- **fdiv.s/d**
- **fsqrt.s/d**

Per le operazioni di confronto:

- **feq.s/d**
- **flt.s/d**
- **fgt.s/d**

i risultati delle operazioni di confronto vengono memorizzati in un registro di “integer” come 0 o 1.

Esempio 3.6 (Da °F a °C: compilazione e codice RISC-V (single precision)). **Funzione C**

```
float f2c(float fahr) {
    return (5.0f/9.0f) * (fahr - 32.0f);
}
```

Assunzioni ABI. (Application Binary Interface) L'argomento **fahr** è passato in **fa0** (f10) e il valore di ritorno viene restituito sempre in **fa0** (f10). Le costanti sono in memoria globale.

Codice RISC-V compilato

```
f2c:
    flw      f0,    const5(gp)    # f0  = 5.0f
    flw      f1,    const9(gp)    # f1  = 9.0f
    fdiv.s   f0,    f0,    f1      # f0  = 5.0f / 9.0f
    flw      f1,    const32(gp)   # f1  = 32.0f
    fsub.s   f10,   f10,    f1      # f10 = fahr - 32.0f
    fmul.s   f10,   f0,    f10      # f10 = (5/9) * (fahr - 32)
    jalr     x0,    0(ra)         # return (fa0=f10)
```

gp=global pointer (registro x3)

Accurate arithmetic È chiaro che la rappresentazione in virgola mobile non può coprire l'intero insieme dei numeri reali. Per questo motivo lo standard IEEE 754 specifica un insieme di modalità di arrotondamento e regole per il trattamento delle eccezioni, al fine di rendere il comportamento numerico più prevedibile e coerente tra diverse piattaforme. Il trade-off è tra la complessità dell'hardware, le performance e i requisiti dell'applicativo.

- **Bit extra di precisione:** durante le operazioni, l'unità in virgola mobile può utilizzare alcuni bit addizionali (detti *guard bits*) per ridurre l'errore di arrotondamento nei risultati.
- **Modalità di arrotondamento:** lo standard prevede diverse modalità, come “round to nearest”, “round toward zero”, “round up” e “round down”, che permettono al programmatore di controllare il comportamento dell'arrotondamento.

Si noti che non tutte le unità in virgola mobile implementano l'intero insieme delle funzionalità previste dallo standard IEEE 754. Inoltre, la maggior parte dei linguaggi di programmazione e delle librerie numeriche si limita a utilizzare le modalità di arrotondamento e le impostazioni predefinite fornite dall'hardware o dal compilatore.

Esempio 3.7 (Errori calcolo parallelo). L'assunzione di associatività potrebbe portare a errori nei calcoli paralleli.

Tabella 2: Errori calcolo parallelo

		$(x + y) + z$	$x + (y + z)$
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.0	0.00E+00

Si rende necessaria la validazione dei risultati delle operazioni parallele per garantire la correttezza del calcolo.

Fused Multiply Add Molte architetture presentano l'istruzione **Fused Multiply Add** (FMA), che esegue il seguente calcolo:

$$a = a + (b \times c)$$

Questa istruzione può eseguire un singolo arrotondamento dopo l'addizione, aumentando la precisione dell'istruzione multiply add.

Le applicazioni grafiche e di audio sfruttano il calcolo parallelo per eseguire operazioni simultanee su vettori più “piccoli”. Ad esempio un 128-bit adder potrebbe eseguire:

- 16 addizioni a 8 bit
- 8 addizioni a 16 bit
- 4 addizioni a 32 bit

Esempio 3.8 (Streaming SIMD Extensions 2 (SSE2)). SSE2 è un'estensione **SIMD** (Single Instruction Multiple Data) dell'architettura **x86**, progettata per migliorare le prestazioni in ambiti come elaborazione multimediale, crittografia e calcolo scientifico. SSE2 introduce **16 registri a 128 bit** (**xmm0-xmm15**) che consentono di eseguire la stessa operazione in parallelo su più dati (interi, float, double), con supporto a operazioni su elementi da 8, 16, 32 e 64 bit.

Esempio 3.9 (Advanced Vector Extensions 512 (AVX-512)). AVX-512 è un'estensione **SIMD** introdotta da Intel nel 2016 per l'architettura **x86**, che opera su vettori larghi **512 bit**, consentendo l'elaborazione parallela di insiemi di dati di grandi dimensioni.

Registri. AVX-512 utilizza **32 registri vettoriali** da 512 bit (`zmm0-zmm31`), ciascuno capace di contenere, ad esempio, 16 valori `float` (32 bit) o 8 valori `double` (64 bit).

Masking support. AVX-512 introduce **8 registri di maschera** (`k0-k7`) che consentono di abilitare o disabilitare selettivamente le operazioni sugli elementi di un vettore. Questo permette, ad esempio, di eseguire un'operazione solo su alcuni elementi (dove il bit di maschera è 1) lasciando invariati gli altri. Le maschere rendono così più efficiente la gestione di condizioni parziali nei calcoli vettoriali (e.g. operazioni su subset di dati).

Set di istruzioni esteso. AVX-512 fornisce istruzioni avanzate per operazioni complesse come *scatter/gather*, operazioni bitwise, confronto vettoriale e arrotondamento efficiente, migliorando sia le prestazioni sia la flessibilità dell'elaborazione vettoriale.

Vector Intrinsics Le **vector intrinsics** sono funzioni in stile C che permettono di utilizzare direttamente le istruzioni vettoriali (come AVX o AVX-512) senza scrivere codice assembly. Ogni tipo di dato (`__m512i`, `__m512`, `__m512d`) rappresenta un registro vettoriale da 512 bit in grado di contenere più elementi: interi, numeri in singola o doppia precisione.

Le intrinsics forniscono un'interfaccia diretta alle istruzioni SIMD per operazioni comuni come:

- `_mm512_load_pd` e `_mm512_store_pd`: caricamento e salvataggio di vettori di 8 numeri `double` (`pd` = packed double);
- `_mm512_broadcastsd_pd`: replica di un valore scalare su tutti gli elementi del vettore;
- `_mm512_fmadd_pd`: esecuzione di un'operazione *fused multiply-add* su vettori di numeri in doppia precisione.

Queste funzioni consentono di sfruttare le capacità SIMD del processore direttamente dal codice C/C++, mantenendo portabilità e leggibilità pur ottenendo prestazioni paragonabili al codice assembly. Per compatibilità con le estensioni precedenti (SSE/SSE2), sono disponibili anche intrinsics come `_mm_load_sd`, che carica un valore scalare in un registro a 128 bit (`__m128d`).

Esempio 3.10 (Moltiplicazione di matrici con AVX-512 intrinsics). Il codice seguente mostra una versione ottimizzata della moltiplicazione di matrici (*DGEMM Double-precision General Matrix Multiplication*) che utilizza le intrinsics AVX-512 per elaborare 8 elementi in parallelo, sfruttando registri vettoriali da 512 bit.

```
#include <x86intrin.h>

void dgemm(int n, double* A, double* B, double* C) {
    for (int i = 0; i < n; i += 8) // elabora 8 elementi per volta
        for (int j = 0; j < n; ++j) {
            __m512d c0 = _mm512_load_pd(C + i + j*n); // carica C[i][j]
            for (int k = 0; k < n; ++k) {
                __m512d bb = _mm512_broadcastsd_pd(_mm_load_sd(B + j*n + k));
                c0 = _mm512_fmadd_pd(_mm512_load_pd(A + n*k + i), bb, c0);
            }
            _mm512_store_pd(C + i + j*n, c0); // salva C[i][j]
        }
}
```

Descrizione.

- Ogni registro `__m512d` contiene 8 numeri in doppia precisione.
- `_mm512_load_pd` e `_mm512_store_pd` caricano e salvano blocchi di 8 elementi da/verso memoria.

- `_mm512_broadcastsd_pd` replica un valore scalare di `B` su tutto il vettore.
- `_mm512_fmadd_pd` esegue in un'unica istruzione la moltiplicazione e somma accumulata: $C[i][j] + = A[i][k] \times B[k][j]$.

La riga `__m512d bb = _mm512_broadcastsd_pd(_mm_load_sd(B + j*n + k))` carica l'elemento `B[k][j]` e lo duplica in tutte le 8 posizioni del registro vettoriale, così da poterlo moltiplicare in parallelo con 8 elementi della colonna `k` di `A` (partendo dalla riga `i`).

Grazie al parallelismo vettoriale, il ciclo interno elabora 8 elementi contemporaneamente, riducendo il numero totale di istruzioni e migliorando l'efficienza computazionale.

Osservazione 3.7. *Le matrici sono memorizzate Column-Major Order, per cui*

4 Progettazione di un processore

Le prestazioni di una CPU dipendono principalmente da tre fattori:

- **Instruction Count** — determinato dall'ISA (Instruction Set Architecture) e dal compilatore;
- **CPI** (*Cycles Per Instruction*) — determinato dall'organizzazione hardware;
- **Tempo di ciclo** — fissato dalla velocità del clock e dal design della pipeline.

In questa sezione analizzeremo due possibili implementazioni del processore **RISC-V**:

- una versione semplificata, utile per comprendere la struttura base;
- una versione più realistica, con esecuzione **pipelined**.

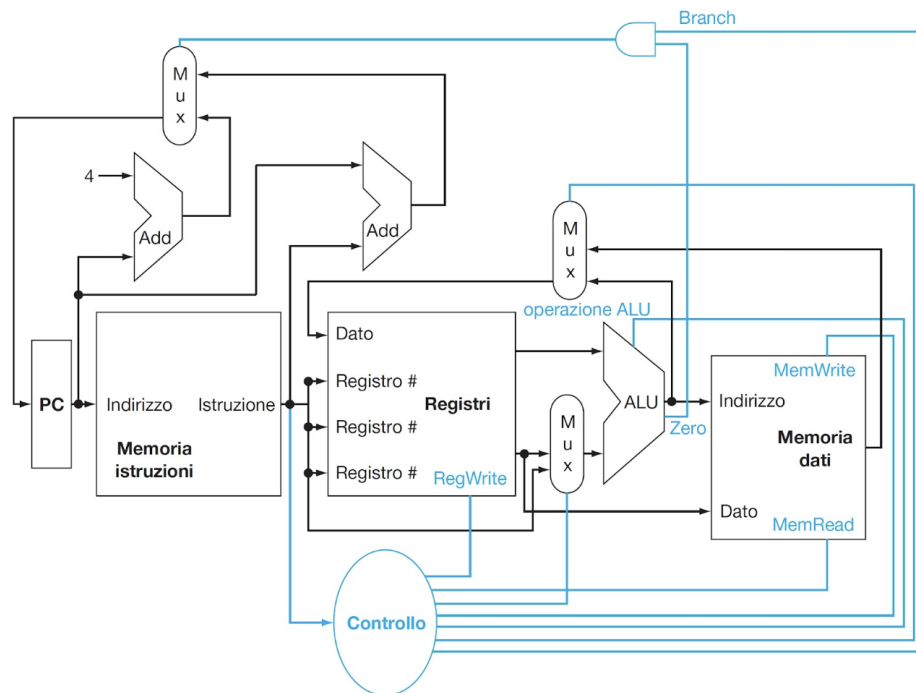
Le istruzioni considerate appartengono a un sottoinsieme rappresentativo, che include:

- **Istruzioni di memoria:** `ld`, `sd`;
- **Istruzioni aritmetico/logiche:** `add`, `sub`, `and`, `or`;
- **Istruzioni di controllo:** `beq`.

Esecuzione delle istruzioni

L'esecuzione di un'istruzione RISC-V segue una sequenza ordinata di operazioni:

- **Fetch:** il **Program Counter (PC)** indica l'indirizzo dell'istruzione da leggere nella memoria delle istruzioni;
- **Decode e lettura registri:** i numeri di registro specificati nell'istruzione vengono usati per accedere al **register file**;
- **Esecuzione:** a seconda del tipo di istruzione,
 - la **ALU** calcola il risultato aritmetico/logico oppure l'indirizzo di memoria per **load/store**;
 - in caso di **load/store**, si accede alla **memoria dati**;
 - il **PC** viene aggiornato con `PC+4` (istruzione successiva) o con un nuovo indirizzo (salto o branch).



Datapath a singolo ciclo (sottinsieme RISC-V)

La figura mostra un datapath che esegue in *un solo ciclo* le classi di istruzioni R (add/sub/and/or), I/S per ld/sd e i branch (beq).

Blocchi dati (nero).

- **PC → Memoria istruzioni:** il PC fornisce l'indirizzo; l'istruzione letta viene decodificata.
- **Register file:** due porte di lettura (**rs1**, **rs2**) e una di scrittura (**rd**). Il dato scritto proviene da un mux di *writeback*.
- **ALU:** riceve **rs1** e un secondo operando scelto da un mux (**rs2** oppure immediato esteso di segno); calcola risultato aritmetico/logico o indirizzi (**base+offset**). Espone il flag **Zero** per i branch.
- **Memoria dati:** indirizzo dall'ALU; dato di scrittura da **rs2**; il dato letto può essere scritto in **rd**.
- **Adders:** uno calcola **PC + 4**; l'altro il **target di branch** (**PC + offset**, con offset già allineato).
- **Mux di PC:** seleziona il *next PC* tra **PC + 4** e il target di branch.

Segnali di controllo (azzurro). Sono generati dal blocco **Contollo** a partire da **opcode/funct** e pilotano mux e memorie:

- **RegWrite:** abilita la scrittura in **rd**;
- **ALUSrc:** seleziona il secondo operando dell'ALU (**rs2** vs immediato);
- **ALUOp:** specifica l'operazione ALU (add/sub/and/or/compare);
- **MemRead/MemWrite:** accesso alla memoria dati;
- **MemToReg:** seleziona il dato da scrivere in **rd** (risultato ALU vs dato letto dalla memoria);
- **Branch:** abilita il cammino di salto condizionato. Il segnale **PCSrc** = **Branch** $\wedge$ **Zero** seleziona il target nel mux del PC.

Flusso per classe d'istruzione (riassunto).

- **R-type** (add/sub/and/or): $ALUSrc = 0$ (usa **rs2**), $ALUOp$ = funzione, $RegWrite = 1$, $MemRead = MemWrite = 0$, $MemToReg = 0$, $PCSrc = 0$.
- **ld**: $ALUSrc = 1$ (somma base+offset), $ALUOp$ = add, $MemRead = 1$, $MemToReg = 1$, $RegWrite = 1$, $MemWrite = 0$, $PCSrc = 0$.
- **sd**: $ALUSrc = 1$, $ALUOp$ = add, $MemWrite = 1$, $RegWrite = 0$, $MemRead = 0$, $PCSrc = 0$.
- **beq**: $ALUSrc = 0$, $ALUOp$ = sub/compare; $Branch = 1$ e $PCSrc = Branch \wedge Zero$ decide tra $PC + 4$ e target.

Istruzione	RegWrite	ALUSrc	ALUOp	MemRead	MemWrite	MemToReg	PCSrc
add/sub/and/or	1	0	func	0	0	0	0
ld	1	1	add	1	0	1	0
sd	0	1	add	0	1	X	0
beq	0	0	cmp	0	0	X	$Branch \wedge Zero$

Lettura della figura. I cammini neri trasportano i **dati** (istruzioni, registri, indirizzi, risultati). I cammini azzurri sono **segnali di controllo** che selezionano gli ingressi dei mux, l'operazione ALU, gli accessi alle memorie e l'aggiornamento del PC.

4.1 Metodologia di clock

La logica *combinatoria* elabora i dati **tra due fronti di clock**; i risultati vengono catturati da elementi di *stato* (registri: PC, register file, ecc.). Il **periodo di clock** è fissato dal *percorso critico*:

$$T_{clk} \geq T_{cq} + T_{comb,max} + T_{setup}$$

dove T_{cq} è il ritardo uscita-registro, T_{setup} il tempo di setup del registro di arrivo. (Il vincolo di *hold* si rispetta progettando i ritardi minimi.)

4.2 Costruire un datapath

Per *datapath* intendiamo i blocchi che trattano dati e indirizzi: **PC**, memoria istruzioni, **register file**, **ALU**, memoria dati, adders e mux. Procediamo per raffinamenti: prima il **fetch**, poi l'esecuzione delle R-type, quindi (a valle) si aggiungono load/store e branch.

Instruction fetch (IF)

- Il **PC** (registro a 64 bit) fornisce l'indirizzo alla **memoria istruzioni**.
- Un adder calcola $PC + 4$ per l'istruzione successiva (codice a 32 bit).
- In assenza di salti/branch, il **next PC** è $PC + 4$.

Istruzioni R-format (add/sub/and/or)

- Dalla codifica si estraggono **rs1**, **rs2**, **rd**; il **register file** legge due operandi.
- La **ALU** esegue l'operazione selezionata da **funct3/funct7**.
- Il risultato viene scritto in **rd** tramite il canale di *write-back*.

Osservazione 4.1. *Per come abbiamo rappresentato l'architettura, al momento **un'intera istruzione** deve attraversare tutti i blocchi del datapath (fetch, decode, ALU, memoria, write-back) prima che possa iniziare la successiva. Questo significa che, anche se la maggior parte dei blocchi resta inattiva durante l'esecuzione di una singola istruzione, l'intero processore è "occupato" da una sola operazione alla volta. Il **throughput** (numero di istruzioni completate per unità di tempo) è molto basso: completa 1 istruzione ogni ciclo di clock lungo quanto il percorso critico.*

4.3 Pipeline

La **pipeline** è una tecnica che permette di eseguire *più istruzioni contemporaneamente*, suddividendo il datapath in **stadi** separati da registri. L'idea chiave è semplice: mentre un'istruzione usa l'ALU, un'altra può già essere in fase di fetch, e una terza può essere in write-back.

Analogia con la lavanderia. Immagina di dover lavare 4 carichi di panni. Hai a disposizione lavatrice, asciugatrice, asse da stiro e armadio. *Senza pipeline:* completi tutte e 4 le fasi per il primo carico, poi inizi il secondo. Totale: $4 \times 4 = 16$ ore. *Con pipeline:* mentre il primo carico asciuga, metti il secondo in lavatrice. Dopo 4 ore (primo carico completo + setup) hai un flusso continuo: ogni ora esce un carico finito. Totale: $4 + 3 = 7$ ore.

Pipeline nel processore. Dividiamo l'esecuzione in 5 stadi:

1. **IF** (Instruction Fetch): lettura istruzione dalla memoria.
2. **ID** (Instruction Decode): decodifica e lettura registri.
3. **EX** (Execute): esecuzione ALU o calcolo indirizzo.
4. **MEM** (Memory): accesso memoria dati (load/store).
5. **WB** (Write-Back): scrittura risultato in registro.

Ogni stadio lavora su un'istruzione diversa nello stesso ciclo di clock. Dopo un breve periodo di "riempimento" (*fill-up*), ogni ciclo *completa* un'istruzione: il **throughput aumenta fino a 5 volte** rispetto al datapath non pipelined, pur mantenendo la stessa **latenza** per la singola istruzione (5 cicli).

Vantaggi.

- **Maggior throughput:** più istruzioni al secondo.
- **Utilizzo efficiente:** tutti i blocchi lavorano in parallelo.
- **Clock più rapido:** ogni stadio è più corto del percorso critico originale.

Pipeline performance Assumendo che il tempo per gli stadi siano:

- **100 ps** per lettura e scrittura dei registri
- **200 ps** per gli altri stadi

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
ld	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
sd	200 ps	100 ps	200 ps	200 ps	—	700 ps
R-format	200 ps	100 ps	200 ps	—	100 ps	600 ps
beq	200 ps	100 ps	200 ps	—	—	500 ps

4.4 Hazard nella pipeline

Gli **hazard** sono situazioni che impediscono all'istruzione successiva di procedere nel ciclo successivo. Si dividono in tre categorie:

- **Hazard strutturali**: due istruzioni vogliono usare la stessa risorsa hardware nello stesso ciclo.
- **Hazard sui dati**: un'istruzione dipende dal risultato di un'istruzione precedente ancora in pipeline.
- **Hazard di controllo**: un branch o un salto cambia il flusso di esecuzione, invalidando le istruzioni già caricate.

4.4.1 Hazard strutturali

Si verifica quando la pipeline necessita di una risorsa già occupata. **Esempio classico**: memoria unica per istruzioni e dati. Se l'istruzione 1 è in stadio MEM (accede alla memoria dati), l'istruzione 5 non può fare IF (accedere alla memoria istruzioni) $\Rightarrow$ *conflitto*.

Soluzioni.

- **Duplicare le risorse**: memorie separate (*Harvard architecture*) per istruzioni e dati; ALU dedicate per calcoli paralleli.
- **Stall (inserire bolle)**: bloccare la pipeline per un ciclo, perdendo throughput. Questa soluzione è usata solo quando la duplicazione è troppo costosa.

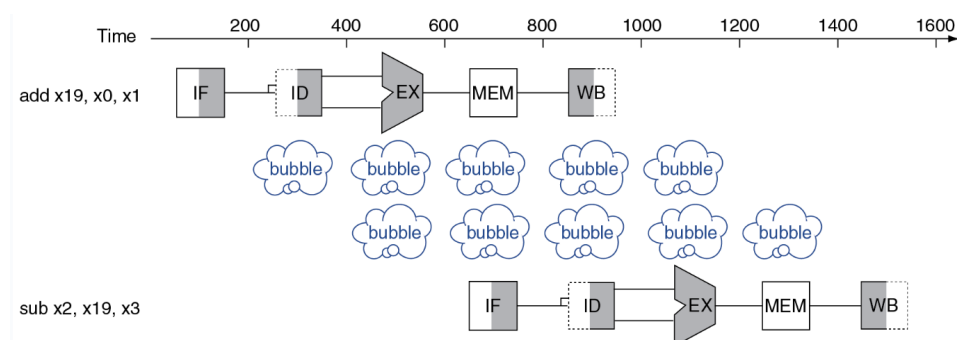
RISC-V evita molti hazard strutturali grazie al register file *dual-ported* (legge 2 registri e ne scrive 1 contemporaneamente) e alla separazione load/store.

4.4.2 Hazard sui dati

Si verifica quando un'istruzione necessita di un dato che un'istruzione precedente sta ancora producendo. **Esempio**:

```
add x19, x0, x1    # scrive x19 in WB (5° ciclo)
sub x2, x19, x3    # legge x19 in ID (2° ciclo)
```

La sub legge x19 prima che add lo scriva $\Rightarrow$ *valore errato*.

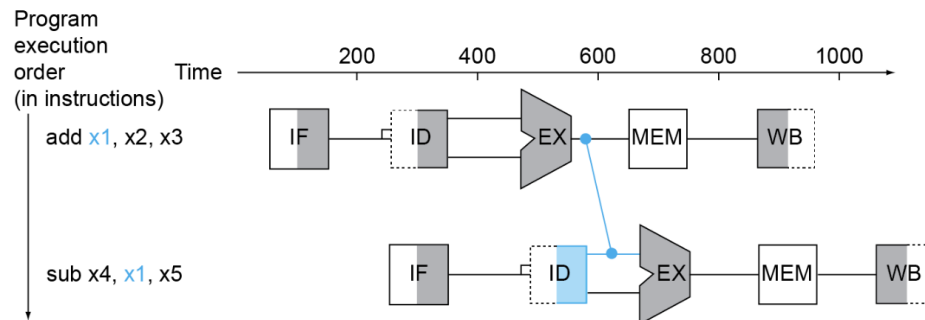


Tipi di data hazard.

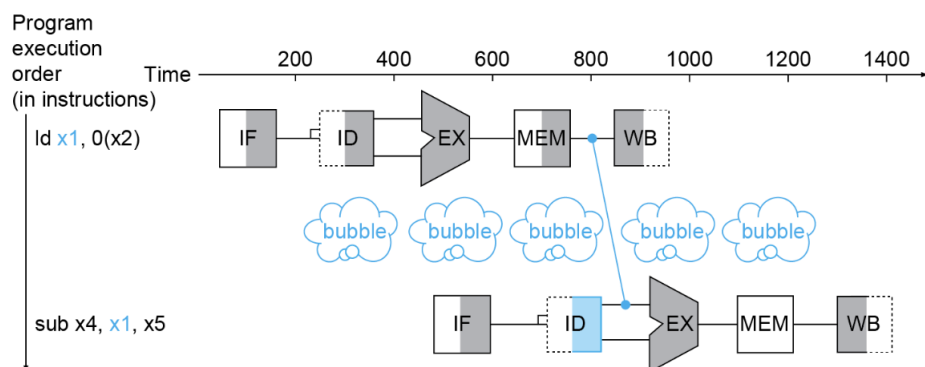
- **Read After Write (RAW)**: il più comune. L'istruzione 2 legge un registro che l'istruzione 1 sta scrivendo.
- **Write After Read (WAR)** e **Write After Write (WAW)**: più rari, si verificano in pipeline con esecuzione fuori ordine.

Soluzioni.

- **Forwarding (bypassing):** il dato appena calcolato in EX o MEM viene *anticipato* allo stadio ID/EX dell'istruzione dipendente, senza attendere il WB. Risolve la maggior parte dei RAW.



- **Stall:** se il forwarding non basta (es. ld seguito da uso immediato del dato caricato), si inserisce una *bolla* (ciclo vuoto) per attendere.



- **Riordinamento software:** il compilatore può spostare istruzioni indipendenti tra quelle dipendenti, riducendo gli stall.

Il forwarding richiede mux aggiuntivi e logica di rilevamento delle dipendenze, ma è essenziale per mantenere alto il throughput.

4.4.3 Hazard di controllo

Si verifica quando il flusso di esecuzione cambia (branch, jump), ma la pipeline ha già caricato le istruzioni successive. Il problema: la decisione sul branch arriva solo in **ID** o **EX**, ma nel frattempo sono state già caricate istruzioni dal percorso sequenziale.

Esempio.

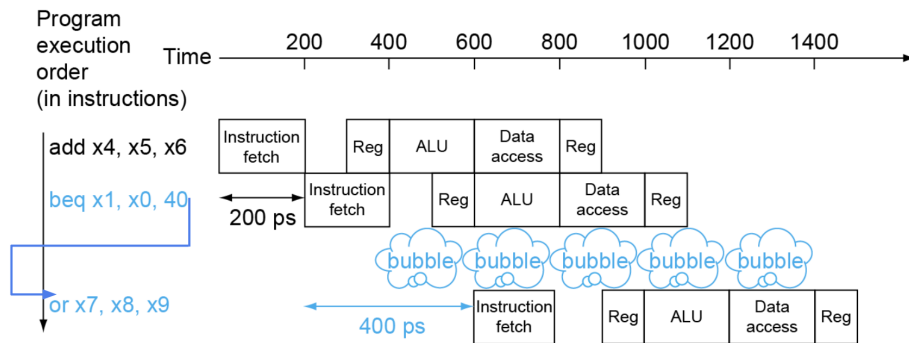
```

beq x1, x2, target    # branch valutato in ID (ciclo 2)
add x3, x4, x5         # già in IF durante ciclo 1
sub x6, x7, x8         # già caricata se branch non preso
  
```

Se il branch è preso, le istruzioni `add` e `sub` devono essere *annullate* (flush).

Soluzioni.

- **Stall:** attendere la risoluzione del branch prima di caricare la prossima istruzione. Semplice ma lento: spreca 1-3 cicli per ogni branch.

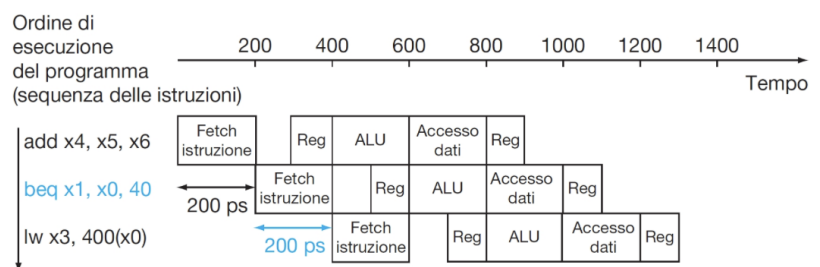


- **Delayed branch:** tecnica RISC classica (usata in MIPS originale): l'istruzione subito dopo il branch viene *sempre eseguita*, indipendentemente dall'esito. Il compilatore riempie lo slot con un'istruzione utile o con `nop`. RISC-V non usa questa tecnica.
- **Anticipare la decisione:** in una pipeline RISC-V, per ridurre il penalty dei branch, si può aggiungere hardware nello stadio **ID** per:
 - calcolare il target del branch ($PC + \text{offset}$) tramite un sommatore dedicato;
 - confrontare i registri (es. `rs1 == rs2` o `rs1 > rs2`) con logica combinatoria.

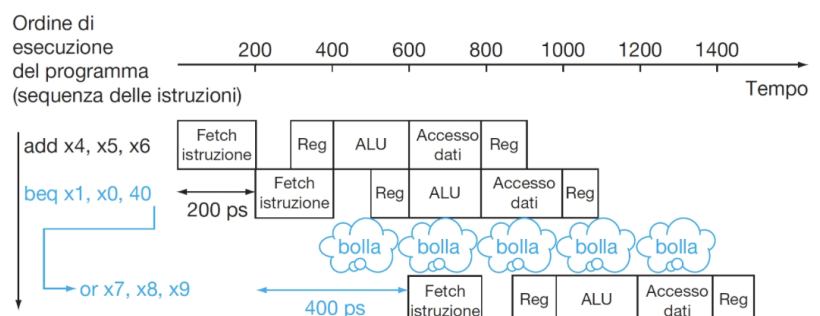
In questo modo, la decisione sul branch è disponibile già alla fine di ID (ciclo 2), riducendo il penalty a **1 solo ciclo** invece di 2-3. Se il branch è preso, si annulla solo l'istruzione in IF; se non è preso, si prosegue normalmente.

- **Branch prediction:** il processore *predice* se il branch sarà preso o meno, e carica speculativamente le istruzioni dal percorso previsto. Se la predizione è corretta, il throughput aumenta. Se la predizione è errata, si deve pulire la pipeline e ricaricare le istruzioni corrette.

Try to fetch the next instruction (branch not taken)



If wrong, flush the pipeline



- **Static branch prediction:** decisione fissa presa a compile-time o per default hardware. Esempi:
 - * *Predict not taken:* assume sempre che il branch non venga preso (continua sequenzialmente).
 - * *Predict taken:* assume sempre che il branch venga preso.
 - * *Backward taken, forward not taken:* i branch all'indietro (loop) sono predetti come presi, quelli in avanti (if-then-else) come non presi.

Semplice ed economico.

- **Dynamic branch prediction:** usa la storia delle esecuzioni passate per predire i branch futuri. Implementato con hardware dedicato (es. *Branch History Table*). Sulla tabella si tiene traccia degli indirizzi dei branch e del loro esito.

Quello che si fa è controllare la tabella e aspettarsi il medesimo risultato. Posso usare, per esempio, un sistema a 1 o 2 bit. Con 1 bit ricorda il risultato dell'ultima esecuzione del branch.

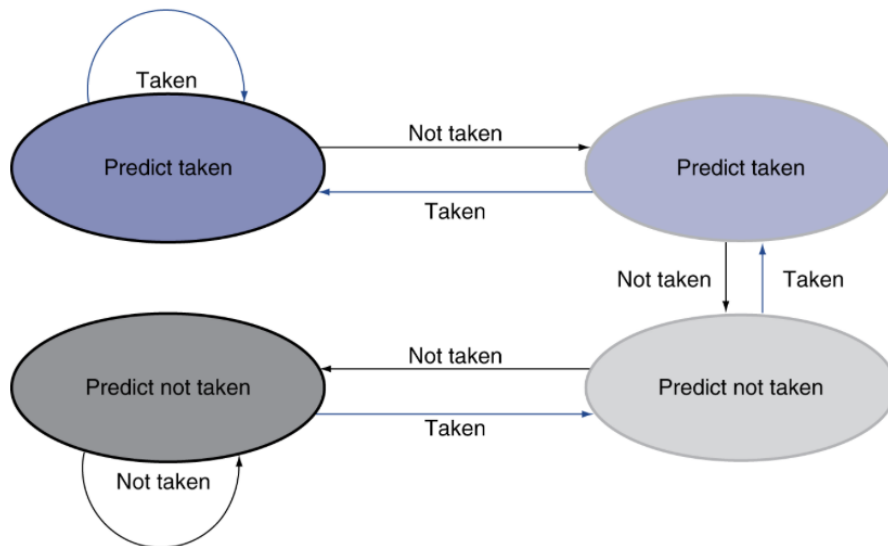


Figura 3: diagramma a stati di un predittore a 2 bit.

Osservazione 4.2. Per implementare pipeline necessitiamo di avere dei registri intermedi tra i vari stadi. Questi registri sono chiamati *pipeline registers*.

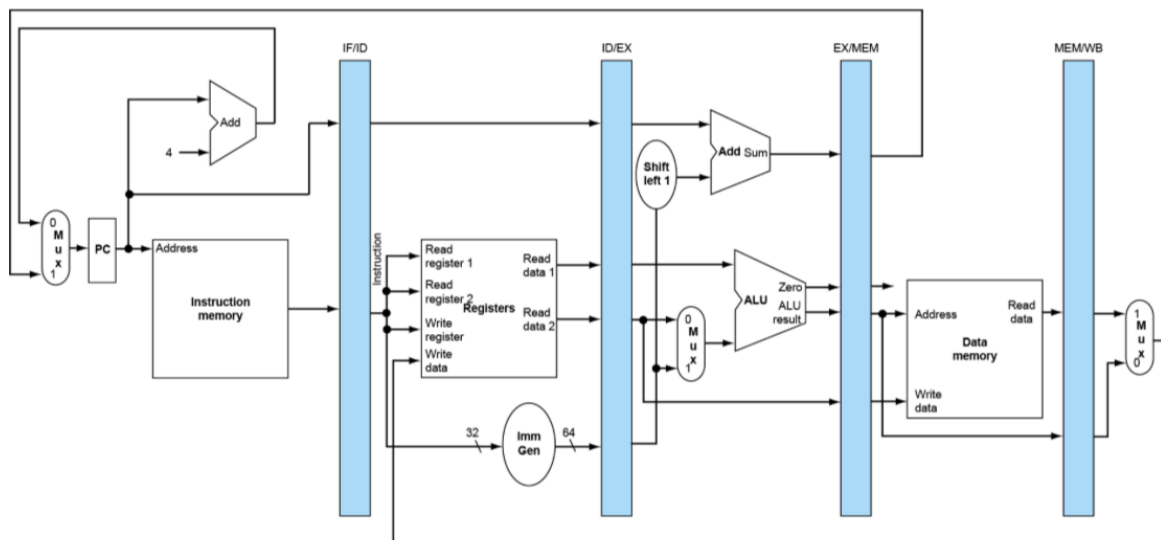


Figura 4: pipeline registers.

4.5 Instruction Level Parallelism

Si è già discusso il fatto che grazie al **pipelining** riusciamo a ad eseguire multiple istruzioni in parallelo. Enunciamo due modi per aumentare l'**ILP**

1. **Pipeline più profonda:** si suddivide l'esecuzione di un'istruzione in un numero maggiore di fasi, ognuna delle quali compie meno lavoro. Così ogni fase può essere completata più velocemente e il processore può avere un ciclo di clock più corto, quindi lavorare a una frequenza più alta.
2. **Pipeline più larga (multiple issues):** più istruzioni nello stesso ciclo di clock. Questo si ottiene replicando i percorsi della pipeline, così che il processore possa gestire diverse istruzioni in parallelo.

Esempio 4.1. Processore con 4 way pipeline ha un peak $CPI = 0.25 \Rightarrow$ peak $IPC = 4$. Se il processore va a 4GHz allora avrà un throughput di 16 BIPS (Billion Instructions Per Second).

Nella pratica le performance saranno peggiori a causa della dipendenza tra dati. Si definiscono 2 tipi di **multiple issue**:

- **statico:** il compilatore corregge le dipendenze e parse le istruzioni in maniera che la cpu esegua senza rischi.
- **dinamico:** la cpu esegue le istruzioni e sceglie quelle che possono essere eseguite in ogni ciclo. Il compilatore riordina le istruzioni mentre la CPU può risolvere gli hazard runtime.

Si parla di **speculazione** quando si esegue un'istruzione in anticipo, prima di sapere se sarà effettivamente necessaria. Se la scelta si rivela corretta, il risultato viene confermato; in caso contrario, si effettua un **rollback** e si esegue l'istruzione alternativa.

La **branch prediction** [4.4.3] è un esempio di **speculation on branch**.

Il compilatore può fare speculazione riordinando le istruzioni (ad esempio anticipando una **load** prima di un **branch**), mentre l'hardware può eseguire in anticipo alcune istruzioni successive e **salvare temporaneamente i risultati in buffer**, fino a verificare se erano effettivamente necessari. Se la speculazione era errata, i buffer vengono semplicemente **svuotati**.

I compilatori raggruppano le istruzioni in **pacchetti** (issue packets - istruzioni che possono essere eseguite in parallelo) ed eventualmente usano delle istruzioni **nop** per evitare hazards da dipendenze.

Esempio 4.2 (RISC-V with Static Dual Issue). Il compilatore raggruppa le istruzioni in **pacchetti da due istruzioni** (*two-issue packets*), ciascuno contenente una istruzione **ALU/branch** e una **load/store**. Le istruzioni di un pacchetto vengono eseguite in parallelo, con un allineamento di 64 bit (32 bit per ciascuna istruzione). Se una delle due posizioni è inutilizzata, viene riempita con una **nop**.

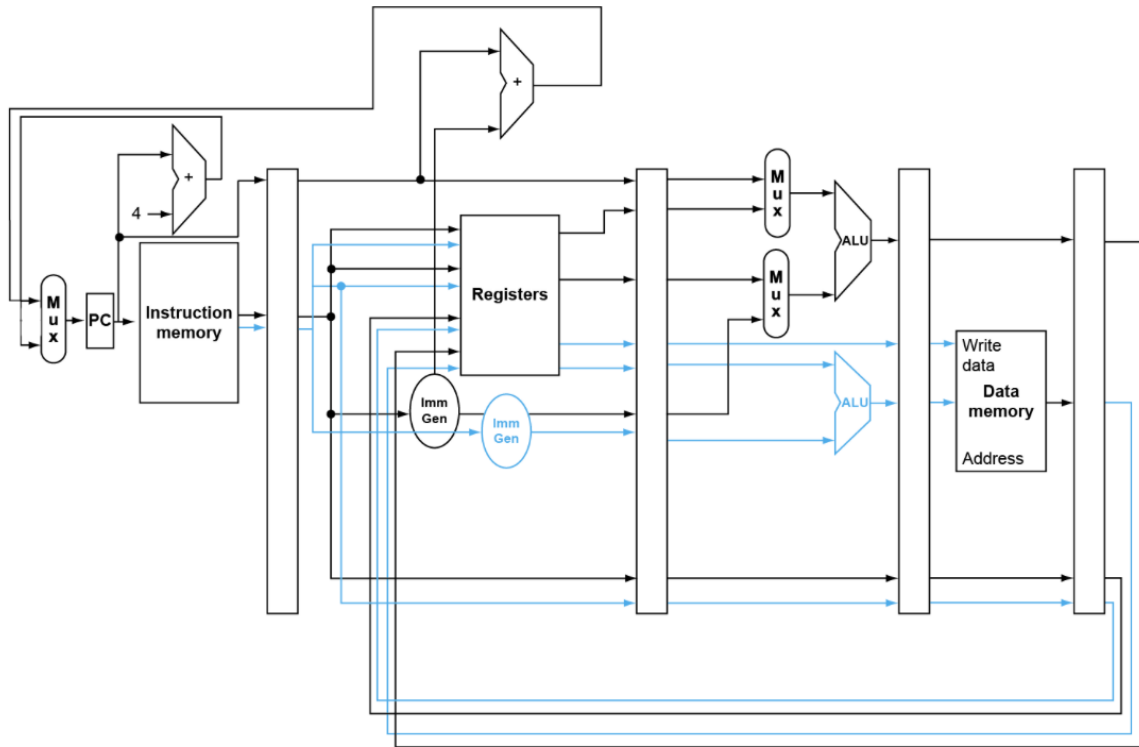


Figura 5: RISC-V with Static Dual Issue.

Hazard nel Dual-Issue

L'esecuzione parallela introduce nuovi problemi rispetto al single-issue:

- **EX data hazard:** Nel single-issue il forwarding evitava stall. Nel dual-issue non si può usare il risultato dell'ALU per una load/store nello stesso pacchetto.

Esempio:

```
add x10, x0, x1
ld  x2, 0(x10)      # non può usare x10 appena calcolato!
```

Il compilatore deve **dividere in due pacchetti**, causando di fatto uno stall.

- **Load-use hazard:** Latenza di una load sempre pari a un ciclo, ma ora impatta **due istruzioni** (entrambe nel pacchetto successivo).
- **Scheduling più aggressivo:** Il compilatore deve riordinare il codice per massimizzare il parallelismo evitando hazard, compito più complesso rispetto al single-issue.

Esempio di Scheduling

Consideriamo un semplice loop che somma uno scalare agli elementi di un array:

```
Loop: ld  x31, 0(x20)    // carica elemento array
      add x31, x31, x21  // somma scalare
      sd  x31, 0(x20)    // salva risultato
```

```

addi x20, x20, -8    // decrementa pointer
blt  x22, x20, Loop // continua se x22 < x20

```

Lo scheduling per dual-issue produce:

	ALU/branch	Load/store	cycle
Loop:	nop	ld x31, 0(x20)	1
	addi x20, x20, -8	nop	2
	add x31, x31, x21	nop	3
	blt x22, x20, Loop	sd x31, 8(x20)	4

$IPC = 5/4 = 1.25$ (contro un IPC ideale di 2). Le `nop` riducono le prestazioni.

Loop Unrolling

Per migliorare l'IPC, si può srotolare il loop replicando il corpo 4 volte. Questo espone più parallelismo e riduce l'overhead di controllo del loop.

Vantaggi:

- Più istruzioni indipendenti da schedulare in parallelo
- Meno branch (uno ogni 4 iterazioni invece che ogni iterazione)

Svantaggi:

- Servono più registri (**register renaming**)
- Codice più grande

Dopo unrolling e scheduling ottimizzato:

	ALU/branch	Load/store	cycle
Loop:	addi x20, x20, -32	ld x28, 0(x20)	1
	nop	ld x29, 24(x20)	2
	add x28, x28, x21	ld x30, 16(x20)	3
	add x29, x29, x21	ld x31, 8(x20)	4
	add x30, x30, x21	sd x28, 32(x20)	5
	add x31, x31, x21	sd x29, 24(x20)	6
	nop	sd x30, 16(x20)	7
	blt x22, x20, Loop	sd x31, 8(x20)	8

$IPC = 14/8 = 1.75$ - miglioramento del 40% rispetto allo scheduling base, ma al costo di usare 4 registri (x28-x31) invece di uno solo e codice più lungo.

Stalli dovuti al branch possono dunque influire considerevolmente sul throughput. I compilatori per architetture VLIW (Very Long Instruction Word) risultano notevolmente più complessi poiché devono:

- Gestire le dipendenze tra dati (*data dependencies*) e schedulare le istruzioni per evitare hazard.
- Ordinare le istruzioni in maniera ottimale per minimizzare i conflitti di accesso in memoria (*memory access conflicts*).
- Gestire il fatto che l'architettura è suscettibile a stalli se un'istruzione richiede un tempo non predicibile (es. cache miss o latenza di memoria variabile), bloccando l'intero pacchetto di istruzioni.

4.6 Dynamic Multiple Issue

Le architetture moderne superano i limiti del VLIW statico adottando un approccio **Superscalare** con **Dynamic Multiple Issue**. In questi processori:

- La CPU decide dinamicamente quante istruzioni inviare in esecuzione (*issue*) in ogni ciclo (0, 1, 2, ...), in base alle risorse disponibili e alle dipendenze.
- Si evitano hazard strutturali e sui dati a livello hardware, senza dipendere esclusivamente dallo scheduling statico del compilatore.
- Il compilatore può comunque ottimizzare il codice, ma la semantica e la correttezza dell'esecuzione sono garantite dall'hardware, permettendo di eseguire codice compilato per versioni precedenti dell'architettura.

4.7 Dynamic Pipeline Scheduling

Per evitare stalli inutili quando un'istruzione è bloccata (ad esempio in attesa di un dato dalla memoria), le CPU moderne implementano il **Dynamic Pipeline Scheduling**, che consente l'**esecuzione fuori ordine** (*out-of-order execution*).

L'idea è dividere la pipeline in tre fasi logiche:

1. **In-order Issue**: le istruzioni vengono prelevate e decodificate in ordine.
2. **Out-of-order Execute**: le istruzioni vengono eseguite non appena i loro operandi sono disponibili e l'unità funzionale è libera, anche se istruzioni precedenti sono in attesa.
3. **In-order Commit**: i risultati vengono scritti nei registri o in memoria rigorosamente in ordine di programma, per mantenere la consistenza e gestire correttamente le eccezioni.

Esempio 4.3. Consideriamo la sequenza:

```
ld    x31, 20(x21)
add   x1,  x31, x2    # attende x31 da ld
sub   x23, x23, x3    # indipendente
andi  x5,  x23, 20    # dipende da sub
```

In una pipeline statica, la `add` stallerebbe la pipeline in attesa della `ld`, bloccando anche la `sub`. Con lo scheduling dinamico, la `sub` può essere avviata ed eseguita mentre la `add` è in attesa, nascondendo la latenza della memoria.

Organizzazione di una CPU a scheduling dinamico Per supportare questo meccanismo, l'architettura si arricchisce di componenti specifici:

- **Reservation Stations**: buffer posti prima delle unità funzionali che contengono le istruzioni in attesa di esecuzione e i loro operandi (se già disponibili).
- **Functional Units**: unità esecutive (ALU intere, FP, Load/Store) che operano in parallelo.
- **Commit Unit / Reorder Buffer (ROB)**: un buffer che riordina i risultati delle istruzioni eseguite fuori ordine, garantendo che le scritture sui registri architetturali avvengano in ordine.

Register Renaming Il **Register Renaming** è una tecnica utilizzata per eliminare le dipendenze fittizie (come *Write After Write* e *Write After Read*) che sorgono dal riutilizzo degli stessi registri architetturali da parte del compilatore.

Poiché il set di registri visibili al programmatore è limitato (es. 32 registri in RISC-V), il compilatore è costretto a riutilizzare spesso gli stessi nomi (es. `x1`) per variabili temporanee diverse in punti diversi del programma. Se l'hardware eseguisse le istruzioni strettamente in ordine, questo non sarebbe un problema. Tuttavia, in una CPU che esegue fuori ordine, il riutilizzo di un nome può bloccare istruzioni che in realtà sono indipendenti tra loro.

Con il register renaming, la CPU mappa dinamicamente i "nomi" dei registri architetturali (quelli usati nell'istruzione) a un insieme più ampio di registri fisici interni. In questo modo:

- Ogni nuova scrittura su un registro (es. `addi x1, ...`) viene indirizzata a una nuova locazione fisica.
- Le letture successive a quella scrittura leggeranno dalla nuova locazione.
- Le istruzioni precedenti che leggevano il "vecchio" valore di `x1` continueranno a leggere dalla vecchia locazione fisica, senza essere influenzate dalla nuova scrittura.

Questo permette di eseguire in parallelo istruzioni che usano lo stesso registro solo per coincidenza di nome, sbloccando il potenziale parallelismo del processore.

Il dynamic scheduling permette di alleggerire il lavoro del compilatore e contemporaneamente di gestire stalli non predicibili. Implementazioni differenti di un ISA hanno latenze ed hazard differenti.

Efficacia del Multiple Issue Nonostante le tecniche avanzate di scheduling dinamico e speculazione, il passaggio da processori scalari a superscalari a n -vie non garantisce un aumento lineare delle prestazioni. I limiti principali sono:

- **Dipendenze reali:** Alcune dipendenze di dati sono intrinseche all'algoritmo e non possono essere eliminate, limitando il parallelismo estraibile (ILP).
- **Finestra di issue limitata:** La CPU può analizzare solo un numero finito di istruzioni alla volta; se in quella "finestra" non ci sono istruzioni indipendenti, le unità funzionali restano inutilizzate.
- **Collo di bottiglia della memoria:** Anche con pipeline perfette, i ritardi dovuti all'accesso in memoria (cache miss) e la banda limitata impediscono di mantenere le pipeline sempre piene.
- **Costo della speculazione:** La speculazione errata (es. branch prediction sbagliato) consuma energia e risorse per eseguire istruzioni che verranno poi scartate.

5 Istruzioni vettoriali (SIMD)

La SIMD (Single Instruction Multiple Data) esegue 8 operazioni in doppia precisione in parallelo con AVX-512. Ogni registro SIMD può ospitare diversi tipi di dati.

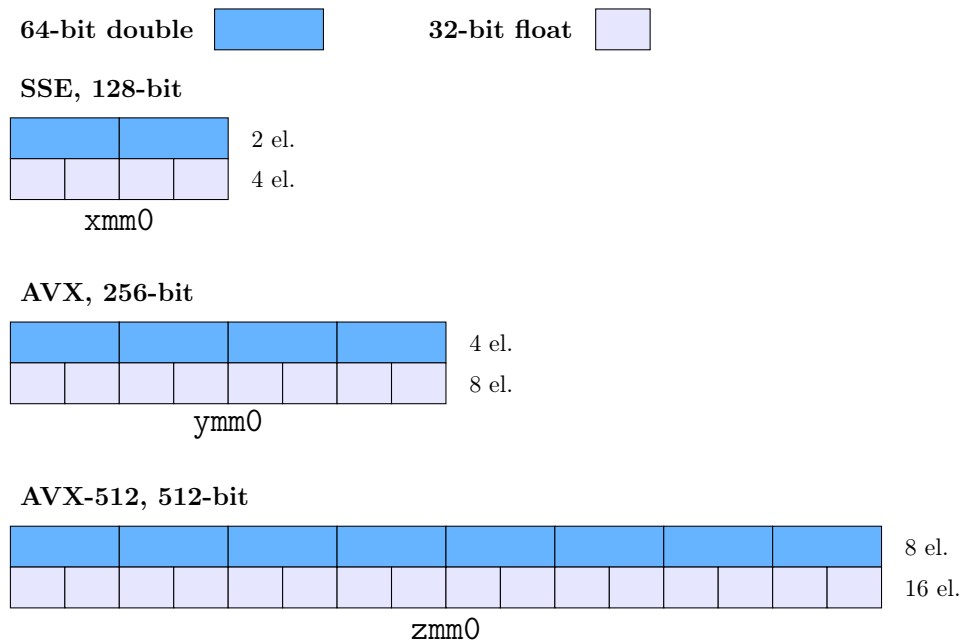


Figura 6: Evoluzione della capacità dei registri SIMD (SSE, AVX, AVX-512).

Si dice **vectorization** quando un programma è riscritto in modo che possa sfruttare le istruzioni SIMD. Inizialmente venivano scritte in Assembly, poi sono state scritte delle funzioni per mappare le istruzioni SIMD su linguaggi di più alto livello. Oggi molti compilatori supportano **auto-vectorization** e possono scrivere direttamente codice SIMD.

Esempio 5.1 (Matrix Multiply Unrolled). Il codice seguente mostra una versione ottimizzata della moltiplicazione di matrici che utilizza **loop unrolling** (con fattore 4) insieme alle intrinseche AVX-512. Rispetto alla versione base (che elabora un solo vettore alla volta), questa variante processa 32 elementi per iterazione (4 vettori da 8 double), riducendo l'overhead del ciclo e aumentando l'Instruction Level Parallelism (ILP).

```
#define UNROLL 4
void dgemv_unrolled(int n, double* A, double* B, double* C) {
    // i incrementa di 32 (4 * 8) elementi alla volta
    for (int i = 0; i < n; i += UNROLL * 8) {
        for (int j = 0; j < n; ++j) {
            __m512d c[UNROLL];
            // Carica 4 vettori di C (C[j][i]...C[j][i+31])
            for (int r = 0; r < UNROLL; r++)
                c[r] = _mm512_load_pd(C + (i + 8 * r) + j * n);

            for (int k = 0; k < n; k++) {
                // Broadcast dello scalare B[j][k]
                __m512d bb = _mm512_broadcastsd_pd(_mm_load_sd(B + j * n + k));
                // FMA su 4 accumulatori indipendenti
                for (int r = 0; r < UNROLL; r++) {
                    __m512d a = _mm512_load_pd(A + k * n + (i + 8 * r));
                    c[r] = _mm512_fmadd_pd(a, bb, c[r]);
                }
            }
            // Salva i risultati
            for (int r = 0; r < UNROLL; r++)
                _mm512_store_pd(C + (i + 8 * r) + j * n, c[r]);
        }
    }
}
```

6 Le memorie

Vedremo come vengono implementate le memorie cache e le memorie più grandi. Ci riferiamo al **principio di località**.

Definizione 6.1 (Principio di località). Il principio di località fa riferimento a due concetti:

- **Località temporale:** dati a cui si ha avuto accesso recentemente è più probabile che siano usati nuovamente in futuro.
- **Località spaziale:** dati vicini a quelli a cui si ha avuto accesso recentemente è più probabile che siano usati in futuro.

Ne consegue che possiamo istituire una **gerarchia di memorie** e in generale:

- Tutto viene permanentemente memorizzato su **disco** (memoria secondaria).
- I dati a cui si accede (e quelli adiacenti, per località spaziale) vengono copiati dal disco alla memoria **DRAM** (Main Memory), che è più piccola ma più veloce.
- I dati usati più di recente vengono ulteriormente copiati dalla DRAM alla memoria **SRAM** (Cache), che è interna o vicina alla CPU, molto piccola e velocissima.

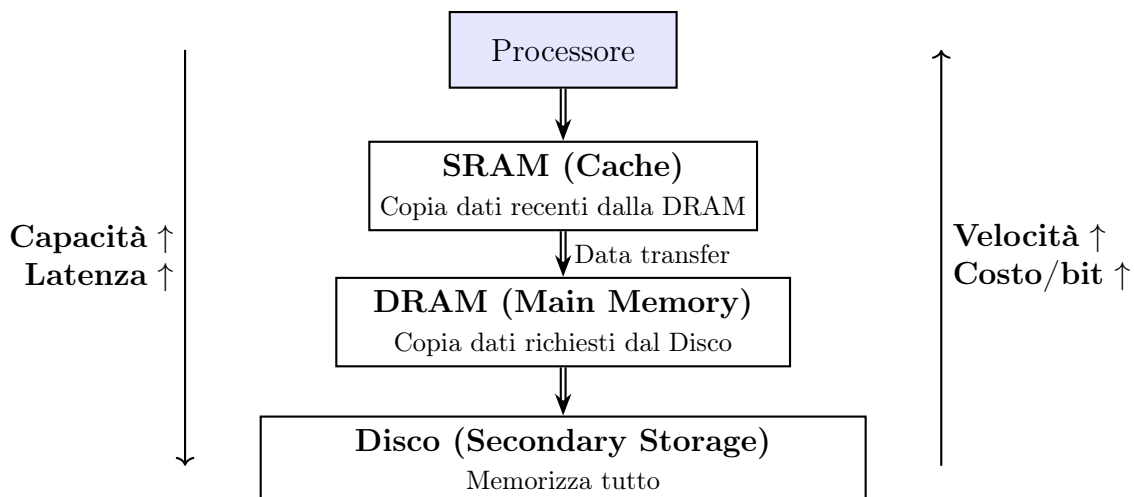


Figura 7: Gerarchia di memoria e flusso dei dati.

6.1 Terminologia della Gerarchia

Il trasferimento dei dati tra i vari livelli avviene per unità di copiatura chiamate **blocchi** (o *linee*). Un blocco può contenere multiple words.

- **Hit:** Si verifica quando il dato richiesto dal processore è presente nel livello superiore (ad esempio in Cache).
 - L'accesso è soddisfatto rapidamente.
 - **Hit ratio:** rapporto tra il numero di hit e il numero totale di accessi.
- **Miss:** Si verifica quando il dato richiesto **non** è presente nel livello superiore.
 - Il blocco contenente il dato deve essere copiato dal livello inferiore (più lento).
 - **Miss penalty:** il tempo necessario per recuperare il blocco dal livello inferiore e consegnarlo al processore.
 - **Miss ratio:** rapporto tra i miss e gli accessi totali ($1 - \text{Hit ratio}$).

6.2 Tecnologia DRAM

I dati sono salvati in un condensatore. Un singolo transistor viene usato per accedere alla cella del dato (condensatore). Questo deve essere periodicamente ricaricato per evitare che il dato si perda. Notare anche che leggere un contenuto scarica il condensatore, per cui deve essere riscritto. La DRAM può leggere/scrivere un'intera riga di dati (**burst mode**), visto che l'accesso a una riga è dispendioso, ma accedere ai dati consecutivi nella riga è molto meno costoso (i.e., diminuisce latenza).

Esistono anche SDRAM (sincronizzate con il clock del processore) e DDR (Double Data Rate) che permettono di incrementare il bandwidth.

6.3 Memoria Cache

A livello di gerarchie di memorie la cache è la più vicina al processore (la più veloce). La cache memorizza copie di dati dalla memoria principale che sono stati recentemente utilizzati. Quando il processore richiede un dato, la cache verifica se lo possiede già (hit) o se deve recuperarlo dalla memoria principale (miss).

Direct-Mapped Cache La forma più semplice di cache è la **direct-mapped cache**, dove ogni blocco di memoria può essere memorizzato in una sola posizione della cache. La posizione è determinata dall'indirizzo del blocco in memoria.

L'indirizzo di memoria viene suddiviso in tre parti:

- **Offset**: identifica il byte specifico all'interno del blocco (i bit meno significativi).
- **Index**: seleziona quale riga della cache utilizzare.
Si calcola come $(Block\ Address) \bmod (\#\ Blocks\ in\ cache)$.
- **Tag**: i bit rimanenti dell'indirizzo, memorizzati nella cache per identificare quale blocco di memoria è presente in quella posizione.

Ogni riga della cache contiene anche un **Valid bit** che indica se il dato presente è valido (1) o se la riga è vuota (0). Inizialmente tutti i valid bit sono a 0.

Indirizzo di Memoria (32 bit):

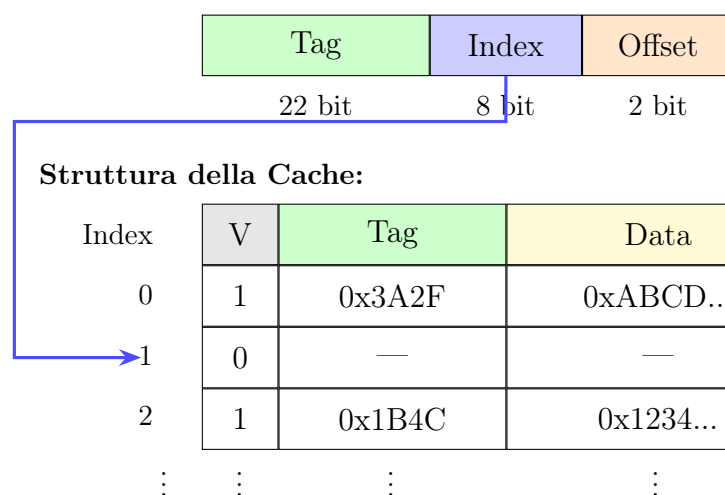


Figura 8: Struttura di una direct-mapped cache e organizzazione dell'indirizzo.

Come funziona l'accesso alla cache Quando il processore richiede un dato:

1. L'**index** seleziona la riga della cache da controllare.
2. Il **valid bit** viene verificato: se è 0, si ha un miss immediato.

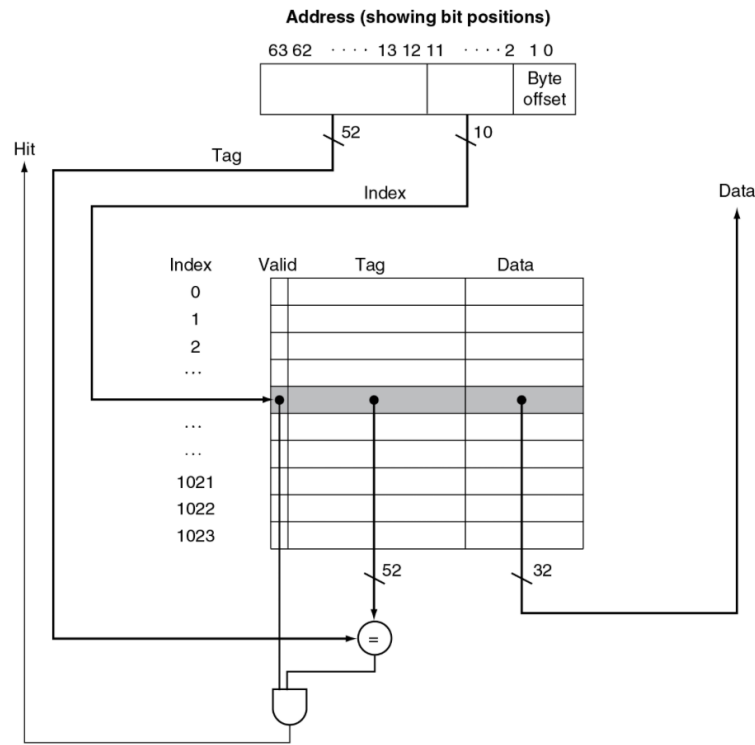


Figura 9: Organizzazione indirizzi (64 bit)

3. Se il valid bit è 1, il **tag** memorizzato viene confrontato con il tag dell'indirizzo richiesto.

- Se i tag corrispondono: **hit**, il dato viene letto usando l'offset.
- Se i tag non corrispondono: **miss**, il blocco deve essere recuperato dalla memoria principale.

Il vantaggio della direct-mapped cache è la sua semplicità: non serve decidere *dove* mettere un blocco, la posizione è determinata automaticamente dall'indirizzo. Lo svantaggio è che blocchi con lo stesso index si sostituiscono a vicenda, anche se ci sono altre righe libere nella cache (**conflict miss**).

Esempio 6.1. Consideriamo un architettura di **64 blocchi** (16 byte per blocco). L'indirizzo 1200 mappa a:

- Block address = $\lfloor \frac{1200}{16} \rfloor = 75$
- Block number = $75 \bmod 64 = 11$

Osservazione 6.1 (Sulla dimensione dei blocchi). Aumentare la dimensione dei blocchi tende a ridurre il **miss rate** sfruttando la **località spaziale**: caricando blocchi più grandi, portiamo in cache più dati adiacenti a quello richiesto, che è probabile vengano acceduti a breve.

Tuttavia, in una cache di dimensione fissa, blocchi più grandi implicano un **numero minore di blocchi** (linee) disponibili. Questo comporta:

- Maggiore competizione per le linee, che può incrementare il miss rate (**conflict miss**).
- Rischio di **pollution**: si caricano dati inutili che occupano spazio prezioso.

Inoltre, blocchi più grandi aumentano il **miss penalty** (tempo di trasferimento). Se questo costo supera il beneficio della riduzione dei miss, le prestazioni peggiorano. Per ridurre l'attesa del processore durante il trasferimento di un blocco, si utilizzano due tecniche principali:

- **Early Restart**: il processore riprende l'esecuzione non appena riceve la word richiesta, senza attendere che l'intero blocco sia arrivato. Tuttavia, il trasferimento dalla memoria avviene sequenzialmente: se la word richiesta è alla fine del blocco, il vantaggio è minimo.

- **Requested Word First** (o *Critical Word First*): si richiede alla memoria di inviare **per prima** la word specifica che serve alla CPU (cache). Appena questa arriva, il processore riprende l'esecuzione, mentre il resto del blocco viene caricato successivamente in background.

Cache misses Se si verifica una **cache hit**, la CPU trova i dati (o le istruzioni) nel livello di cache corrente e prosegue senza ritardi. Una **cache miss** invece costringe la CPU a *stallare* la pipeline: il blocco richiesto deve essere recuperato dal livello di memoria successivo (ad es. L2, L3 o RAM). Una volta che il blocco è stato caricato in cache, l'esecuzione può riprendere normalmente.

6.4 Gestione delle Scritture (Writes)

Quando la CPU deve scrivere un dato, la gestione è più complessa rispetto alla lettura, poiché bisogna mantenere la coerenza tra la cache e la memoria principale. Distinguiamo due casi: quando il dato è già in cache (*Write Hit*) e quando non c'è (*Write Miss*).

Politiche su Write Hit Se il blocco da modificare è presente in cache, abbiamo due strategie principali:

1. **Write-Through:** il dato viene scritto **simultaneamente** sia nella cache che nella memoria principale.
 - **Pro:** La memoria è sempre consistente (aggiornata) rispetto alla cache. Implementazione semplice.
 - **Contro:** Le scritture sono lente perché limitate dalla velocità della DRAM (*CPI* aumenta drasticamente).
 - **Soluzione:** Si utilizza un **Write Buffer**. La CPU scrive i dati in questa piccola coda veloce e prosegue l'esecuzione senza attendere la memoria. Lo stallo avviene solo se il buffer si riempie (scritture troppo frequenti).
2. **Write-Back:** il dato viene scritto **solo nella cache**.
 - Il blocco modificato viene segnato con un bit detto **Dirty bit** (segnala che il blocco è stato modificato rispetto alla memoria principale, che dunque ha il dato non aggiornato).
 - La copia in memoria principale avviene **solo quando il blocco viene rimpiazzato** (ovvero espulso dalla cache per far posto a un altro).
 - **Pro:** Molto più veloce (velocità della cache) e riduce il traffico sul bus (più scritture sullo stesso blocco vengono accorpate).
 - **Contro:** La memoria principale non è aggiornata (inconsistenza temporanea) e la gestione hardware è più complessa.

Politiche su Write Miss Se il blocco da scrivere non è in cache, bisogna decidere se portarlo dentro o meno:

- **Write Allocate** (tipico del Write-Back): Il blocco viene prima caricato in cache (fetch) e poi modificato. Si basa sull'idea che, per località temporale, il dato scritto verrà probabilmente riletto o riscritto a breve.
- **No-Write Allocate** (o *Write Around*, tipico del Write-Through): Il dato viene scritto direttamente in memoria principale **senza** caricare il blocco in cache. È utile quando si scrivono grandi quantità di dati che **non verranno rilette subito**.

Average Memory Access Time (AMAT) L'Average Access Time (AMAT) è il tempo medio di accesso alla memoria.

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Esempio 6.2 (Calcolo AMAT). Consideriamo una CPU con clock di 1 ns. Supponiamo che l'accesso alla cache richieda 1 ciclo (hit time), mentre il recupero dalla memoria principale richieda 20 cicli (miss penalty). Con un miss rate della cache istruzioni del 5%, il tempo medio di accesso risulta:

$$\text{AMAT} = 1 + 0.05 \times 20 = 1 + 1 = 2 \text{ ns}$$

Poiché il ciclo di clock è 1 ns, un AMAT di 2 ns implica che, in media, ogni accesso alla memoria costa 2 cicli di clock.

Osservazione 6.2 (Impatto delle prestazioni CPU sulla memoria). *All'aumentare delle prestazioni della CPU, il comportamento della memoria diventa critico:*

- **Aumento performance CPU:** rende il miss penalty più significativo in proporzione.
- **Riduzione CPI base:** aumenta la proporzione di tempo spesa in stalli di memoria.
- **Aumento frequenza di clock:** gli stalli di memoria consumano un numero maggiore di cicli di clock (il tempo assoluto di accesso alla memoria resta uguale, ma "costa" più cicli).

Non è quindi possibile valutare le prestazioni di sistema trascurando il comportamento della cache.

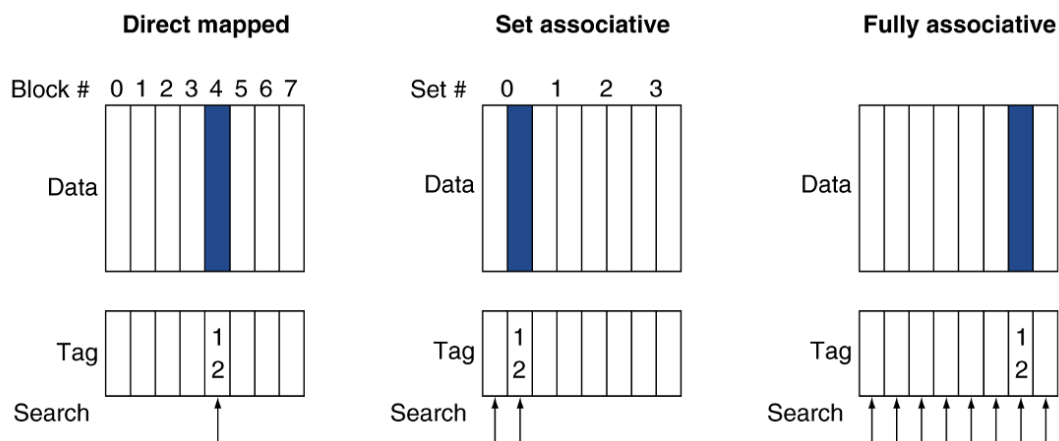
6.5 Memoria Cache Associativa

Fully Associative In una cache completamente associativa, un blocco può essere memorizzato in *qualsiasi* posizione. Questo offre la massima flessibilità, ma richiede di cercare il dato in **tutte** le entry contemporaneamente. Poiché serve un comparatore per ogni riga della cache, questa soluzione è molto costosa.

N-way Set Associative È un compromesso tra direct-mapped e fully associative. La cache è divisa in insiemi (*sets*), ognuno contenente N linee (vie). Un blocco mappa direttamente a un insieme specifico secondo la formula:

$$\text{Index} = (\text{Block Address}) \bmod (\text{Numero di Sets})$$

All'interno dell'insieme selezionato, il blocco può occupare una qualsiasi delle N posizioni. La ricerca avviene in parallelo solo sulle N vie del set, limitando il numero di comparatori necessari a N .



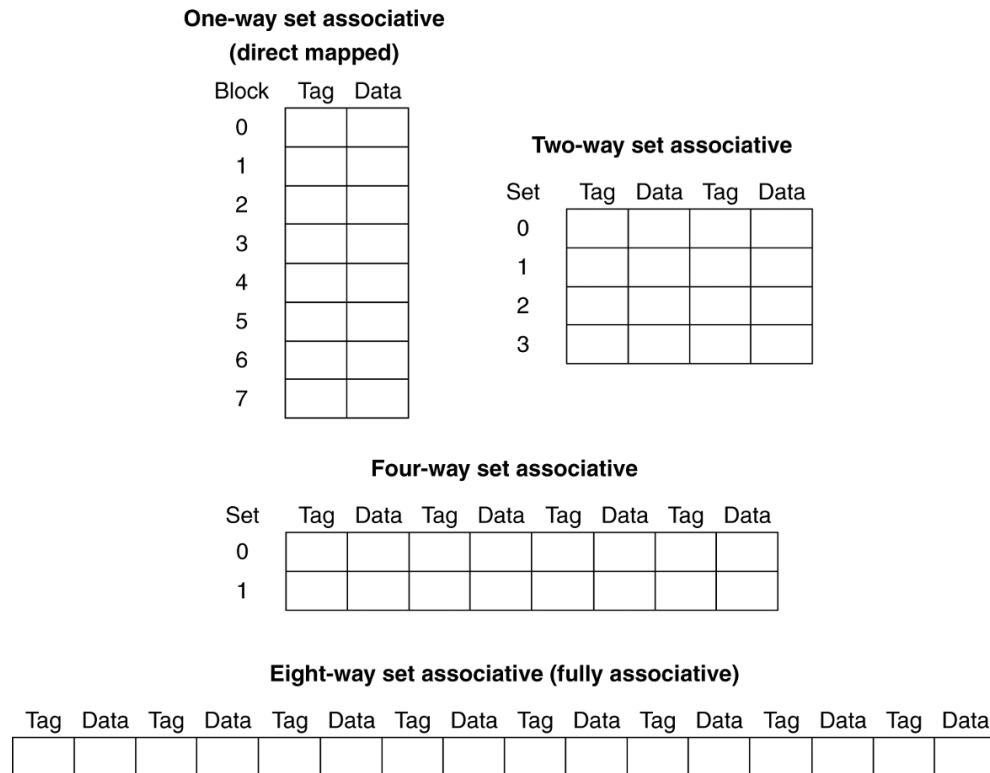


Figura 10: Cache con 8 entries

Esempio 6.3 (Confronto Direct Mapped, Set Associative e Fully Associative). Consideriamo una sequenza di accessi ai blocchi di memoria: **0, 8, 0, 6, 8**. Analizziamo il comportamento per una cache di **4 blocchi** totali, usando tre strategie diverse.

Direct Mapped Ogni blocco mappa su una posizione fissa: $\text{Index} = \text{BlockAddr} \bmod 4$.

Block Addr	Cache Index	Hit/Miss	Contenuto Cache dopo l'accesso			
			0	1	2	3
0	0	Miss	Mem[0]			
8	0	Miss	Mem[8]			
0	0	Miss	Mem[0]			
6	2	Miss	Mem[0]		Mem[6]	
8	0	Miss	Mem[8]		Mem[6]	

Risultato: 5 Miss. Conflitto continuo all'indice 0 tra i blocchi 0 e 8.

2-Way Set Associative La cache ha 2 insiemi (Set 0, Set 1), ciascuno con 2 vie. $\text{Index} = \text{BlockAddr} \bmod 2$.

Block Addr	Cache Index	Hit/Miss	Contenuto Cache dopo l'accesso			
			Set 0		Set 1	
0	0	Miss	Mem[0]			
8	0	Miss	Mem[0]	Mem[8]		
0	0	Hit	Mem[0]	Mem[8]		
6	0	Miss	Mem[0]	Mem[6]		
8	0	Miss	Mem[8]	Mem[6]		

Risultato: 4 Miss, 1 Hit. L'associatività riduce i conflitti permettendo a 0 e 8 di coesistere nel Set 0, finché non arriva il blocco 6 (LRU replacement).

Fully Associative Qualsiasi blocco può andare in qualsiasi posizione. Nessun indice calcolato.

Block Addr	Hit/Miss	Contenuto Cache dopo l'accesso			
		Way 0	Way 1	Way 2	Way 3
0	Miss	Mem[0]			
8	Miss	Mem[0]	Mem[8]		
0	Hit	Mem[0]	Mem[8]		
6	Miss	Mem[0]	Mem[8]	Mem[6]	
8	Hit	Mem[0]	Mem[8]	Mem[6]	

Risultato: 3 Miss, 2 Hit. La piena flessibilità evita tutti i conflict miss di questo esempio.

Osservazione 6.3 (Trade-off dell'associatività). *Aumentare il grado di associatività riduce il Miss Rate (eliminando i conflict miss), ma comporta degli svantaggi:*

- Aumenta l'**Hit Time**: la logica per selezionare il dato tra le varie vie (multiplexer) introduce ritardo.
- Aumenta la complessità hardware e il consumo energetico (più comparatori attivi simultaneamente).

Replacement Policy Quando si verifica un miss in una cache associativa (fully o set-associative), bisogna decidere quale blocco esistente rimuovere per far spazio al nuovo.

- **Direct Mapped**: Non c'è scelta. Ogni blocco mappa su un'unica posizione, quindi l'entry esistente viene sovrascritta obbligatoriamente.
- **Set Associative / Fully Associative**: Si può scegliere quale via del set rimpiazzare.
 - Si preferisce sempre rimpiazzare un'entry **non valida** (vuota), se presente.
 - Se tutte le entry del set sono valide, serve una politica per scegliere la "vittima".

Le politiche più comuni sono:

Least Recently Used (LRU) Sceglie il blocco inutilizzato da più tempo.

- Sfrutta la *località temporale*: se un dato è stato usato di recente, è probabile che serva ancora.

Random Sceglie a caso un blocco candidato.

- Molto semplice da implementare in hardware (nessuna memoria di stato aggiuntiva).
- Per associatività elevate, le prestazioni sono sorprendentemente vicine a LRU.

I processori moderni presentano 3 livelli di cache:

- **L1**: Cache più vicina al processore, molto piccola e veloce.
- **L2**: Cache più lontana dal processore, più grande e più lenta
- **L3**: Cache più lontana dal processore, ancora più grande e ancora più lenta.

E poi si considera la **main memory** (RAM) da cui risolvere i miss.

Esempio 6.4 (Cache Multilivello vs Singolo Livello). Confrontiamo le prestazioni (CPI) di un sistema con solo cache L1 rispetto a uno con cache multilivello (L1 + L2).

Dati comuni:

- Base CPI = 1, Clock = 4 GHz
- Miss rate L1 (per istruzione) = 2%
- Tempo accesso Main Memory = 100 ns

Caso 1: Solo Cache Primaria (L1)

- Miss penalty (verso RAM) = $100 \text{ ns} / 0.25 \text{ ns} = 400$ cicli.
- **Effective CPI** = $1 + 0.02 \times 400 = 1 + 8 = 9$.

Caso 2: Aggiunta Cache L2

- Access time L2 = 5 ns.
- Global miss rate (verso RAM) = 0.5%.
- Miss penalty L1 (verso L2) = $5 \text{ ns} / 0.25 \text{ ns} = 20$ cicli.
- Miss penalty L2 (verso RAM) = 400 cicli (come prima).

Calcolo del CPI:

$$\text{CPI} = \text{Base CPI} + (\text{Miss Rate}_{L1} \times \text{Penalty}_{L1}) + (\text{Global Miss Rate} \times \text{Penalty}_{L2})$$

$$\text{CPI} = 1 + (0.02 \times 20) + (0.005 \times 400) = 1 + 0.4 + 2 = \mathbf{3.4}$$

Conclusione: L'aggiunta della cache L2 ha migliorato le prestazioni di $9/3.4 \approx \mathbf{2.6}$ volte.

Viene da sé che la L1 punta ad abbassare l'hit time, mentre la L2 punta ad abbassare il miss rate (evita accesso RAM).

6.6 Interazioni con CPU Avanzate e Software

CPU Out-of-Order Nelle CPU **out-of-order**, un miss non blocca tutto: le istruzioni indipendenti proseguono, mascherando parte della latenza. L'impatto reale dipende quindi dal parallelismo (ILP) disponibile nel codice.

Interazioni con il Software Le prestazioni dipendono dai pattern di accesso alla memoria: la scelta dell'algoritmo e le ottimizzazioni del compilatore (es. migliorare la località dei dati) sono determinanti per ridurre i miss.

Esempio 6.5 (Confronto Radix Sort vs Quicksort). Sebbene il **Radix Sort** abbia complessità inferiore ($O(N)$) rispetto al **Quicksort** ($O(N \log N)$), nella pratica risulta spesso più lento.

La causa è il **Memory Access Pattern**: il Radix Sort accede alla memoria in modo sparso, generando un alto numero di **cache miss**. Il Quicksort, lavorando su array contigui, sfrutta meglio la località spaziale, compensando il maggior numero di istruzioni con un hit rate superiore.

Esempio 6.6 (Ottimizzazione DGEMM con Blocking). Il prodotto matriciale (DGEMM) $C = A \times B$ può essere drasticamente velocizzato migliorando l'uso della cache.

- **Versione Naive:** esegue tre loop annidati. Se le matrici sono grandi, i dati vengono continuamente caricati e scaricati dalla cache (basso riutilizzo temporale), causando un collo di bottiglia sulla memoria.
- **Blocked Version (Tiling):** divide le matrici in sottomatrici (blocchi) che entrano interamente in cache. In questo modo si massimizza il riutilizzo dei dati caricati in cache prima di passare al blocco successivo.

Le tecniche moderne (come in AVX) combinano il blocking con il parallelismo a livello di istruzione (SIMD), offrendo uno speedup significativo.

Sources of Misses Possiamo classificare i cache miss in tre categorie principali:

- **Compulsory misses** (o Cold Start): avvengono ai primi accessi a un blocco. Non possono essere evitati aumentando la dimensione della cache, ma solo aumentando la dimensione del blocco (prefetching spaziale).
- **Capacity misses**: avvengono quando la cache è piena e non può contenere tutti i blocchi necessari al programma durante la sua esecuzione. Un blocco viene rimpiazzato e poi richiesto di nuovo. Si riducono aumentando la dimensione della cache.
- **Conflict misses** (o Collision): avvengono nelle cache *non* fully associative. Più blocchi competono per lo stesso set (o indice), costringendo a rimpiazzare un blocco anche se la cache non è globalmente piena. Si riducono aumentando l'associatività.

Cache Design Trade-offs Ogni scelta progettuale comporta dei compromessi tra miss rate, latenza (hit time) e costi.

Design Change	Effect on Miss Rate	Negative Performance Effect
↑ Cache Size	↓ Capacity misses	Aumenta l'access time (più logica di indirizzamento)
↑ Associativity	↓ Conflict misses	Aumenta l'access time (mux delay) e complessità hardware
↑ Block Size	↓ Compulsory misses	Aumenta il Miss Penalty (più dati da trasferire)

7 Processori Multi-Core e Cache Coherence

Quando passiamo da un sistema single-core a uno multi-core, sorge il problema della **coerenza della cache**. Poiché ogni core ha la propria cache privata ma condivide la memoria, è possibile che due core abbiano copie diverse dello stesso dato.

7.1 Snooping Protocols

I protocolli di *snooping* sono i più diffusi nei sistemi multicore connessi a bus condiviso. L'idea è che ogni cache monitora costantemente il bus per rilevare le transazioni di memoria effettuate dagli altri core.

Write Invalidate Protocol È l'approccio più comune:

- Per leggere un dato, il core lo carica nella sua cache (stato *Shared*).
- Quando un core vuole **scrivere** su un dato, deve prima ottenere l'accesso esclusivo. Invia un messaggio di **Invalidate** sul bus.
- Tutte le altre cache che hanno una copia di quel dato lo marcano come **Invalido**.
- Se un altro core prova successivamente a leggere quel dato, troverà un Miss e dovrà farselo inviare dal core che lo ha modificato (o dalla memoria).

Esempio 7.1 (Invalidating Snooping). Consideriamo due CPU (A e B) e una variabile X (valore iniziale 0 in memoria).

CPU Activity	Bus Activity	Cache A	Cache B	Mem
		-	-	0
CPU A reads X	Cache miss per X	0	-	0
CPU B reads X	Cache miss per X	0	0	0
CPU A writes 1 to X	Invalidate for X	1	-	0
CPU B reads X	Cache miss per X	1	1	1

Nota: Al passo 4, quando A scrive, la copia in B viene invalidata. Al passo 5, B prova a leggere, trova un miss e ottiene il valore aggiornato (1) fornito dalla cache di A (o tramite write-back in memoria).

7.2 False Sharing

Un effetto collaterale indesiderato della coerenza basata su blocchi è il *false sharing*. Si verifica quando due core accedono a variabili **diverse** che però si trovano casualmente nella **stessa linea di cache**.

- Se CPU A scrive sulla variabile **Var1** e CPU B legge la variabile **Var2**.
- Se **Var1** e **Var2** sono nello stesso blocco, la scrittura di A invalida l'intera linea nella cache di B.
- B subirà un miss (e dovrà ricaricare il blocco) anche se **Var2** non è cambiata affatto.

Questo fenomeno causa un traffico inutile sul bus e degrada le prestazioni ("ping-pong" della cache line), risolvibile allineando le strutture dati ai confini della cache line.

7.3 Introduzione al Parallelismo

L'obiettivo principale del calcolo parallelo è connettere più processori per ottenere prestazioni superiori rispetto a una singola CPU. Oltre alle performance pure, i sistemi multiprocessore offrono vantaggi in termini di:

- **Scalabilità:** possibilità di aumentare le prestazioni aggiungendo risorse.
- **Disponibilità:** se un processore fallisce, gli altri possono continuare a lavorare.
- **Efficienza energetica:** più processori a frequenza più bassa possono consumare meno di un singolo processore ad altissima frequenza.

Possiamo distinguere due livelli principali di parallelismo:

Task-level (o Process-level) Parallelism Esecuzione di più processi indipendenti contemporaneamente. Ogni processo ha il proprio stato e risorse. Questo approccio punta ad aumentare il *throughput* totale del sistema.

Parallel Processing Program Un *singolo* programma viene eseguito su più processori contemporaneamente. Il programma è diviso in parti che collaborano per risolvere un problema comune più velocemente (es. calcolo scientifico, rendering).

7.4 Hardware e Software nel Parallelismo

La distinzione fondamentale risiede nell'interazione tra l'architettura fisica e il programma che essa esegue:

- **Hardware:** può essere *Serial* o *Parallel*.
- **Software:** può essere scritto in modo *Sequenziale* (e.g., moltiplicazione tra matrici) o *Concorrente* (e.g., Sistemi Operativi).

Un aspetto cruciale è che **software sequenziale o concorrente può girare su entrambi i tipi di hardware**. La vera sfida odierna non è tanto costruire hardware parallelo, quanto scrivere software in grado di *sfruttarlo efficacemente*.

Abbiamo già coperto ILP (Instruction Level Parallelism) 4.5, DLP (Data Level Parallelism) 5 e gerarchia di memoria 6.

7.5 Amdahl's Law

La Legge di Amdahl definisce il limite teorico dello speedup ottenibile parallelizzando un programma. Essa afferma che lo speedup è limitato dalla frazione del programma che **deve** essere eseguita in modo sequenziale.

$$\text{Speedup} = \frac{1}{(1 - F_{\text{parallelizable}}) + \frac{F_{\text{parallelizable}}}{N}} \quad (7.1)$$

dove:

- $F_{\text{parallelizable}}$ è la frazione del programma che può essere parallelizzata.
- $(1 - F_{\text{parallelizable}})$ è la frazione sequenziale che non può essere migliorata.
- N è il numero di processori.

Esempio 7.2 (Limite dello Speedup). Supponiamo di avere $N = 100$ processori e di voler ottenere uno speedup di $90\times$. Applicando la formula inversa, scopriamo che è necessario che:

$$F_{\text{parallelizable}} \approx 0.999$$

Ovvero, la parte sequenziale deve essere ridotta allo **0.1%** del tempo totale. Basta una minima parte non parallelizzabile per abbattere drasticamente i guadagni, indipendentemente da quanti processori aggiungiamo.

Esempio 7.3 (Scaling Example). Consideriamo un programma (es. somma matriciale) con una parte fissa di overhead sequenziale e una parte parallelizzabile.

- **Single Processor:** Tempo = 10 (overhead) + 10000 (lavoro) = 10010.
- **10 Processors:** Tempo = $10 + \frac{10000}{10} = 1010$. Speedup = $\frac{10010}{1010} \approx 9.9$. (99% del potenziale).
- **100 Processors:** Tempo = $10 + \frac{10000}{100} = 110$. Speedup = $\frac{10010}{110} \approx 91$. (91% del potenziale).

All'aumentare di N , il peso relativo dell'overhead sequenziale (10) aumenta rispetto al tempo di esecuzione parallelo (che si riduce), diminuendo l'efficienza dello scaling.

Osservazione 7.1 (Cos'è l'Overhead Sequenziale). *L'overhead che limita lo scaling è costituito da quelle operazioni che **non** possono essere divise tra i core, come:*

- **Input/Output:** lettura dati da disco.
- **Gestione thread:** tempo speso dal sistema operativo per avviare e coordinare i processi.
- **Sincronizzazione:** comunicazione tra core e assemblaggio dei risultati parziali.

Questo tempo rimane costante e rappresenta un limite inferiore invalicabile per il tempo di esecuzione.

7.6 Classificazione dei Flussi (Flynn) e Multithreading

Una classificazione fondamentale delle architetture parallele è la **Tassonomia di Flynn**, che distingue i calcolatori in base al numero di flussi di istruzioni e di dati gestiti simultaneamente.

	Single Data Stream	Multiple Data Stream
Single Instruction	SISD (Single Instruction, Single Data) Un unico flusso di istruzioni agisce su un unico flusso di dati. È il modello sequenziale classico.	SIMD (Single Instruction, Multiple Data) La stessa istruzione viene applicata simultaneamente a più elementi di dati diversi.
Multiple Instruction	MISD (Multiple Instruction, Single Data) Più istruzioni operano simultaneamente sullo stesso flusso di dati.	MIMD (Multiple Instruction, Multiple Data) Ogni processore esegue il proprio flusso di istruzioni su dati indipendenti.

SPMD (Single Program Multiple Data) È il modello di programmazione più comune su macchine MIMD. Un *singolo* programma viene eseguito su tutti i processori, ma ciascuno elabora una porzione diversa dei dati e può seguire percorsi condizionali differenti.

Multithreading Hardware Il multithreading permette di eseguire più flussi di controllo (thread) sullo stesso core fisico, mascherando le latenze di memoria e dipendenze. L'hardware replica solo le risorse essenziali (PC, registri) per ogni thread, mentre le unità funzionali e le cache sono condivise. Il cambio di contesto (context switch) è rapidissimo.

Fine-grain Multithreading Il processore cambia thread attivo ad **ogni ciclo** di clock (in modo round-robin). Se un thread è in stallo, viene saltato.

Coarse-grain Multithreading Il cambio di thread avviene solo in presenza di **stalli lunghi** (es. miss cache L2).

Simultaneous Multithreading (SMT) Utilizzato in processori superscalari (multiple-issue). Invece di alternare i thread, le istruzioni di più thread indipendenti vengono emesse ed eseguite **contemporaneamente** nello stesso ciclo di clock, sfruttando al massimo le unità funzionali disponibili che altrimenti rimarrebbero inutilizzate. Tra thread diversi le dipendenze sono risolte con scheduling o register renaming.

7.7 Processori Multicore e Memoria Condivisa

I processori multicore sono costituiti da più core fisici. Questo approccio vince rispetto all'incremento di frequenza (power wall limitations) perché permette di aumentare il throughput mantenendo la stessa frequenza. Differentemente da altre tecniche, il multicore presenta impatti significativi sulle prestazioni del software.

Shared Memory Multiprocessor (SMP) In un sistema a memoria condivisa (SMP), tutti i processori accedono a un **unico spazio di indirizzamento fisico**. Questo semplifica la programmazione, poiché i dati possono essere scambiati leggendo e scrivendo variabili in memoria, ma richiede meccanismi di **sincronizzazione** (es. lock) per coordinare l'accesso alle variabili condivise ed evitare race condition.

L'accesso alla memoria può essere classificato in:

- **UMA (Uniform Memory Access)**: Il tempo di accesso alla memoria è identico per tutti i processori (tipico dei multicore semplici connessi a un bus comune).
- **NUMA (Non-Uniform Memory Access)**: Il tempo di accesso dipende dalla posizione fisica della memoria rispetto al processore (più veloce per la memoria locale, più lento per quella remota).

Sincronizzazione e Lock La sincronizzazione coordina il comportamento di thread su processori diversi. Un meccanismo comune è il **Lock**:

- Solo un processore alla volta può **acquisire** (*acquire*) il lock per accedere a un dato condiviso.
- Gli altri devono **attendere** (*wait*) finché chi lo possiede non lo **rilascia** (*release*).

Per implementare i lock senza *race condition*, servono istruzioni hardware **atomiche** (indivisibili).

Esempio 7.4 (Sum Reduction Parallela). Vogliamo sommare 64.000 numeri utilizzando 64 processori in un sistema UMA. Ogni processore ha un ID univoco P_n (da 0 a 63).

Fase 1: Somme Locali Dividiamo il lavoro: ogni processore somma una porzione di 1000 numeri.

$$\text{sum}[P_n] = \sum_{i=1000 \cdot P_n}^{1000 \cdot (P_n+1) - 1} A[i]$$

Questa fase è "imbarazzantemente parallela" e non richiede comunicazione.

Fase 2: Riduzione (Global Sum) Ora dobbiamo sommare le 64 somme parziali. Un approccio ingenuo (un solo processore somma tutto) creerebbe un collo di bottiglia. Usiamo invece un approccio **divide-et-impera** ad albero:

1. Metà dei processori sommano il proprio risultato con quello di un "vicino" (es. $P_0 \leftarrow P_0 + P_{32}$).
2. Si ripete dimezzando i processori attivi ad ogni passo ($64 \rightarrow 32 \rightarrow 16 \dots \rightarrow 1$).

È fondamentale usare una **barriera di sincronizzazione** (`synch()`) tra i passi per garantire che i dati siano pronti.

```
half = 64;
do {
    synch();                // Aspetta che tutti abbiano finito il passo precedente
    half = half / 2;        // Divide i processori attivi
    if (Pn < half) {
        sum[Pn] += sum[Pn + half]; // Somma il contributo del "vicino"
    }
} while (half > 1);
```

Alla fine, il processore P_0 conterrà la somma totale. Questo metodo riduce i passi da 63 (lineare) a $\log_2(64) = 6$ (logaritmico).

Message Passing Multiprocessor e Clusters Alternativo al modello a memoria condivisa, nel **message passing** ogni processore possiede uno **spazio di indirizzamento privato**. Non esiste memoria condivisa: la comunicazione avviene scambiando **messaggi espliciti**.

Un'implementazione tipica sono i **Loosely Coupled Clusters**: insiemi di computer indipendenti (con propria memoria e OS) connessi tramite una rete I/O standard (Ethernet, Switch).

- **Applicazioni ideali**: Task indipendenti (Web server, database, simulazioni).
- **Vantaggi**: Alta disponibilità, scalabilità ed economicità.
- **Problemi**: Costi di amministrazione e banda di interconnessione bassa (rispetto ai bus SMP).

Esempio 7.5 (Sum Reduction con Message Passing). In un cluster, la somma di 64.000 numeri su 64 nodi richiede l'invio esplicito dei risultati parziali.

- Ogni nodo calcola la sua somma parziale locale (nessuna comunicazione).

- Si esegue la riduzione ad albero scambiando messaggi.

```

limit = 64; half = 64;
do {
    half = (half + 1) / 2;          // Divide chi invia da chi riceve
    if (Pn >= half && Pn < limit)
        send(Pn - half, sum);      // Invia al processore "inferiore"
    if (Pn < (limit / 2))
        sum += receive();          // Riceve e somma (sincronizzazione implicita)
    limit = half;                  // Nuovo limite processori attivi
} while (half > 1);

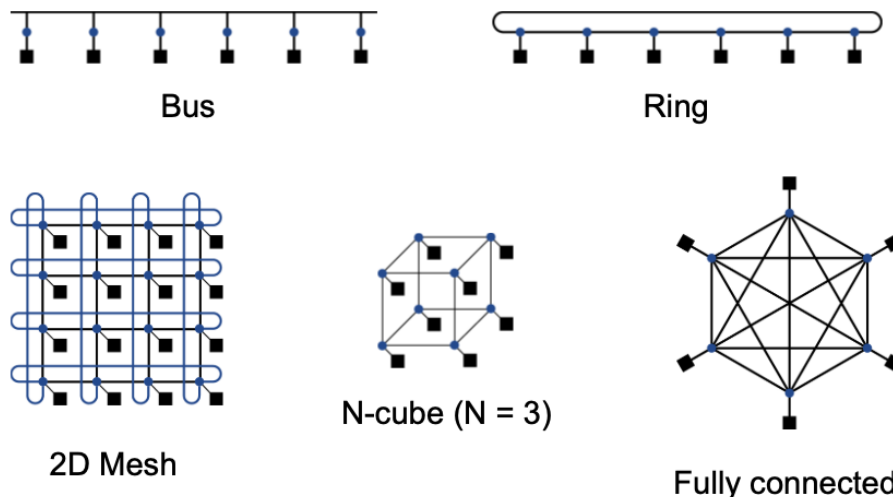
```

Le funzioni `send/receive` garantiscono anche la **sincronizzazione**: il processore che riceve attende l'arrivo del dato, coordinando l'esecuzione senza lock espliciti.

7.8 Interconnection Networks

La topologia di rete definisce come processori, switch e collegamenti sono disposti. Le più comuni includono:

- **Bus**: Canale condiviso (semplice ma con banda limitata).
- **Ring**: Anello unidirezionale o bidirezionale.
- **2D Mesh**: Griglia bidimensionale (tipica nei Manycore e NoC).
- **N-cube**: Ipercubo n-dimensionale (bassa latenza, molti collegamenti).
- **Fully Connected**: Ogni nodo è connesso a tutti gli altri (impraticabile su larga scala).



Metriche di Performance

- **Latenza**: Tempo di trasferimento per messaggio (a rete scarica).
- **Throughput (Bandwidth)**: Banda totale della rete e *Bisection bandwidth* (banda minima tagliando la rete in due parti uguali).
- **Congestione**: Ritardi dipendenti dal traffico.

Il software moderno è interpretato o dinamicamente tipizzato: scrittura del codice semplificata, ma esecuzione più lenta. Per quanto riguarda l'hardware esistono diverse architetture specifiche per dominio (e.g., ASIC).

8 Architetture specifiche per dominio (DSA)

8.1 Motivazioni e Opportunità

La crescita delle prestazioni dei processori general-purpose (CPU) ha subito un rallentamento significativo a causa della fine della *Legge di Moore* e del *Dennard Scaling*. Le tecniche microarchitetture tradizionali (come l'esecuzione fuori ordine, la predizione dei salti o l'aumento della cache) non offrono più guadagni di efficienza sufficienti per soddisfare la domanda di calcolo odierna.

Allo stesso tempo, il software moderno (spesso scritto in linguaggi interpretati) privilegia la facilità di scrittura rispetto all'efficienza di esecuzione pura. Per colmare questo divario tra software ad alto livello e limitazioni hardware, l'industria si è spostata verso le **Domain Specific Architectures (DSA)**: architetture hardware progettate su misura per accelerare una specifica famiglia di applicazioni (un "dominio"), piuttosto che cercare di eseguire bene qualsiasi codice generico.

8.2 Differenza tra DSA e ASIC

È importante distinguere i DSA dagli ASIC tradizionali:

- **ASIC (Application-Specific Integrated Circuit)**: Hardware fisso, progettato per una singola applicazione specifica. Offre la massima efficienza ma zero flessibilità (non è programmabile).
- **DSA (Domain Specific Architecture)**: Un compromesso tra la generalità delle CPU e l'efficienza degli ASIC. Un DSA accelera un intero dominio (es. Reti Neurali, Grafica 3D, Elaborazione Video), mantenendo un grado di **programmabilità** per adattarsi all'evoluzione degli algoritmi nel tempo.

Esempi tipici di DSA sono le GPU (per la grafica e il calcolo parallelo), le TPU (Tensor Processing Unit per il ML) e gli switch di rete programmabili.

8.3 Perché i DSA sono più efficienti?

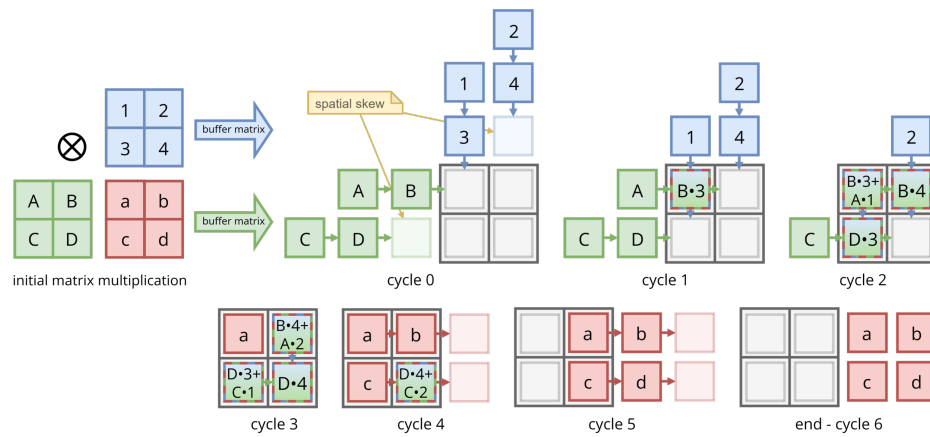
I DSA ottengono prestazioni ed efficienza energetica superiori rispetto alle CPU general-purpose sfruttando la conoscenza specifica del dominio:

1. **Parallelismo Specializzato**: Invece del costoso parallelismo MIMD (Multiple Instruction Multiple Data) delle CPU, i DSA usano spesso modelli SIMD (Single Instruction Multiple Data) o VLIW, che richiedono meno logica di controllo per ogni operazione eseguita.
2. **Gestione della Memoria Esplicita**: Le CPU usano cache complesse per gestire i dati automaticamente. I DSA spesso usano **memorie locali** (scratchpad) gestite dal software. Questo permette di ottimizzare manualmente gli spostamenti dei dati, massimizzando l'uso della banda (bandwidth) e riducendo i consumi.
3. **Precisione Ridotta**: Molti domini (come il Deep Learning) non richiedono la precisione dei numeri in virgola mobile a 32 o 64 bit (IEEE 754 standard). I DSA possono usare formati ridotti (es. INT8, Float16, Bfloat16), riducendo drasticamente la complessità dell'hardware e il volume di dati da spostare.
4. **Riduzione dell'Overhead**: L'hardware specializzato elimina gran parte dell'overhead tipico delle CPU (fetch, decode complessi) dedicando più transistor al calcolo effettivo.

8.3.1 Systolic Arrays (Array Sistolici)

Un array sistolico è una rete di unità di elaborazione (PE - Processing Elements) connesse in modo regolare, solitamente a griglia. Il nome deriva dall'analogia con il sistema circolatorio: i dati vengono "pompati" attraverso l'array in modo ritmico.

- Ogni PE riceve dati dai suoi vicini, esegue una piccola computazione (es. una moltiplicazione e accumulo, MAC) e passa il risultato al vicino successivo al ciclo di clock seguente. Dunque i dati fluiscono attraverso l'array e si presentano agli ingressi dei PE **al momento giusto**: i PE non devono eseguire operazioni di indirizzamento o fetch dalla memoria, ma elaborano semplicemente ciò che arriva dai vicini ad ogni ciclo di clock.
- **Vantaggio**: Questo massimizza il **riuso dei dati**. Un singolo dato letto dalla memoria viene utilizzato da decine o centinaia di PE mentre attraversa l'array, riducendo drasticamente la necessità di accessi alla memoria globale (che sono lenti ed energivori).



8.3.2 Double Buffering

Per nascondere la latenza di accesso alla memoria, si utilizza la tecnica del *double buffering* (o ping-pong buffering). Mentre l'unità di calcolo lavora sui dati presenti nel **buffer A**, il sistema carica i dati successivi nel **buffer B** in background. Quando l'elaborazione su A è terminata, l'unità passa istantaneamente a B, e il buffer A diventa disponibile per caricare nuovi dati. Questo garantisce che le unità di calcolo siano sempre attive e non debbano andare in stallo attendendo il caricamento dei dati dalla memoria principale.

8.4 Programmazione dei DSA

Programmare un DSA è più complesso di una CPU standard. Non si usa tipicamente l'assembly diretto, ma si fa affidamento su stack software sofisticati:

- **Framework e DSL (Domain Specific Languages)**: Linguaggi come TensorFlow, PyTorch o Halide permettono di descrivere gli algoritmi ad alto livello.
- **Compilatori Specializzati**: Il compilatore ha il compito cruciale di mappare queste operazioni sull'hardware, gestendo l'allocazione delle memorie locali, la parallelizzazione e il trasferimento dati (DMA), nascondendo la complessità al programmatore.

8.5 Sfide nella Progettazione dei DSA

8.5.1 Il "Memory Wall" negli Acceleratori

Anche se la specializzazione e il parallelismo aumentano drasticamente la capacità di calcolo (throughput), queste unità di elaborazione devono essere alimentate continuamente con dati.

- L'area e il consumo energetico di molti acceleratori sono dominati dalla **memoria**, non dalla logica di calcolo. Ad esempio, nell'acceleratore EIE (Efficient Inference Engine), oltre il 90% dell'area è occupato dalla memoria (SRAM locale).
- Il sistema è spesso *memory bound* (limitato dalla memoria) piuttosto che *compute bound*.

Per mitigare questo problema, si adottano diverse strategie:

1. **Canali di Memoria Multipli:** Utilizzare più canali paralleli per aumentare la banda verso la memoria globale (DRAM).
2. **Compressione dei Dati:** Ridurre la dimensione dei dati da trasferire aumenta la banda "effettiva" e la capacità della memoria locale. Tecniche comuni includono:
 - *Rappresentazione Sparsa:* Memorizzare solo i valori non nulli (utile nelle Reti Neurali dove molti pesi sono zero).
 - *Codebook/Quantizzazione:* Usare indici brevi (es. 4 bit) che puntano a una tabella di valori reali condivisi, ottenendo compressioni notevoli (es. 32-64x).

8.5.2 Riduzione dell'Overhead Energetico

Un processore general-purpose semplice spende oltre il 90% della sua energia in **overhead**: instruction fetch, decodifica, gestione della pipeline e controllo. Solo una minima parte serve per l'operazione aritmetica vera e propria (es. una somma intera a 32 bit).

- I DSA eliminano quasi completamente questo overhead. Non ci sono istruzioni da recuperare e decodificare continuamente.
- I dati provengono da registri dedicati o memorie locali, evitando l'energia costosa per accedere a grandi register file multiporta o cache complesse.

8.6 Approcci Implementativi: Istruzioni Speciali vs Motori Dedicati

Esistono due approcci principali per integrare l'accelerazione hardware:

8.6.1 1. Istruzioni Specializzate in Processori General-Purpose

Si aggiungono istruzioni complesse a processori esistenti (come le GPU o anche CPU).

- **Esempio:** Istruzioni **HMMA** (Half-Precision Matrix Multiply-Accumulate) nelle GPU NVIDIA Volta/Turing. Queste istruzioni eseguono operazioni matriciali (es. 4x4 o 8x8) in un solo colpo.
- **Vantaggi:** Si mantiene la flessibilità del processore generico per tutto il resto del codice (OS, I/O, parti non parallelizzabili dell'algoritmo).
- **Efficienza:** Anche se meno efficiente di un chip dedicato, l'istruzione complessa ammortizza enormemente il costo di fetch/decode (una sola istruzione lancia centinaia di operazioni aritmetiche).

8.6.2 2. Acceleratori Dedicati (Motori Specializzati)

Si costruisce un chip o un blocco IP dedicato esclusivamente a quel compito (es. Google TPU).

- **Vantaggi:** Massima efficienza energetica possibile (circa 13-23% meglio rispetto all'approccio a istruzioni speciali su GPU, secondo alcune stime).
- **Svantaggi:** Minore flessibilità. Se l'algoritmo cambia radicalmente, l'hardware potrebbe diventare obsoleto o inutilizzabile. Inoltre, il chip dedicato ha costi di progettazione e produzione elevati se non prodotto in volumi massicci.

9 Google TPU: Caso di Studio di un DSA

La **Tensor Processing Unit (TPU)** di Google, introdotta nel 2015, è uno degli esempi più celebri di DSA. È stata progettata specificamente per accelerare l'inferenza (e successivamente anche il training) delle **Deep Neural Networks (DNN)**, offrendo speedup di 15-30x e miglioramenti di efficienza energetica di 30-80x rispetto a CPU e GPU di tecnologia comparabile.

9.1 Cenni sulle Deep Neural Networks

Per comprendere l'architettura della TPU, è utile richiamare brevemente il funzionamento delle DNN e le loro caratteristiche computazionali.

Operazione Base: il Neurone Un neurone artificiale calcola una **somma pesata** dei suoi input e applica una **funzione di attivazione** non lineare:

$$y = f\left(\sum_i w_i \cdot x_i + b\right) = f(\mathbf{w}^T \mathbf{x} + b)$$

Le funzioni di attivazione comuni sono ReLU, Sigmoid e Tanh. I neuroni sono organizzati in **layer** (strati): l'output di un layer diventa l'input del successivo.

Multi-Layer Perceptron (MLP) In un layer fully-connected con n neuroni in input e m in output:

- **Numero di pesi:** $n \times m$ (una matrice $\mathbf{W}$).
- **Operazioni:** $2 \times n \times m$ (una moltiplicazione e una somma per ogni peso).
- **Intensità operativa:** ≈ 2 operazioni per peso letto dalla memoria.

L'operazione dominante è la **moltiplicazione matrice-vettore**: $\mathbf{y} = f(\mathbf{W} \cdot \mathbf{x} + \mathbf{b})$.

Convolutional Neural Networks (CNN) Le CNN, usate nella computer vision, applicano **filtri** (kernel) a mappe di feature 2D. Ogni layer alza il livello di astrazione: i primi layer riconoscono linee e bordi, quelli intermedi forme geometriche, quelli finali oggetti complessi.

- L'operazione base è lo **stencil** (convoluzione): un piccolo kernel (es. 3×3) viene applicato a ogni posizione della mappa.
- **Intensità operativa:** molto più alta degli MLP (~ 400 ops/peso), poiché lo stesso peso viene riutilizzato su molte posizioni spaziali.

Ottimizzazioni per l'Hardware Due tecniche comuni aumentano l'efficienza:

- **Batching:** Elaborare più input contemporaneamente permette di riutilizzare i pesi una volta caricati dalla memoria, aumentando l'intensità operativa.
- **Quantizzazione:** Usare interi a 8 o 16 bit invece di float a 32 bit riduce memoria, banda e complessità hardware con perdita di accuratezza trascurabile.

Kernel Computazionali Un acceleratore per DNN deve supportare efficientemente:

- Moltiplicazione matrice-vettore e matrice-matrice
- Convoluzione (stencil)
- Funzioni di attivazione (ReLU, Sigmoid, Tanh)

Questa regolarità computazionale rende le DNN ideali per l'accelerazione hardware.

9.2 Architettura della TPUv1

La TPU è un **coproprocessore** connesso alla CPU host tramite bus PCIe. Non esegue codice autonomamente: le istruzioni le vengono inviate dalla CPU.

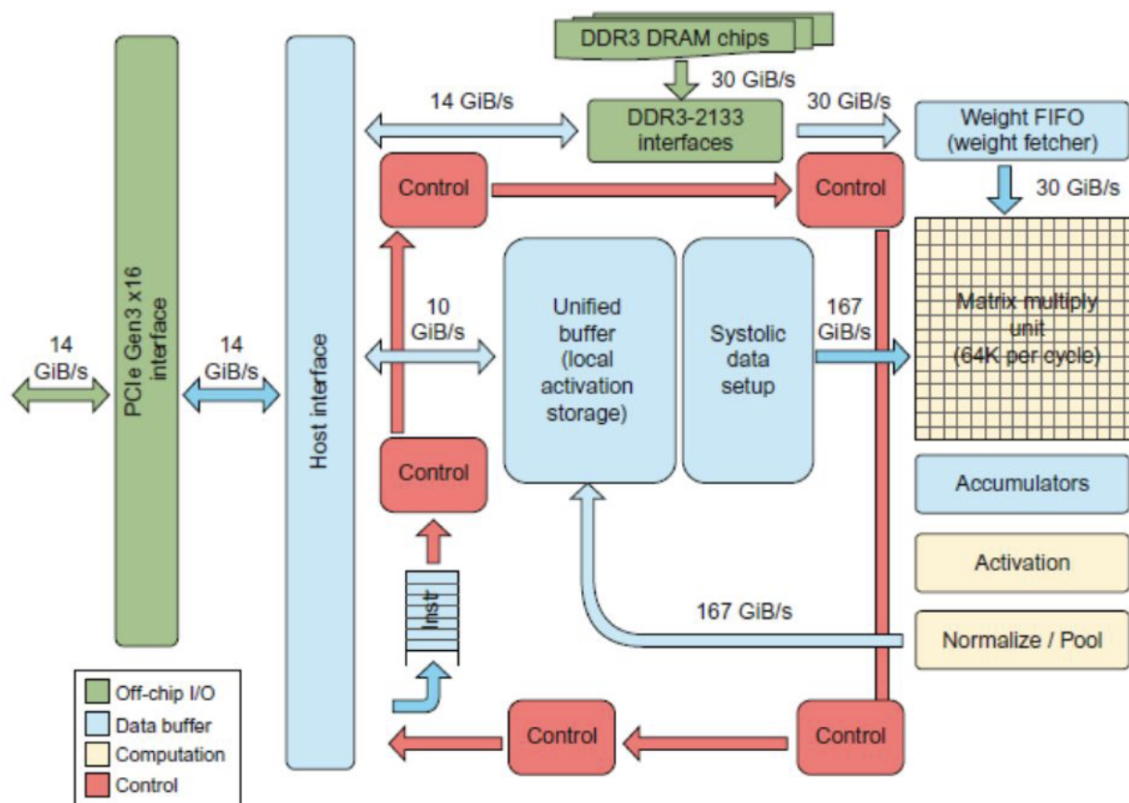


Figura 11: Schema a blocchi della TPU. In azzurro i buffer dati, in giallo le unità di calcolo, in rosso il controllo, in verde le interfacce I/O.

I componenti principali sono:

Matrix Multiply Unit (MXU) Il cuore della TPU è un **array sistolico** di $256 \times 256 = 65.536$ unità MAC (Multiply-Accumulate) a 8 bit. Può eseguire fino a 65.536 moltiplicazioni e accumuli per ciclo di clock. Le moltiplicazioni producono risultati a 16 bit che vengono accumulati in registri a 32 bit.

Accumulators Un banco di **4 MiB di accumulatori a 32 bit** raccoglie i risultati parziali dalla MXU. Gli accumulatori memorizzano 4096 vettori da 256 elementi ciascuno.

Activation Unit e Normalize/Pool Queste unità hardware calcolano le **funzioni di attivazione** non lineari (ReLU, Sigmoid, ecc.) e le operazioni di normalizzazione e pooling direttamente sui dati in uscita dagli accumulatori, evitando round-trip verso la memoria.

Unified Buffer Una SRAM on-chip da **24 MiB** funge da memoria locale per le attivazioni (input e output intermedi dei layer). È gestita esplicitamente dal software (scratchpad), non è una cache.

Weight FIFO e Weight Memory I pesi della rete neurale sono memorizzati in una **DRAM** esterna da **8 GiB** (Weight Memory). Prima di essere usati dalla MXU, vengono caricati in una **FIFO on-chip** (Weight FIFO) che li alimenta all'array sistolico al ritmo richiesto.

9.3 ISA della TPU

La TPU utilizza un set di istruzioni **CISC** molto compatto (circa una dozzina di istruzioni). Caratteristiche distintive:

- **Nessun Program Counter:** la TPU non ha un PC interno né istruzioni di salto. Le istruzioni vengono inviate sequenzialmente dalla CPU host.
- **CPI elevato:** ogni istruzione CISC richiede tipicamente 10-20 cicli di clock, ma esegue un'enorme quantità di lavoro (es. una moltiplicazione matriciale completa).

Le istruzioni principali sono:

- **Read_Host_Memory:** Trasferisce dati dalla memoria della CPU all'Unified Buffer.
- **Read_Weights:** Carica i pesi dalla Weight Memory nella Weight FIFO.
- **MatrixMultiply/Convolve:** Esegue la moltiplicazione matriciale o la convoluzione.
- **Activate:** Applica la funzione di attivazione.
- **Write_Host_Memory:** Scrive i risultati nella memoria della CPU.

9.4 Distribuzione dell'Area del Chip (Floorplan)

La TPU è un esempio lampante di come, nei DSA, la logica di *controllo* sia minimizzata a favore di memoria e calcolo.

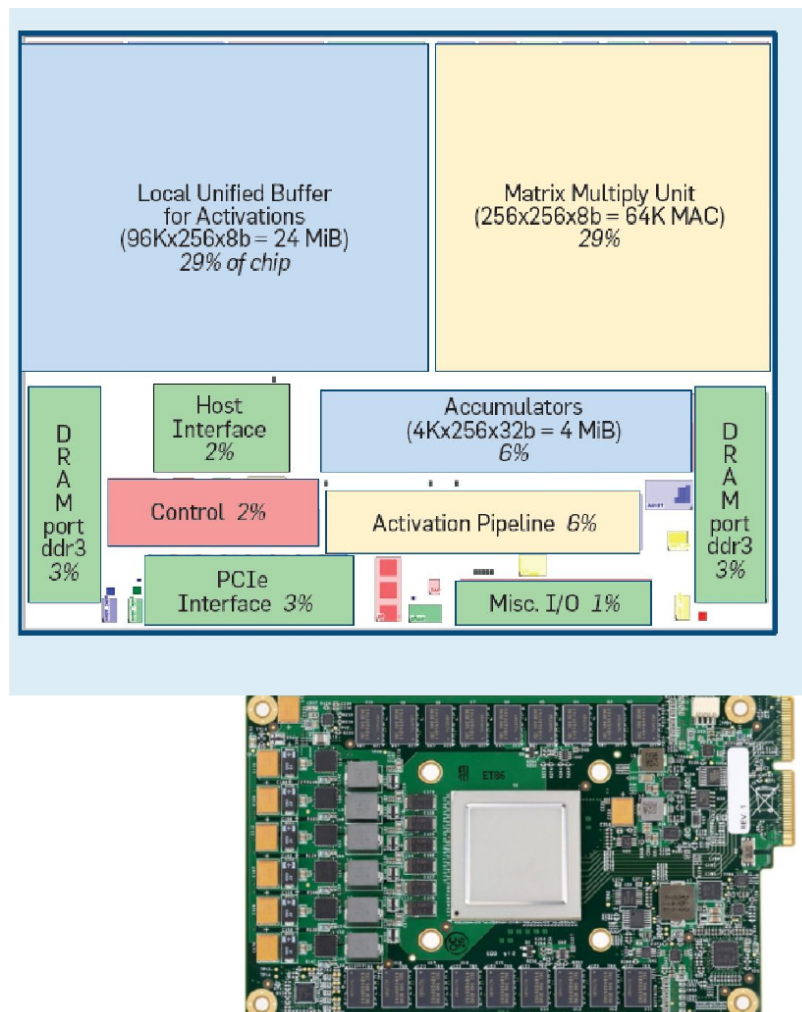


Figura 12: Floorplan della TPUv1. La maggior parte dell'area è dedicata a memoria (Unified Buffer) e calcolo (Matrix Multiply Unit).

- **35% Memoria:** Unified Buffer (24 MiB) e Accumulatori (4 MiB).
- **35% Calcolo:** Matrix Multiply Unit (array sistolico 256x256).
- **12% I/O:** Interfacce PCIe e DDR3.
- **2% Controllo:** Logica di controllo. In una CPU o GPU questa percentuale sarebbe molto più alta (e più difficile da progettare).

9.5 Roofline Model

Il **Roofline Model** è uno strumento visuale per analizzare le prestazioni di un sistema in funzione dell'**intensità aritmetica** (Arithmetic Intensity) di un algoritmo.

Definizione L'intensità aritmetica è il rapporto tra operazioni floating-point (o intere) eseguite e byte letti dalla memoria:

$$\text{Intensità Aritmetica} = \frac{\text{Operazioni}}{\text{Byte letti dalla memoria}} \quad \left[\frac{\text{FLOP}}{\text{Byte}} \right]$$

Interpretazione del Grafico Il modello traccia le prestazioni massime raggiungibili (GFLOPs/s) in funzione dell'intensità aritmetica:

- **Zona Memory-Bound** (bassa intensità): Le prestazioni crescono linearmente con l'intensità. Il sistema è limitato dalla banda di memoria: $\text{Perf} = \text{BW}_{\text{mem}} \times \text{Intensità}$.
- **Zona Compute-Bound** (alta intensità): Le prestazioni raggiungono un plateau orizzontale pari alla capacità di picco dell'unità di calcolo.
- **Ridge Point:** Il punto di intersezione ("cresta") indica l'intensità minima necessaria per sfruttare appieno la capacità di calcolo.

$$\text{Prestazioni Raggiungibili} = \min(\text{Peak BW} \times \text{Intensità}, \text{Peak FLOP/s})$$

Applicazione alla TPU L'analisi di diverse reti neurali sulla TPU mostra che la maggior parte dei workload è **memory-bound**: il collo di bottiglia è la banda verso la memoria dei pesi (DRAM esterna), non la capacità di calcolo della MXU. Questo spiega perché aumentare la banda di memoria (o usare tecniche di compressione dei pesi) è spesso più efficace che aggiungere ulteriori unità di calcolo.

9.6 Linee Guida per la Progettazione di DSA

Le scelte progettuali della TPU esemplificano le linee guida generali per la progettazione di architetture specifiche per dominio:

1. **Ridurre dimensione e tipo dei dati** al minimo necessario per il dominio.
 - *TPUv1*: Usa principalmente interi a 8 bit invece di floating-point a 32 bit.
2. **Ridurre l'overhead**: investire le risorse in più unità aritmetiche o memorie più grandi, non in logica di controllo.
 - *TPUv1*: Il 60% dell'area è dedicato a memoria, con 250x più unità aritmetiche di una CPU server.

3. **Usare memorie dedicate** per minimizzare lo spostamento dei dati.
 - *TPUv1*: 24 MiB di Unified Buffer + 4 MiB di accumulatori on-chip.
4. **Sfruttare il parallelismo più semplice** adatto al dominio.
 - *TPUv1*: Parallelismo SIMD 2D tramite l'array sistolico 256x256.
5. **Usare un linguaggio specifico per il dominio** (DSL).
 - *TPUv1*: Programmata tramite TensorFlow, che nasconde la complessità hardware.

10 Graphic Processing Unit (GPU)

La **Graphics Processing Unit (GPU)** è un dispositivo equipaggiato con un microprocessore altamente parallelo (*many-core*) e una memoria locale privata a banda molto elevata. Sebbene sia nata per rispondere alla crescente domanda di rendering grafico 3D ad alta definizione, oggi la GPU è ampiamente impiegata per molti altri compiti computazionali (GPGPU - *General Purpose GPU*), come il machine learning, l'analisi dati e le simulazioni scientifiche.

Le GPU sono specializzate per problemi classificabili come **calcoli intensivi data-parallel**: lo stesso set di operazioni viene eseguito molte volte in parallelo su dati diversi.

GPGPU per AI/ML

L'uso delle GPU general purpose ha rivoluzionato il campo dell'Intelligenza Artificiale fornendo la potenza di calcolo necessaria per addestrare ed eseguire modelli complessi in modo efficiente. Le GPU possono accelerare il training in diversi modi:

- **Moltiplicazioni di matrici**: velocizzano il calcolo dei pesi e degli input.
- **Funzioni di attivazione**: computano simultaneamente le funzioni non lineari per un gran numero di neuroni.
- **Backpropagation**: accelerano il calcolo dei gradienti durante la fase di apprendimento.

Una volta addestrato il modello, le GPU sono utilizzate anche per velocizzare la fase di **inferenza**.

10.1 Confronto Architeturale: CPU vs GPU

Per comprendere appieno il ruolo delle GPU, è utile confrontarle con le CPU, evidenziando come i due dispositivi siano progettati con obiettivi diametralmente opposti: la CPU è orientata alla **latenza**, la GPU al **throughput**.

CPU: Design Orientato alla Latenza La CPU è progettata per eseguire un singolo thread (o pochi thread) il più velocemente possibile. Le sue caratteristiche principali sono:

- **Frequenza di clock elevata**: per eseguire velocemente le istruzioni sequenziali.
- **Grandi Cache**: occupano una parte significativa del chip per convertire accessi alla memoria a lunga latenza in accessi alla cache a bassa latenza.
- **Controllo Sofisticato**: implementa tecniche complesse come la *Branch Prediction* (per ridurre la latenza dei salti) e il *Data Forwarding* (per ridurre la latenza dei dati).
- **ALU Potenti**: progettate per ridurre la latenza delle singole operazioni.

La struttura del chip vede quindi un'area predominante dedicata alla cache e al controllo, con un numero ridotto di unità aritmetiche (ALU).

GPU: Design Orientato al Throughput La GPU sacrifica la velocità del singolo thread per massimizzare il numero totale di operazioni completate per unità di tempo. Le sue caratteristiche sono:

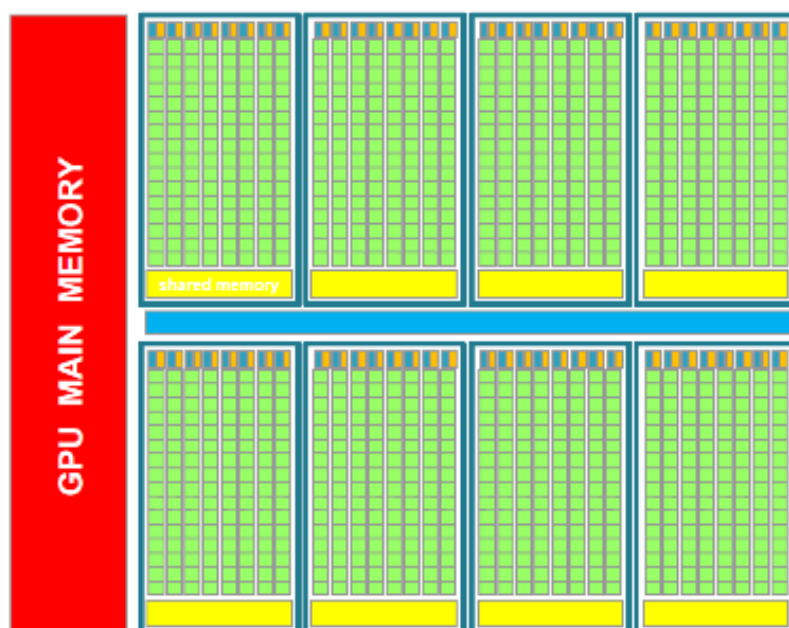
- **Frequenza di clock moderata:** solitamente inferiore rispetto alle CPU.
- **Cache Piccole:** servono principalmente a sostenere il throughput della memoria (Memory Throughput), non a nascondere la latenza.
- **Controllo Semplice:** non vi è branch prediction né data forwarding complessi, risparmiando area e potenza.
- **ALU Efficienti energeticamente:** sono presenti in numero massiccio. Sono spesso fortemente pipelinate ma con latenza singola più alta (long latency).

Per tollerare le latenze, la GPU richiede un **numero massiccio di thread**: se un thread si blocca, l'hardware passa istantaneamente ad un altro, mantenendo le unità di calcolo occupate. Le applicazioni moderne beneficiano dell'uso combinato di entrambi i processori:

- La **CPU** gestisce le parti **sequenziali** dove la latenza è critica (può essere 10+ volte più veloce della GPU in questo).
- La **GPU** gestisce le parti **parallele** dove vince il throughput (può essere 10+ volte più veloce della CPU su codice parallelo).

Tipicamente l'**architettura** di una GPU è composta da:

- **Main Global Memory:** La memoria principale esterna, caratterizzata da un'elevata banda passante (High Bandwidth) necessaria per alimentare continuamente le unità di calcolo.
- **Streaming Multiprocessor (SMP):** È l'unità fondamentale di elaborazione che raggruppa al suo interno diversi core indipendenti e le unità di controllo. Una componente cruciale dell'SMP è la **Shared Memory**, una memoria locale ad accesso rapidissimo che permette ai thread in esecuzione sullo stesso processore di condividere dati efficientemente, riducendo gli accessi alla memoria globale.
- **CUDA Core:** Unità di elaborazione elementare che contiene molteplici ALU e gli scheduler delle istruzioni, responsabile dell'esecuzione effettiva dei calcoli.



Funzionamento del CUDA Core: Il Modello SIMD Il cuore dell'elaborazione avviene secondo il modello **SIMD** (Single Instruction, Multiple Data): ogni istruzione controlla simultaneamente più unità di calcolo. Nello specifico, un core di elaborazione agisce come un processore SIMD dotato di **16 linee di esecuzione** (SIMD Lanes).

- **Esecuzione dei Thread:** Le istruzioni vengono processate su gruppi di 32 elementi (thread, detti **warp**). Poiché l'hardware fisico dispone di 16 linee (cuda cores), l'esecuzione di un gruppo da 32 avviene dinamicamente in **2 cicli** di clock consecutivi.
- **Registri e Context Switch:** Per gestire l'alto parallelismo senza penalità di tempo, l'architettura dispone di un **Register File** (es. $32K \times 32\text{-bit}$) distribuito tra le linee. Questo permette di assegnare contesti privati (es. 64 registri) a ciascun thread: quando un thread si blocca, l'hardware può passare istantaneamente ad un altro thread i cui dati sono già pronti nei registri, nascondendo così le latenze.

Modello di Programmazione GPGPU e CUDA

Dal punto di vista del programmatore, la GPU viene vista come un **coprocessore ausiliario** dotato di migliaia di core e di una memoria globale ad alta banda, a cui affidare le parti del programma computazionalmente intensive e altamente parallele (data-parallel). Il principio fondamentale per ottenere alte prestazioni è utilizzare un numero elevatissimo di thread, in modo da:

- Assegnare a ciascun thread l'elaborazione di un diverso elemento di dati in modo indipendente.
- Nascondere le latenze di memoria o di calcolo grazie al rapidissimo context-switch (i thread GPU sono "leggeri" e non pagano penalità nel cambio di contesto avendo registri dedicati).

CUDA (Compute Unified Device Architecture) CUDA è la piattaforma di calcolo parallelo proprietaria di NVIDIA che ha reso accessibile la programmazione delle GPU per scopi generali. Essa fornisce:

- Un set di estensioni per usare la GPU come coprocessore per task paralleli.
- Un nuovo paradigma di programmazione **multi-thread gerarchico**.
- Estensioni sintattiche a linguaggi di alto livello (come C/C++ o Fortran) che permettono di esprimere il parallelismo dei thread all'interno di un ambiente di sviluppo familiare.
- Strumenti di sviluppo completi (compilatori, debugger, profiler).

10.2 Programmazione Scalabile e Gerarchia CUDA

Il modello di programmazione CUDA si basa su una gerarchia di astrazioni che permette di scrivere codice parallelizzabile in modo **scalabile**, ovvero capace di adattarsi automaticamente a GPU con un numero diverso di core fisici. Un programma CUDA tipico gestisce l'allocazione della memoria sul dispositivo (GPU), il trasferimento dei dati da Host (CPU) a Device, l'esecuzione della funzione parallela (chiamata **Kernel**) e infine il recupero dei risultati.

Organizzazione dei Thread e Calcolo degli Indici Quando viene lanciato un Kernel, migliaia di thread eseguono la **stessa funzione** simultaneamente. Per coordinare questa massa di esecuzioni e assegnare a ciascun thread una porzione specifica di dati, CUDA organizza i thread in una gerarchia a due livelli: una *Griglia* composta da *Blocchi*, a loro volta composti da *Thread*. Sia la Griglia che i Blocchi possono essere strutturati in 1, 2 o 3 dimensioni (x, y, z). Questa organizzazione semplifica l'indirizzamento di dati multidimensionali (es. immagini 2D o volumi 3D).

Ogni thread possiede variabili intrinseche (builtin) assegnate dal runtime CUDA per identificare la propria posizione:

- **threadIdx**: Coordinate (x, y, z) del thread all'interno del blocco.
- **blockIdx**: Coordinate (b_x, b_y, b_z) del blocco all'interno della griglia.
- **blockDim**: Dimensioni (B_x, B_y, B_z) di ciascun blocco.
- **gridDim**: Dimensioni (G_x, G_y, G_z) della griglia.

Combinando queste coordinate, il thread calcola il proprio ID globale univoco, indispensabile per accedere alla corretta cella di memoria da elaborare. In questo modello, la posizione spaziale funge da puntatore: si realizza una mappatura diretta tra la geometria di esecuzione e la struttura dei dati, dove l'indice spaziale sostituisce l'iterazione temporale dei cicli classici (e.g. iterare su un array con un ciclo `for`).

Il **threadID** è unico solo all'interno del blocco. Per ottenere un indice univoco globale e accedere linearmente alla memoria, si combinano queste variabili.



Le tre astrazioni chiave sono:

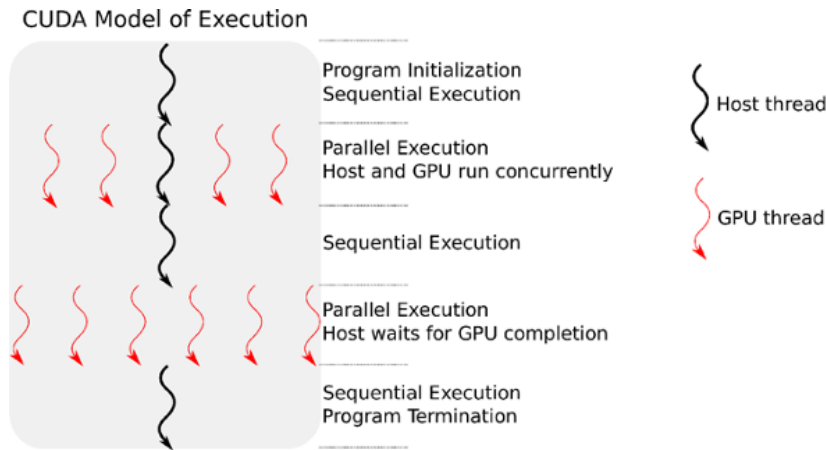
1. **Gerarchia di Thread**: Il problema viene scomposto in sottoproblemi più piccoli.
 - **Griglia (Grid)**: L'insieme totale dei thread lanciati per eseguire un kernel.
 - **Blocchi (Thread Blocks)**: La griglia è divisa in blocchi indipendenti. I blocchi possono essere eseguiti in parallelo in qualsiasi ordine su diversi Streaming Multiprocessors (SM). Questa indipendenza permette la scalabilità: una GPU con più SM eseguirà più blocchi in parallelo, completando il lavoro prima, senza dover ricompilare il codice.
 - **Thread**: All'interno di un blocco, i thread possono cooperare tra loro e sincronizzarsi.
2. **Memorie Condivise (Shared Memory)**: I thread dello stesso blocco possono condividere dati tramite una memoria locale veloce.
3. **Sincronizzazione a Barriera**: Meccanismo per coordinare i thread all'interno di un blocco.

Hardware vs Programmer View of CUDA Memories Dal punto di vista della memoria, è utile distinguere tra la visione logica e quella fisica:

- **Per-Thread**: Ogni thread ha una sua zona privata (Registri) per le variabili locali.

- **Per-Block:** Tutti i thread di un blocco condividono la *Shared Memory*. Questa memoria risiede fisicamente all'interno dello stesso SM che esegue il blocco, garantendo accessi rapidissimi e comunicazione interna.
- **Per-Grid:** Tutti i thread (di qualsiasi blocco) accedono alla *Global Memory*.

Questa struttura a due livelli (indipendenza tra blocchi, cooperazione dentro i blocchi) permette di combinare parallelismo **coarse-grained** (tra blocchi) e **fine-grained** (tra thread).



Esempio 10.1 (Hello World in CUDA). Il seguente esempio illustra come identificare univocamente i thread e lanciare un kernel. La funzione `printThreadID` viene eseguita in parallelo da ogni thread. Per calcolare il proprio ID globale lineare (`threadID`), ogni thread combina:

- `threadIdx.x`: il proprio indice all'interno del blocco.
- `blockIdx.x`: l'indice del blocco a cui appartiene.
- `blockDim.x`: la dimensione del blocco (numero di thread per blocco).

La formula $ID = threadIdx.x + blockIdx.x \times blockDim.x$ permette di "appiattire" la gerarchia in un indice lineare unico.

```
// CUDA kernel function: eseguita sulla GPU
__global__ void printThreadID() {
    // Calcola l'ID univoco globale del thread
    int threadID = threadIdx.x + blockIdx.x * blockDim.x;

    // Stampa l'ID (ogni thread stampa il suo)
    printf("Thread ID: %d\n", threadID);
}

int main() {
    // Configurazione: 2 blocchi, ognuno con 4 thread
    int threadsPerBlock = 4;
    int numberOfBlocks = 2;

    // Lancia il kernel sulla GPU
    // Sintassi <<<blocchi, thread>>>
    printThreadID<<<numberOfBlocks, threadsPerBlock>>>();

    // Attende che la GPU finisca prima di chiudere (necessario per vedere l'output)
    cudaDeviceSynchronize();
}
```

```
    return 0;
}
```

Nell'esempio, vengono lanciati in totale $2 \times 4 = 8$ thread. Ognuno calcolerà un ID diverso (da 0 a 7) e lo stamperà.

Esempio 10.2 (Somma di Vettori: Host vs Device Code). L'obiettivo è calcolare la somma di due vettori $C = A + B$. Confrontiamo l'approccio sequenziale classico (CPU) con quello parallelo eterogeneo (CPU+GPU).

Codice Sequenziale (C tradizionale) La CPU esegue un ciclo `for` che itera su tutti gli elementi uno alla volta.

```
void vecAdd(float* A, float* B, float* C, int n) {
    for (int i = 0; i < n; i++)
        C[i] = A[i] + B[i];
}
```

Codice Host (Heterogeneous Computing) Il codice Host (eseguito sulla CPU) gestisce la memoria e lancia il calcolo sulla GPU. Per gestire i dati sulla GPU, CUDA fornisce funzioni simili al C standard ma con prefisso `cuda`:

- `cudaMalloc(void** devPtr, size_t size)`: Alloca memoria nella **Global Memory** del dispositivo. Richiede l'indirizzo del puntatore e la dimensione in byte.
- `cudaMemcpy(void* dst, const void* src, size_t count, kind)`: Trasferisce dati tra Host e Device. Il parametro `kind` specifica la direzione (es. `cudaMemcpyHostToDevice`). È importante notare che il trasferimento verso il device è **sincrono** (bloccante).
- `cudaFree(void* devPtr)`: Libera la memoria allocata sul dispositivo.

```
void vecAdd(float* A, float* B, float* C, int n) {
    int size = n * sizeof(float);
    float *A_d, *B_d, *C_d; // Puntatori per la memoria Device

    // 1. Allocazione memoria sulla GPU e copia dati Input (Host -> Device)
    cudaMalloc((void**)&A_d, size);
    cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
    // ... (allocazione e copia simili per B_d) ...
    cudaMalloc((void**)&C_d, size);

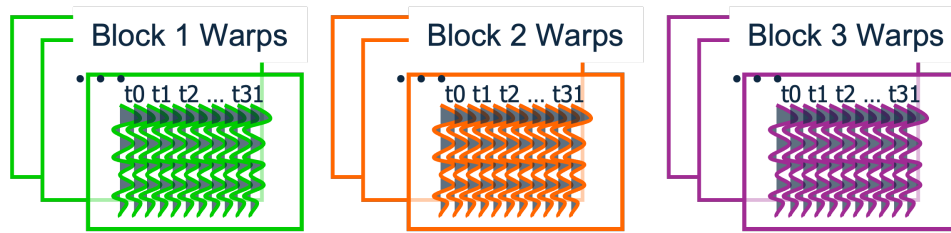
    // 2. Lancio del Kernel (GridSize, BlockSize)
    // Ogni thread calcolerà una singola somma A[i] + B[i]
    vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);

    // 3. Copia risultato Output (Device -> Host) e rilascio memoria
    cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
    cudaFree(A_d); cudaFree(B_d); cudaFree(C_d);
}
```

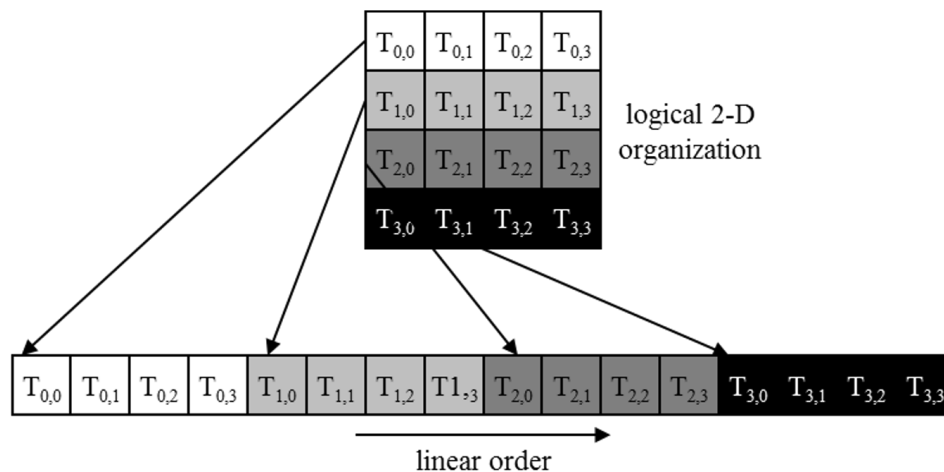
Panoramica delle Memorie CUDA Le API mostrate operano sulla **Global Memory**, accessibile sia dall'Host (per i trasferimenti) che da tutti i thread della griglia (per lettura/scrittura). Parallelamente, ogni thread possiede i propri registri privati (velocissimi ma limitati) per le variabili locali.

10.3 Scheduling dei Thread e Warps

L'esecuzione fisica dei thread sulla GPU non avviene singolarmente, ma in gruppi chiamati **Warp**. Un blocco di thread viene assegnato a un singolo Streaming Multiprocessor (SM) ed è suddiviso in warp (attualmente composti da **32 thread**).



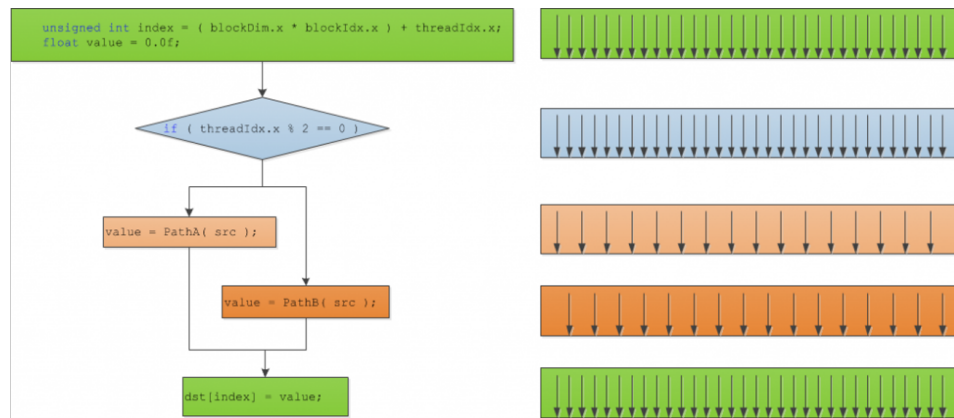
Linearizzazione dei Blocchi Multidimensionali Anche se i blocchi possono essere dichiarati come strutture 2D o 3D per comodità del programmatore, l'hardware li vede come una sequenza lineare 1D. La linearizzazione avviene in ordine *row-major*: prima la dimensione x , poi y , infine z . Questo significa che i thread con indici adiacenti (es. $T_{0,0}$ e $T_{1,0}$) finiranno nello stesso warp ed eseguiranno le istruzioni insieme.



Esecuzione SIMD e Control Divergence All'interno di un warp, l'esecuzione è rigorosamente **SIMD**: tutti i 32 thread devono eseguire la **stessa istruzione** nello stesso istante. Questo meccanismo è efficientissimo quando tutti i thread seguono lo stesso percorso di controllo (control flow path), come ad esempio iterare lo stesso numero di volte o prendere tutti lo stesso ramo di un `if`. Tuttavia, se la condizione di un `branch` dipende dall'indice del thread (es. `if (threadIdx.x > 2)`), si verifica il fenomeno della **Control Divergence**:

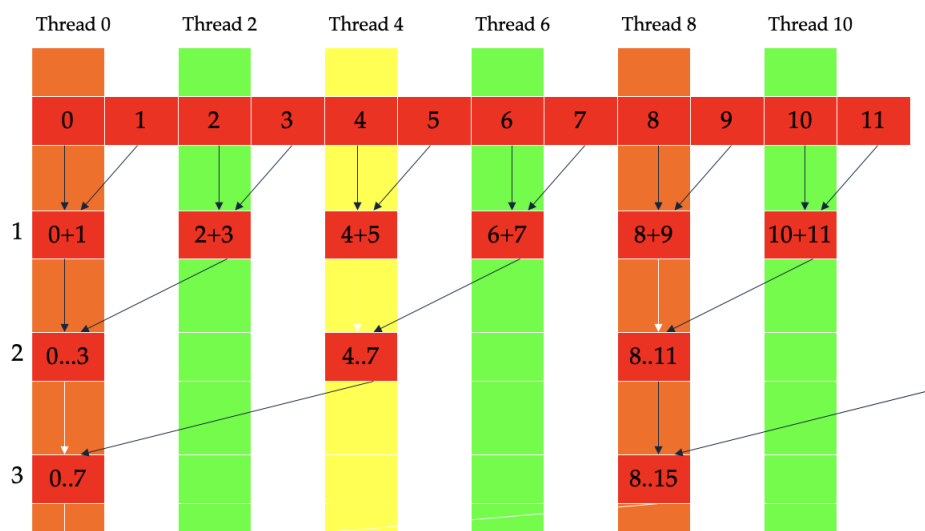
- Alcuni thread del warp devono eseguire il ramo *then*, altri il ramo *else* (o nessun ramo).
- L'hardware serializza l'esecuzione: prima esegue i thread attivi nel ramo *then* (disabilitando gli altri), poi esegue quelli del ramo *else*.

La divergenza riduce drasticamente le prestazioni, poiché in ogni istante una parte delle unità di calcolo rimane inutilizzata.

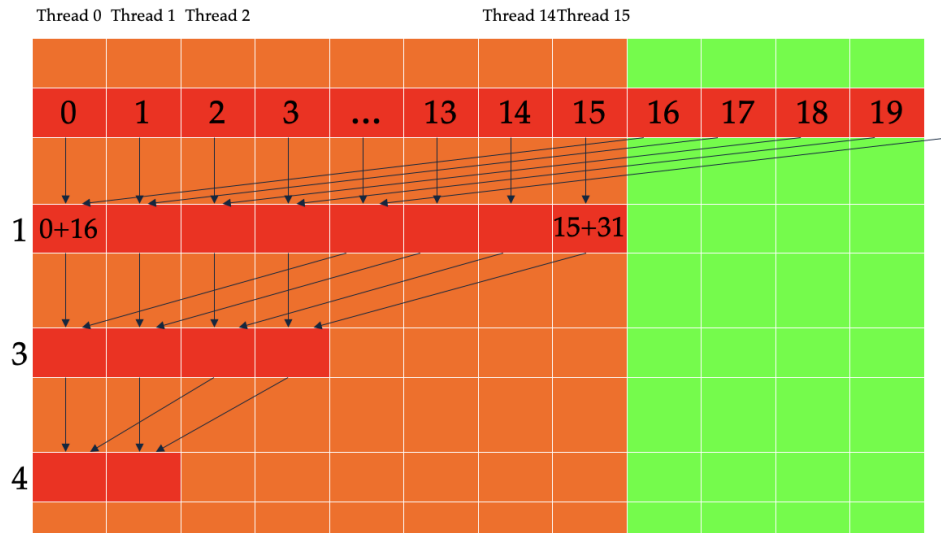


Esempio 10.3 (Sum Reduction e Divergenza). La **Sum Reduction** (sommare tutti gli elementi di un array in un unico valore) è un classico algoritmo parallelo sensibile alla divergenza. Ricordando che il Warp (32 thread) è l'unità minima di esecuzione, confrontiamo due strategie per ridurre 1024 elementi a 512 in un singolo passo (512 somme totali in entrambi i casi):

- **Approccio Interleaved (Inefficiente):** I thread attivi sono alternati (0, 2, 4...).
 - Ogni warp avrà 16 thread attivi e 16 inattivi.
 - Risultato: Tutti i 32 warp vengono attivati, ma lavorano al 50% della capacità. Spreco di risorse hardware.



- **Approccio Index-based (Ottimizzato):** I thread attivi sono contigui (0, 1, ..., 511).
 - I primi 16 warp sono **completamente pieni** (tutti i 32 thread lavorano).
 - Gli ultimi 16 warp sono **completamente vuoti** e non vengono nemmeno schedati.
 - Risultato: Massima efficienza e risorse liberate per altri task.



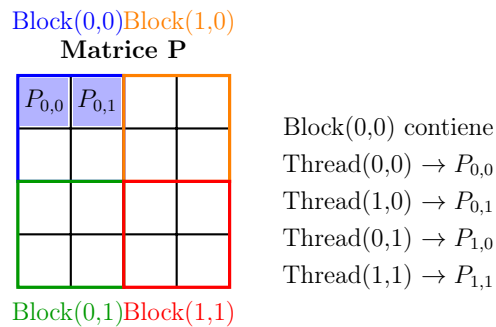
Esempio 10.4 (Moltiplicazione di Matrici in CUDA). La moltiplicazione di matrici $P = M \times N$ è un'operazione fondamentale nel calcolo scientifico e nel Machine Learning. Per matrici quadrate di lato `Width`, ogni elemento $P_{i,j}$ è il prodotto scalare tra la riga i di M e la colonna j di N :

$$P_{i,j} = \sum_{k=0}^{\text{Width}-1} M_{i,k} \cdot N_{k,j}$$

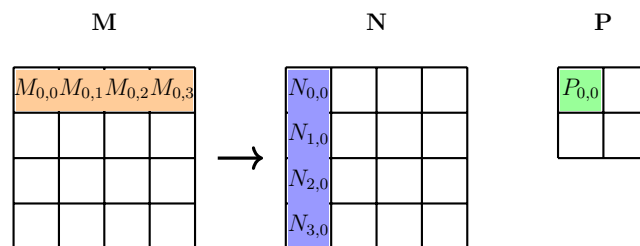
Mappatura Thread $\rightarrow$ Elemento di P In CUDA, assegniamo a **ogni thread** il calcolo di un **singolo elemento** della matrice risultato P . Usando blocchi e thread bidimensionali, le coordinate (Row, Col) dell'elemento da calcolare si ottengono combinando gli indici:

$$Row = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$Col = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$



Calcolo di un Elemento Ogni thread esegue un ciclo `for` per iterare su tutti gli elementi della riga di M e della colonna di N , accumulando il prodotto scalare.



$$P_{0,0} = M_{0,0} \cdot N_{0,0} + M_{0,1} \cdot N_{1,0} + M_{0,2} \cdot N_{2,0} + M_{0,3} \cdot N_{3,0}$$

Codice del Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {
    // Calcola le coordinate (Row, Col) dell'elemento P da calcolare
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    // Controllo limiti (necessario se Width non è multiplo di blockDim)
    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0;
        // Prodotto scalare: riga Row di M * colonna Col di N
        for (int k = 0; k < Width; ++k) {
            Pvalue += M[Row * Width + k] * N[k * Width + Col];
        }
        P[Row * Width + Col] = Pvalue;
    }
}
```

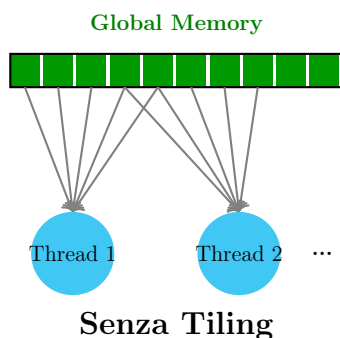
Nota: Ogni thread accede a un'intera riga di M e un'intera colonna di N , causando molti accessi alla Global Memory (lenta).

10.4 Ottimizzazione: Tiling e Shared Memory

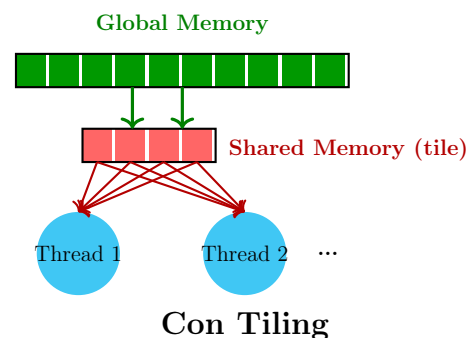
Nel kernel base della moltiplicazione matriciale, ogni thread legge un'intera riga e un'intera colonna dalla Global Memory. Poiché thread diversi condividono molti dati in comune (es. tutti i thread che calcolano elementi sulla stessa riga di P leggono la stessa riga di M), si verificano moltissimi accessi ridondanti alla memoria lenta.

Idea del Tiling La tecnica del **Tiling** (o *Blocking*) risolve questo problema sfruttando la Shared Memory:

1. Si divide la Global Memory in piccole porzioni chiamate **tile**.
2. Ogni blocco di thread carica **cooperativamente** un tile dalla Global Memory alla Shared Memory (veloce, on-chip).
3. I thread lavorano sui dati nella Shared Memory, riutilizzandoli più volte senza doverli rileggere dalla Global Memory.



Ogni thread accede a molte celle della Global Memory (lenta). Accessi ridondanti.

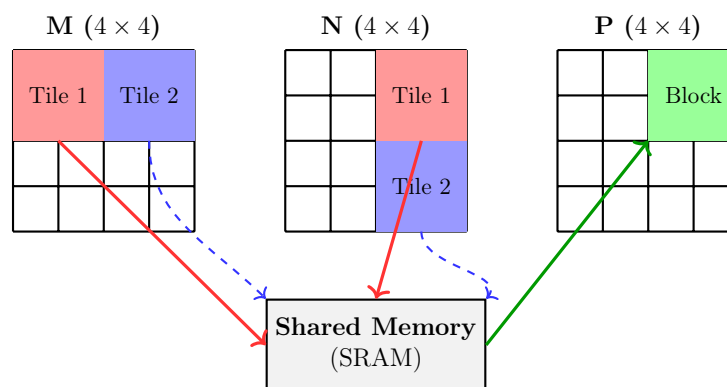


Il tile viene caricato una volta in Shared Memory (veloce) e riutilizzato da tutti i thread.

Dichiarazione di Variabili in Shared Memory Per allocare memoria condivisa all'interno di un kernel, si usa il qualificatore `__shared__`:

```
__global__ void kernel(...) {
    __shared__ float tile[TILE_WIDTH][TILE_WIDTH];
    // 'tile' è condiviso tra tutti i thread del blocco
    ...
}
```

Tiled Matrix Multiplication Nel caso specifico della moltiplicazione matriciale, applicare il tiling significa spezzare l'esecuzione in **fasi**. In ogni fase, un blocco di thread si concentra su un tile (di dimensione `BLOCK_SIZE × BLOCK_SIZE`) estratto da *M* e uno da *N*, invece di accedere all'intera riga e colonna.



Logica Tiling:

- **Fase 1:** Carica M(Sinistra) e N(Alto) → Calcola parziale.
- **Fase 2:** Carica M(Destra) e N(Basso) → Somma al parziale.

Il flusso dell'algoritmo per ogni fase è:

1. **Identificare** il tile di Global Memory che verrà usato da più thread.
2. **Caricare cooperativamente** il tile dalla Global Memory alla Shared Memory.
3. **Barriera** (`__syncthreads()`): attendere che tutti i thread abbiano completato il caricamento.
4. **Calcolare:** ogni thread usa i dati dalla Shared Memory per accumulare il proprio risultato parziale.
5. **Barriera:** attendere che tutti abbiano terminato prima di sovrascrivere la Shared Memory.
6. **Passare al tile successivo** e ripetere.

Barrier Synchronization La funzione `__syncthreads()` è fondamentale per coordinare l'esecuzione parallela all'interno di un blocco. Agisce come una **barriera**:

- Assicura che *tutti* i thread del blocco abbiano raggiunto quel punto del codice prima che *qualcuno* possa proseguire.
- Nell'algoritmo Tiled, è usata per coordinare le fasi:
 1. **Dopo il caricamento (Sync 1):** Garantisce che l'intero tile sia stato caricato in Shared Memory prima che inizi il calcolo (evita di leggere dati non ancora scritti).
 2. **Dopo il calcolo (Sync 2):** Garantisce che tutti i thread abbiano finito di usare i dati del tile corrente prima che vengano sovrascritti dal caricamento della fase successiva.

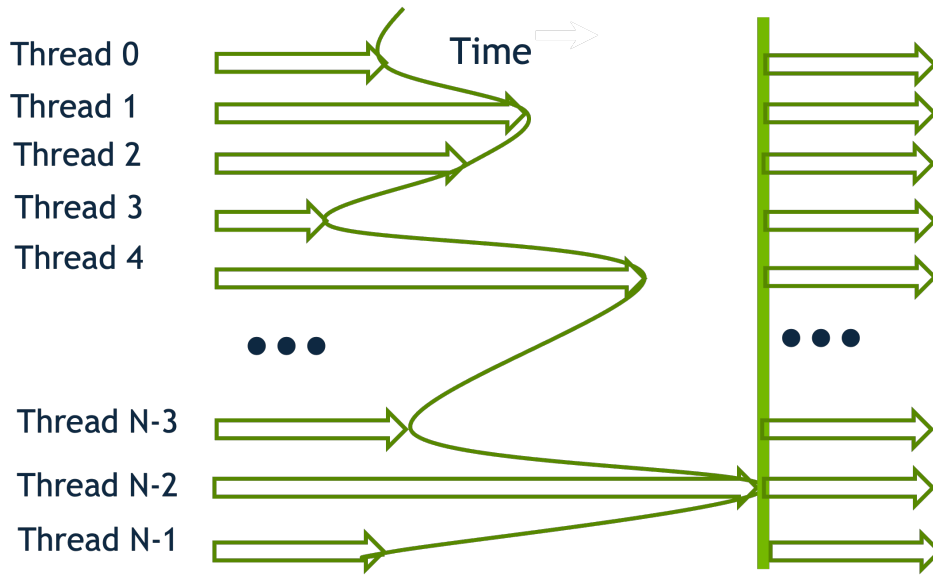
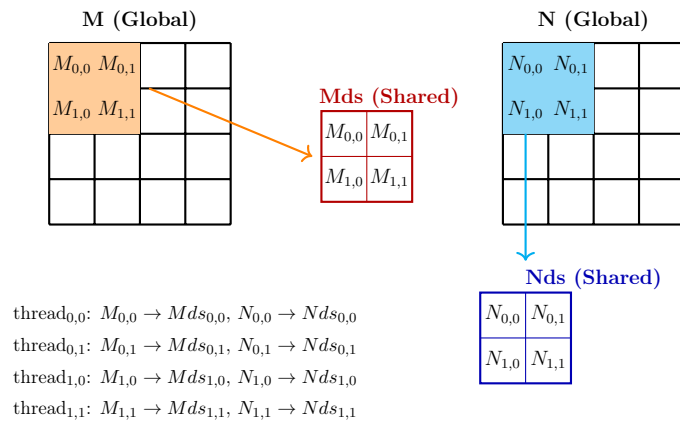


Figura 13: Funzionamento della Barriera di Sincronizzazione

Al termine di tutte le fasi, ogni thread scrive il proprio risultato finale in P . Questo approccio riduce drasticamente gli accessi alla Global Memory: invece di leggere $2 \times \text{Width}$ elementi per thread, si leggono solo $2 \times \text{Width}/\text{BLOCK_SIZE}$ elementi, con un risparmio proporzionale alla dimensione del tile.

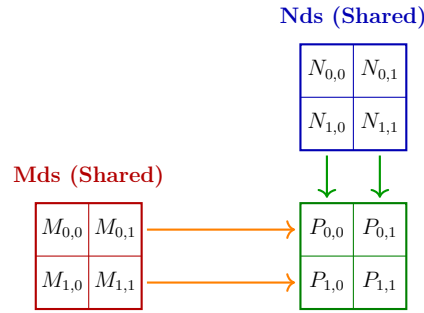
Esempio 10.5 (Esecuzione Dettagliata del Tiled Matrix Multiply). Consideriamo matrici 4×4 con $\text{TILE_WIDTH} = 2$. Il blocco di thread $(0,0)$ calcola il sottoblocco $P_{0:1,0:1}$ (4 elementi). Il calcolo si svolge in 2 fasi.

Iterazione 0 - Step 1: Caricamento Cooperativo Ogni thread del blocco carica **un elemento** di M e **un elemento** di N nella Shared Memory:



`__syncthreads()` $\leftarrow$ Barriera: tutti i thread hanno caricato i dati

Iterazione 0 - Step 2: Calcolo dai Tile Ogni thread calcola il proprio risultato parziale accedendo **solo** dalla Shared Memory:



Ogni thread calcola il prodotto scalare tra una riga di Mds e una colonna di Nds:

$$\begin{aligned}
 P_{0,0} &+= M_{0,0} \cdot N_{0,0} + M_{0,1} \cdot N_{1,0} \\
 P_{0,1} &+= M_{0,0} \cdot N_{0,1} + M_{0,1} \cdot N_{1,1} \\
 P_{1,0} &+= M_{1,0} \cdot N_{0,0} + M_{1,1} \cdot N_{1,0} \\
 P_{1,1} &+= M_{1,0} \cdot N_{0,1} + M_{1,1} \cdot N_{1,1}
 \end{aligned}$$

`__syncthreads()` $\leftarrow$ *Barriera: tutti hanno finito prima di sovrascrivere Mds/Nds*

Fase 1 Si ripete il processo con i tile successivi ($M_{0:1,2:3}$ e $N_{2:3,0:1}$), accumulando i prodotti parziali. Al termine, *PValue* contiene il risultato finale che viene scritto in *P*.

Vantaggio: Ogni elemento caricato in Shared Memory viene usato da **più thread** (es. $M_{0,0}$ è usato sia da `thread0,0` che da `thread0,1`), riducendo gli accessi ridondanti alla Global Memory.

10.5 Considerazioni Avanzate: Occupancy e Shared Memory

L'uso della Shared Memory è fondamentale per le prestazioni, ma è una risorsa limitata (es. 16KB o 48KB per SM). Questa limitazione influenza direttamente il numero massimo di Thread Block che possono risiedere simultaneamente su un SM, e quindi l'*Occupancy* (il grado di utilizzo della GPU).

Consideriamo un SM con **16KB** di Shared Memory:

- Con `TILE_WIDTH = 16`: Ogni blocco usa 2 array (uno per la matrice M e uno per la matrice N) 16×16 di `float` (4 byte).

$$\text{Memoria per blocco} = 2 \times (16 \times 16 \times 4 \text{ Byte}) = 2 \text{ KB}$$

Numero massimo di blocchi attivi per SM: $16 \text{ KB} / 2 \text{ KB} = \mathbf{8 \text{ blocchi}}$.

- Con `TILE_WIDTH = 32`: Ogni blocco usa 2 array 32×32 di `float`.

$$\text{Memoria per blocco} = 2 \times (32 \times 32 \times 4 \text{ Byte}) = 8 \text{ KB}$$

Numero massimo di blocchi per Shared Memory: $16 \text{ KB} / 8 \text{ KB} = \mathbf{2 \text{ blocchi}}$.

Tuttavia, bisogna considerare anche il limite fisico dei **thread per SM** (es. 1536 thread/SM).

- Un blocco 32×32 contiene 1024 thread.
- Possiamo attivare **solo 1 blocco**, perché 2 blocchi richiederebbero 2048 thread (> 1536).

Per cui aumentare la dimensione del tile riduce la bandwidth verso la Global Memory, ma può limitare drasticamente il parallelismo (Occupancy) esaurendo la Shared Memory o i registri/thread disponibili.

10.6 Shared Memory Banks e Conflitti

La Shared Memory è fisicamente suddivisa in moduli chiamati **banchi** (Banks). Nelle GPU moderne (Compute Capability ≥ 2.0) ci sono **32 banchi**.

- **Accesso Parallelo:** Se i thread di un warp accedono a indirizzi che mappano su **banchi diversi**, l'accesso avviene simultaneamente (massima banda).
- **Bank Conflict:** Se più thread di un warp accedono a indirizzi diversi dello **stesso banco**, le richieste vengono **serializzate** (eseguite una dopo l'altra), degradando le prestazioni. Un conflitto a 2 vie raddoppia il tempo di accesso, a 32 vie lo aumenta di 32 volte.
- **Broadcast:** Se tutti i thread leggono lo **stesso identico indirizzo**, non c'è conflitto: il dato viene "trasmesso" a tutti in un solo ciclo (meccanismo di broadcast).

Esempio 10.6 (Pattern di Accesso e Conflitti). Consideriamo 32 banchi (indirizzi mappati modulo 32). Il conflitto avviene **solo** se più thread richiedono **indirizzi diversi** che si trovano nello **stesso banco**. Se richiedono lo **stesso indirizzo**, avviene un broadcast (veloce).

(a) **Stride 2 (Conflict):** `data[2 * ID]` Ogni indirizzo è mappato rigidamente al banco: $\text{Banco} = \text{Indirizzo} \% 32$.

- Thread 0 richiede l'indirizzo **0** $\rightarrow$ Banco 0.
- Thread 16 richiede l'indirizzo **32** $\rightarrow$ Banco 0 (perché $32 \% 32 = 0$).

Due thread accedono allo stesso banco richiedendo **indirizzi diversi** (0 e 32). Il banco non può servirli insieme: deve serializzare le richieste (**conflitto a 2 vie**).

(b) **Multicast (No Conflict):** `data[ID % 4]`

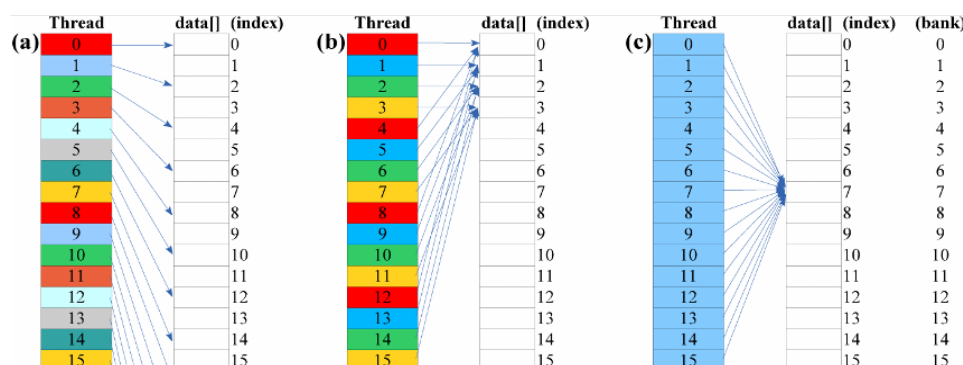
- Thread 0, 4, 8, ... richiedono l'indirizzo **0** $\rightarrow$ Banco 0.
- Thread 1, 5, 9, ... richiedono l'indirizzo **1** $\rightarrow$ Banco 1.

Anche se più thread (es. 0 e 4) accedono allo stesso Banco 0, essi richiedono lo **stesso identico dato** (indirizzo 0). L'hardware ottimizza leggendo il dato una sola volta e inviandolo a tutti i richiedenti (**Multicast**). Nessun conflitto.

(c) **Broadcast (No Conflict):** `data[7]`

- Tutti i 32 thread richiedono l'indirizzo **7** $\rightarrow$ Banco 7.

Tutti convergono sullo stesso banco e sullo stesso indirizzo. Il dato viene letto una volta e trasmesso a tutti in broadcast. Nessun conflitto.



Esempio 10.7 (Kernel Moltiplicazione di Matrici con Tiling). Di seguito il codice completo del kernel CUDA che implementa l'algoritmo Tiled Matrix Multiplication. Si noti l'uso della Shared Memory (`__shared__`) e della doppia barriera di sincronizzazione.

Listing 1: Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    // Allocazione statica della Shared Memory per i tile
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    // Calcolo degli indici del thread e del blocco
    int bx = blockIdx.x; int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // Identificazione della riga e colonna globale dell'elemento P da
    // calcolare
    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop sui tile necessari per calcolare l'elemento P (fasi)
    // p itera sui blocchi della larghezza della matrice
    for (int p = 0; p < Width/TILE_WIDTH; ++p) {

        // — FASE 1: Caricamento Cooperativo —
        // Ogni thread carica un elemento di M e N nella Shared Memory
        // M viene acceduta in base alla riga globale (Row) e alla
        // colonna del tile corrente
        ds_M[ty][tx] = M[Row * Width + (p * TILE_WIDTH + tx)];
        // N viene acceduta in base alla riga del tile corrente e alla
        // colonna globale (Col)
        ds_N[ty][tx] = N[(p * TILE_WIDTH + ty) * Width + Col];

        // Barriera 1: Attendi che tutto il tile sia caricato prima di
        // calcolare
        __syncthreads();

        // — FASE 2: Calcolo parziale dal Tile —
        // Moltiplica la riga (da ds_M) per la colonna (da ds_N)
        for (int i = 0; i < TILE_WIDTH; ++i) {
            Pvalue += ds_M[ty][i] * ds_N[i][tx];
        }

        // Barriera 2: Attendi che tutti abbiano finito di usare i dati
        // prima di sovrascriverli nella prossima iterazione
        __syncthreads();
    }

    // Scrittura del risultato finale in Global Memory
    P[Row * Width + Col] = Pvalue;
}
```

10.7 Accesso alla Global Memory e Coalescing

L'accesso alla Global Memory è l'operazione più costosa in termini di latenza, ma è mitigato dalla presenza di cache (L1 per SM, L2 globale). L'hardware gestisce le transazioni di memoria in blocchi di **32, 64 o 128 Byte** (linee di cache).

Memory Coalescing Quando un warp esegue un'istruzione di accesso alla memoria globale, l'hardware cerca di raggruppare (coalesce) le richieste dei singoli thread in **un'unica transazione** verso la VRAM. Affinché questo avvenga in modo efficiente, devono essere soddisfatte due condizioni:

1. **Località:** I thread del warp devono accedere a indirizzi di memoria **contigui** (o vicini) che rientrano nello stesso blocco allineato.
2. **Allineamento:** L'indirizzo base del blocco richiesto deve essere un multiplo della dimensione del blocco stesso.

Se i thread accedono a dati sparsi o disallineati, l'hardware è costretto a generare **più transazioni** separate per soddisfare un'unica istruzione del warp, riducendo drasticamente la bandwidth effettiva.

Ruolo delle Cache L1 e L2 La granularità dell'accesso dipende da dove si trova il dato:

- **Hit in L1:** Se i dati sono in cache L1 (locale all'SM), l'accesso avviene a blocchi di **128 Byte**.
- **Hit in L2 / Global Memory:** Se i dati sono in L2 (condivisa) o in memoria globale, l'accesso avviene a segmenti di **32 Byte**.

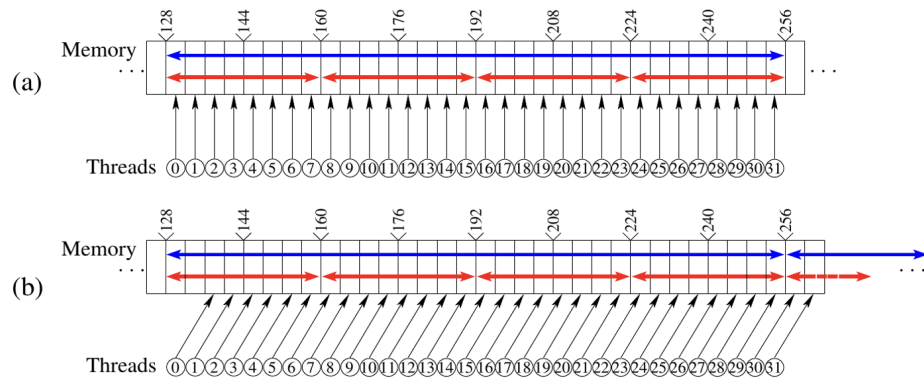
Esempio 10.8 (Efficienza di Accesso: Coalesced vs Uncoalesced). Consideriamo un warp che richiede 32 float (4 Byte ciascuno). Totale dati richiesti: $32 \times 4 = 128$ Byte.

- **Caso (a) - Coalesced:** I thread accedono sequenzialmente (Thread 0 $\rightarrow$ addr X , Thread 1 $\rightarrow$ addr $X + 4$, ...). Tutti gli indirizzi cadono in un unico blocco da 128 Byte allineato.
 - **L1:** 1 transazione da 128 Byte.
 - **L2:** 4 transazioni da 32 Byte.

Efficienza 100%: tutti i byte trasferiti sono utili.

- **Caso (b) - Disallineato/Sparso:** I thread accedono in modo disordinato o con offset (es. shift di qualche byte). Gli indirizzi richiesti **attraversano il confine tra due blocchi di cache consecutivi** (ad esempio, parte dei dati cade nel blocco allineato a 128 Byte e parte nel successivo a 256 Byte).
 - **L1:** Servono **2 transazioni** da 128 Byte (totale 256 Byte trasferiti) per ottenerne solo 128 utili.
 - **L2:** Servono **5 transazioni** da 32 Byte (le 4 centrali + 1 extra per la coda).

La bandwidth viene sprecata trasferendo dati non richiesti.



Case	Data in L1 cache	Data in L2 cache or in global memory
(a)	1 128-byte at addr. 128	4 32-byte at addr. 128, 160, 192 and 224
(b)	2 128-byte at addr. 128 and 256	5 32-byte at addr. 128, 160, 192, 224 and 256

10.8 Trasferimento Dati Host-Device e Asincronia

Uno dei principali colli di bottiglia nel calcolo eterogeneo (CPU + GPU) è il trasferimento dati attraverso il bus **PCIe**. Come mostrato in tabella, la bandwidth del bus PCIe (es. 16 GB/s per Gen3) è molto inferiore rispetto a quella della memoria interna (DDR3/4 o GDDR della GPU).

Tecnologia	Bandwidth di Picco
PCIe Gen2	8 GB/s
PCIe Gen3	16 GB/s
DDR3 (Host)	26 GB/s (single channel)

Serializzazione vs Asincronia Normalmente, l'uso di `cudaMemcpy` blocca la CPU finché il trasferimento non è completo (**sincrono**). Questo porta a una esecuzione serializzata inefficiente:

H2D (Copy) → Kernel (Compute) → D2H (Copy)

In questo scenario, mentre avviene la copia, la GPU (o la CPU) rimane inattiva (idle).

Device Overlap e CUDA Streams Le GPU moderne supportano il **Device Overlap**, ovvero la capacità di eseguire contemporaneamente un kernel e una copia di memoria (o copie multiple in direzioni opposte). Per sfruttarlo, si divide il lavoro in "flussi" indipendenti (**CUDA Streams**) e si usano versioni asincrone delle API:

- `cudaMemcpyAsync(...)`: Ritorna immediatamente il controllo alla CPU, accodando l'operazione.
- `cudaDeviceSynchronize()`: Forza la CPU ad attendere che la GPU abbia finito tutto il lavoro in coda.

Dividendo vettori di grandi dimensioni in segmenti più piccoli e usando gli stream, è possibile sovrapporre (overlap) il trasferimento del segmento $i + 1$ con il calcolo del segmento i , nascondendo quasi completamente la latenza del bus PCIe (pipeline software).

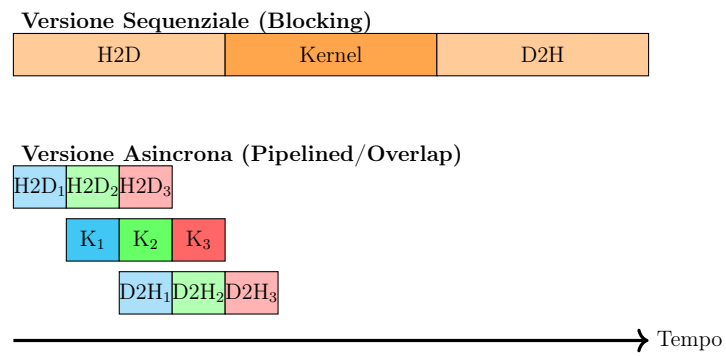


Figura 14: Confronto visuale: Esecuzione Sequenziale vs Overlap con Streams